학습 교안

FastAPI 웹 API 통합 교안

웹을 처음 배우는 학생을 위한 — 첫 API 서버부터 AI 모델 서빙까지

차례

0장 웹과 API 첫걸음 (새로 추가)

1장 FastAPI 시작하기

2장 Pydantic 으로 요청과 응답 다루기

3장 CRUD API 만들기

4장 의존성 · 비동기 · 수명주기 · 미들웨어

5장 파일 업로드와 이미지 처리

6장 머신러닝 모델 서빙

7장 프로젝트 구조 · 테스트 · 배포

부록 용어 사전 · 명령어 모음 · 오류 해결 모음

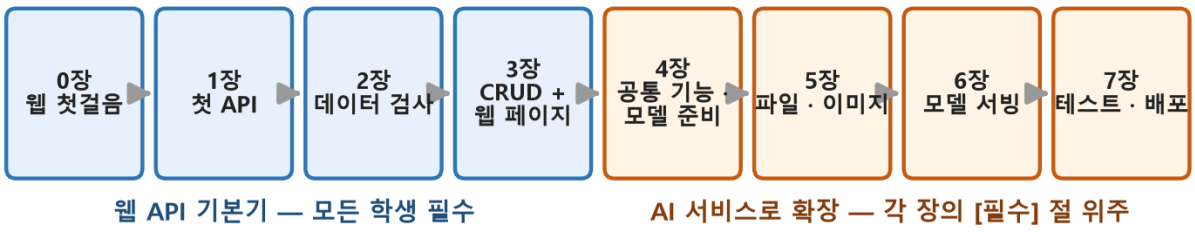
예제 코드 : github.com/sinaboro/ai-service-course → fastapi 폴더

Python 3.12 · FastAPI 0.162 · Pydantic 2.14 기준, 모든 실행 결과는 실제 출력

강사 김대철

이 교안을 쓰는 방법

이 교안은 **웹을 처음 배우는 학생**이 FastAPI 로 웹 API 서버를 만들고, 마지막에는 학습한 AI 모델을 API 로 서비스(모델 서빙)하는 데까지 가도록 만든 통합 교안이에요. 장별 Word 교안 7개(1~7장)를 한 권으로 합치고, 웹 초보를 위한 **0장(웹과 API 첫걸음)** 과 화면 그림, 3장 웹 페이지 실습, 부록을 더했어요.



각 절 제목 옆의 [필수] 는 꼭 하고, [심화] 는 시간이 있을 때 또는 2회차에 공부해요.

학습 지도 — 0~3장은 웹 API 기본기, 4~7장은 AI 서비스로 확장

표시 읽는 법

표시	뜻
[필수]	기본 과정에서 꼭 하는 절. 절 제목 끝에 붙어 있어요
[심화]	시간이 있을 때, 또는 기본 과정을 마친 뒤 2회차에 하는 절. 건너뛰어도 다음 장을 따라갈 수 있어요
웹 초보 안내 (보라색 상자)	각 장 첫머리 — 이 장을 한 문장으로 요약하고, 필수/심화 절을 알려 줘요
직접 해 보기 (하늘색 상자)	서버를 켜고 브라우저로 따라 하는 순서
코드 상자의 줄 번호	아래 "한 줄씩 읽기" 표의 줄 번호와 짝이 맞아요
▶ 실행 결과	교안의 실행 결과는 모두 실제로 실행한 출력이에요 (시각·시간 값은 실행할 때마다 달라요)

진행표 (예시)

회차	내용	시간	과정
1	0장 웹과 API 첫걸음 — 환경 준비, 첫 서버	3시간	기본
2	1장 FastAPI 시작하기 — 경로·쿼리, `/docs`	3시간	기본
3	2장 Pydantic 1~4절 — POST 본문, 자동 검사	3시간	기본
4	3장 CRUD 1~4절 + 6절 웹 페이지	4시간	기본
5	4장 1·2·4절 — Depends, API 키, lifespan	2시간	기본
6	5장 1~4절 — 파일 업로드와 이미지	3시간	기본
7	6장 1~5절 — 모델 서빙 `/predict`	3시간	기본
8	7장 1~5절 — 프로젝트 구조, pytest	3시간	기본
+	각 장의 [심화] 절 (async, 미들웨어·CORS, 이미지 전처리, 이미지 분류, Docker …)	8~10시간	심화

알아 두기 | 준비물

- Python 3.10 이상 (권장 3.12), VS Code, 웹 브라우저(Chrome·Edge)
- 저장소 : `github.com/sinaboro/ai-service-course` 의 `fastapi` 폴더 — 각 장 폴더에 예제 코드와 연습문제 정답이 있어요
- 설치 명령은 0장 6절에 있어요. 설치 후 `00_웹과_API_첫걸음/check_setup.py` 로 점검해요
- 파이썬 기초(함수·딕셔너리·리스트)를 알고 있다고 가정해요. FastAPI 에 필요한 문법은 0장 5절에서 다시 짚어요

도움말 | 예제를 실행하는 두 가지 방법

- **서버로 실행** : `ex01_hello.py` 같은 앱 파일은 `uvicorn ex01_hello:app --reload` 로 켜고 브라우저·`/docs` 로 확인해요.
- **파이썬으로 실행** : `ex01_hello_client.py` 같은 `_client.py` 파일과 연습문제 정답은 `python 파일.py` 로 실행해요. 서버를 켜지 않아도 결과가 나와요.
- 어느 쪽이든 **그 예제가 있는 폴더로** `cd` **한 뒤** 실행해야 해요.

목차

이 교안을 쓰는 방법 . . . 2

0장 웹과 API 첫걸음 . . . 7

1. 웹은 어떻게 움직일까? [필수] . . . 7
2. URL(주소) 읽는 법 [필수] . . . 8
3. HTTP 메서드와 상태 코드 [필수] . . . 9
4. JSON 과 파이썬 딕셔너리 [필수] . . . 9
5. FastAPI 코드를 읽기 위한 파이썬 문법 [필수] . . . 10
6. 실습 환경 준비 — 폴더 · 가상환경 · 설치 [필수] . . . 12
7. 첫 서버 켜고 브라우저로 접속하기 [필수] . . . 13
8. 자주 막히는 곳과 정리 . . . 15

연습문제 . . . 15

연습문제 정답 및 해설 . . . 16

1장 FastAPI 시작하기 . . . 17

1. 왜 FastAPI 를 배우나요? — 모델 서빙 [필수] . . . 18
2. API · HTTP · JSON [필수] . . . 18
3. 설치와 첫 API [필수] . . . 19
4. 서버 실행과 자동 문서 [필수] . . . 19
5. TestClient 로 요청 보내기 [필수] . . . 22
6. 경로 매개변수와 쿼리 매개변수 [필수] . . . 23
7. 자주 하는 실수와 정리 . . . 26

연습문제 . . . 27

연습문제 정답 및 해설 . . . 28

2장 Pydantic 으로 요청과 응답 다루기 . . . 30

1. Pydantic 이란? [필수] . . . 30
2. 요청 본문(body) 받기 [필수] . . . 31
3. Field 로 범위 정하기 [필수] . . . 35
4. 중첩 모델과 리스트 [필수] . . . 38
5. 나만의 검사 규칙 [심화] . . . 41
6. 자주 하는 실수와 정리 . . . 44

연습문제 . . . 45

연습문제 정답 및 해설 . . . 46

3장 CRUD API 만들기 . . . 48

1. CRUD 와 HTTP 메서드 [필수] . . . 49
2. 할 일 API 만들기 [필수] . . . 49

3. HTTPException — 오류 응답 [필수] . . . 53
4. PUT 과 PATCH [필수] . . . 54
5. APIRouter 로 파일 나누기 [심화] . . . 54
6. 웹 페이지에서 할 일 API 쓰기 [필수] . . . 57
7. 자주 하는 실수와 정리 . . . 61
연습문제 . . . 62
연습문제 정답 및 해설 . . . 63

4장 의존성 · 비동기 · 수명주기 · 미들웨어 . . . 65

1. Depends — 의존성 주입 [필수] . . . 66
2. API 키 인증 [필수] . . . 68
3. async def 와 await [심화] . . . 70
4. lifespan — 시작할 때 모델 불러오기 [필수] . . . 72
5. 미들웨어와 CORS · 6. 설정과 환경 변수 [심화] . . . 74
7. 정리 . . . 76
연습문제 . . . 76
연습문제 정답 및 해설 . . . 78

5장 파일 업로드와 이미지 처리 . . . 80

1. 왜 파일 업로드가 필요할까? [필수] . . . 81
2. 실습 이미지 만들기 [필수] . . . 81
3. UploadFile 기본 [필수] . . . 82
4. 이미지 검사와 처리 [필수] . . . 84
5. 딥러닝 입력으로 전처리 [심화] . . . 88
6. 자주 하는 실수와 정리 . . . 91
연습문제 . . . 91
연습문제 정답 및 해설 . . . 92

6장 머신러닝 모델 서빙 . . . 93

1. 모델 서빙의 전체 흐름 [필수] . . . 94
2. 모델 학습하고 저장하기 [필수] . . . 94
3. 모델 클래스 만들기 [필수] . . . 96
4. 예측 API 서버 [필수] . . . 97
5. 실제 서버로 예측 요청하기 [필수] . . . 102
6. 모델이 없을 때 [심화] . . . 103
7. 이미지 분류 API [심화] . . . 104
8. 딥러닝 모델로 바꿔 끼우기 (참고 코드) [심화] . . . 106
9. 정리 . . . 107
연습문제 . . . 107
연습문제 정답 및 해설 . . . 108

7장 프로젝트 구조 · 테스트 · 배포 110

1. 실무형 폴더 구조 [필수] 111
2. 설정 · 스키마 · 모델 [필수] 111
3. 라우터와 메인 앱 [필수] 114
4. pytest 로 자동 테스트 [필수] 116
5. 실행과 패키지 정리 [필수] 119
6. Docker 로 배포 (참고) [심화] 120
7. 다음 단계와 정리 121

연습문제 122

연습문제 정답 및 해설 123

부록 124

부록 A. 용어 사전 124

부록 B. 명령어와 코드 모음 125

부록 C. 오류 해결 모음 125

00장

웹과 API 첫걸음

웹이 움직이는 원리를 알고, 실습 환경을 준비해 첫 서버를 켜 봅니다

항목	내용
예제 폴더	`fastapi/00_웹과_API_첫걸음`
예제 파일	`ex01_json_basics.py`, `ex02_python_warmup.py`, `ex03_first_server.py` + `_client.py`, `check_setup.py`
준비	Python 3.10 이상, VS Code, 웹 브라우저
예상 학습 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

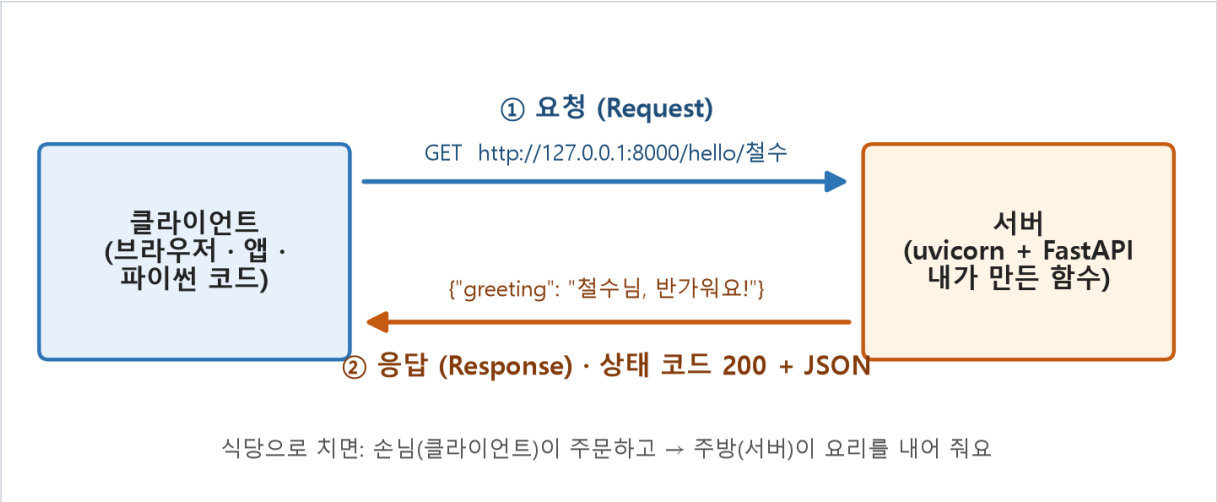
- 클라이언트 · 서버 · 요청 · 응답 · API 가 무엇인지 그림으로 설명할 수 있어요.
- URL 을 호스트 · 포트 · 경로 · 쿼리로 나눠 읽을 수 있어요.
- HTTP 메서드(GET·POST…)와 상태 코드(200·404·422·500)의 뜻을 알아요.
- JSON 과 파이썬 딕셔너리를 서로 바꿀 수 있어요.
- 가상환경을 만들고 패키지를 설치해 실습 환경을 점검할 수 있어요.
- `uvicorn` 으로 서버를 켜고, 브라우저로 접속하고, `Ctrl+C` 로 끌 수 있어요.

도움말 | 이번 장의 구성

- 1. 웹은 어떻게 움직일까? **[필수]**
- 2. URL(주소) 읽는 법 **[필수]**
- 3. HTTP 메서드와 상태 코드 **[필수]**
- 4. JSON 과 파이썬 딕셔너리 **[필수]**
- 5. FastAPI 코드를 읽기 위한 파이썬 문법 **[필수]**
- 6. 실습 환경 준비 — 폴더 · 가상환경 · 설치 **[필수]**
- 7. 첫 서버 켜고 브라우저로 접속하기 **[필수]**
- 8. 자주 막히는 곳과 정리
- 연습문제 · 정답 및 해설

1. 웹은 어떻게 움직일까? 필수

브라우저 주소창에 `naver.com` 을 입력하고 Enter 를 누르면 무슨 일이 일어날까요? 내 컴퓨터의 브라우저가 멀리 있는 **서버** 컴퓨터에 "첫 화면 주세요" 라고 **요청(request)** 을 보내고, 서버는 화면에 쓸 내용을 **응답(response)** 으로 돌려줘요. 웹의 모든 일은 이 **요청 → 응답** 한 번의 반복이에요.



요청과 응답 — 클라이언트가 요청하고 서버가 응답해요

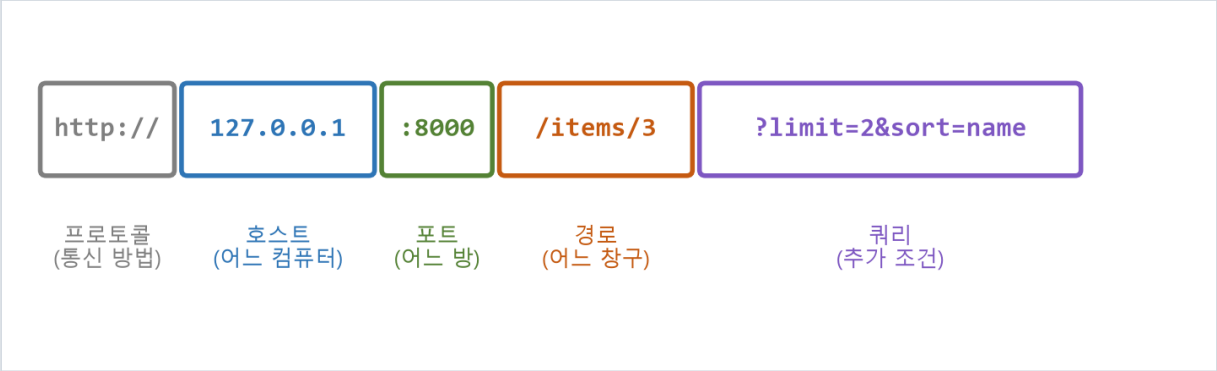
말	뜻	이 과정에서는
클라이언트	요청을 보내는 쪽	브라우저, `/docs` 화면, 파이썬 `TestClient`
서버	요청을 받아 처리하고 응답하는 프로그램	`uvicorn` 으로 켜는 FastAPI 앱
API	프로그램끼리 정해진 방식으로 데이터를 주고받는 창구	우리가 만드는 주소(`/hello`, `/predict` …)
JSON	데이터를 주고받는 글자 형식	`{"name": "철수", "age": 20}`

비유로 이해하기 | 웹 페이지와 웹 API 는 무엇이 다를까?

웹 페이지는 사람이 보는 **화면(HTML)** 을 돌려주고, 웹 API 는 프로그램이 쓰기 좋은 **데이터(JSON)** 를 돌려줘요. 날씨 앱은 날씨 API 에서 JSON 을 받아 예쁜 화면으로 그려 줘요. 이 과정에서 우리는 주로 **API(JSON 을 돌려주는 서버)** 를 만들고, 3장에서는 그 API 를 쓰는 웹 페이지도 붙여 봐요.

2. URL(주소) 읽는 법 필수

API 를 쓸 때는 주소를 정확히 읽을 줄 알아야 해요. 주소 하나는 다섯 부분으로 나뉘어요.



URL 은 프로토콜 · 호스트 · 포트 · 경로 · 쿼리로 나뉘어요

부분	예	뜻
호스트	`127.0.0.1` (= `localhost`)	**내 컴퓨터 자신**을 가리키는 특별한 주소. 실습 서버는 모두 내 컴퓨터에서 돌아요
포트	`:8000`	한 컴퓨터 안의 "방 번호". 서버 프로그램마다 다른 방을 써요 (`uvicorn` 기본값 8000)
경로	`/items/3`	어느 창구(API)에 요청하는지. FastAPI 의 `@app.get("/items/{item_id}")` 와 짝이 맞아요

부분	예	뜻
쿼리	`?limit=2&sort=name`	추가 조건. `?` 뒤에 `이름=값` 을 `&` 로 이어 붙여요

> **도움말 | 한글 주소가 이상한 글자로 바뀌어요**
> 주소에 한글을 넣으면 `/hello/%ED%99%8D...` 처럼 바뀌어 보낼 때가 있어요. 주소에는 영문·숫자만 안전하게 쓸 수 있어서 한글을 이렇게 바꿔(인코딩) 보내는 것이고, 서버는 다시 한글로 되돌려 받아요. 고장이 아니에요.

3. HTTP 메서드와 상태 코드 필수

요청에는 "무엇을 하고 싶은지" 를 나타내는 **메서드** 가 붙고, 응답에는 "어떻게 됐는지" 를 나타내는 **상태 코드** 숫자가 붙어요. 지금은 아래 표만 눈에 익혀 두면 돼요. 1~3장에서 하나씩 직접 써 봐요.

메서드	뜻	예
GET	가져오기(조회)	글 목록 보기 — 브라우저 주소창에 입력하면 항상 GET
POST	보내서 만들기 · 처리 요청	회원 가입, 예측 요청
PUT / PATCH	고치기 (전체 / 일부)	내 정보 수정
DELETE	지우기	글 삭제

상태 코드	뜻	언제 보게 되나요?
200 OK	성공	정상 응답
201 Created	만들기 성공	POST 로 새 데이터를 만들었을 때 (3장)
404 Not Found	그런 주소·데이터가 없어요	주소 오타, 없는 번호 조회
422 Unprocessable	보낸 값의 형식이 틀렸어요	숫자 자리에 글자를 보냈을 때 (1~2장)
500 Internal Error	서버 코드에서 오류가 났어요	내 함수 안의 버그 → 서버 터미널의 오류 메시지 확인

> **알아 두기 | 첫 자리만 보면 돼요**
> **2xx** 는 성공, **4xx** 는 "요청한 쪽(클라이언트)이 뭔가 잘못 보냈어요", **5xx** 는 "서버 쪽 코드에 문제가 있어요" 예요. 오류가 나면 첫 자리부터 보고 어디를 고칠지 정해요.

4. JSON 과 파이썬 딕셔너리 필수

서버와 클라이언트는 네트워크로 **글자(문자열)** 만 주고받을 수 있어요. 그래서 파이썬 딕셔너리를 **JSON 이라는 글자 형식**으로 바꿔 보내고, 받은 쪽은 다시 딕셔너리로 바꿔 써요. FastAPI 는 이 변환을 자동으로 해 주지만, 무슨 일이 일어나는지 한 번 직접 해 봐요.

예제 파일 `fastapi/00_웹과_API_첫걸음/ex01_json_basics.py`

ex01_json_basics.py

```python
# JSON 과 파이썬 딕셔너리 — 웹에서 데이터를 주고받는 글자 형식 익히기 (python 으로 실행)
# 서버는 파이썬 딕셔너리를 JSON 글자로 바꿔 보내고, 받는 쪽은 그 글자를 다시 딕셔너리로 바꿔요.
import json

# 1) 파이썬 딕셔너리 (내 프로그램 안에서 쓰는 값)
student = {"name": "홍길동", "age": 20, "scores": [90, 85], "is_member": True, "memo": None}
print("딕셔너리:", student)
print("타입:", type(student))

# 2) 딕셔너리 → JSON 글자 (서버가 응답을 보낼 때 하는 일)
#    ensure_ascii=False: 한글을 홍 같은 기호 대신 글자 그대로
text = json.dumps(student, ensure_ascii=False)
```

ex01_json_basics.py

```
print("\nJSON 글자:", text)
print("타입:", type(text))                    # str — 네트워크로는 글자만 보낼 수 있어요

# 파이썬과 JSON 표기 차이: True → true, None → null, 작은따옴표 → 큰따옴표

# 3) JSON 글자 → 딕셔너리 (응답을 받은 쪽이 하는 일)
back = json.loads(text)
print("\n다시 딕셔너리:", back)
print("이름만 꺼내기:", back["name"])
print("첫 번째 점수:", back["scores"][0])

# 4) 보기 좋게 출력하기 (indent=2: 두 칸 들여쓰기) — 이 과정의 _client.py 파일들이 쓰는 방법
print("\n보기 좋게:")
print(json.dumps(student, ensure_ascii=False, indent=2))
```

▶ 실행 결과

```
딕셔너리: {'name': '홍길동', 'age': 20, 'scores': [90, 85], 'is_member': True, 'memo': None}
타입: <class 'dict'>

JSON 글자: {"name": "홍길동", "age": 20, "scores": [90, 85], "is_member": true, "memo": null}
타입: <class 'str'>

다시 딕셔너리: {'name': '홍길동', 'age': 20, 'scores': [90, 85], 'is_member': True, 'memo': None}
이름만 꺼내기: 홍길동
첫 번째 점수: 90

보기 좋게:
{
  "name": "홍길동",
  "age": 20,
  "scores": [
    90,
    85
  ],
  "is_member": true,
  "memo": null
}
```

파이썬	JSON	주의
`True` / `False`	`true` / `false`	소문자
`None`	`null`	
`'글자'` 또는 `"글자"`	`"글자"`	JSON 은 큰따옴표만
`{"키": 값}`	`{"키": 값}`	키는 항상 글자

5. FastAPI 코드를 읽기 위한 파이썬 문법 필수

FastAPI 예제에는 **타입 힌트, 기본값,** `str | None`**, 데코레이터(@), import** 다섯 가지가 계속 나와요. 처음 보는 문법이 있다면 아래 예제를 실행하며 익혀 두세요. 특히 **데코레이터**는 "함수 위에 붙이는 등록 스티커" 라고만 이해해도 충분해요.

예제 파일 `fastapi/00_웹과_API_첫걸음/ex02_python_warmup.py`

ex02_python_warmup.py

```python
# FastAPI 코드를 읽기 위한 파이썬 문법 미리보기 (python 으로 실행)
# FastAPI 예제에는 아래 다섯 가지가 계속 나와요. 여기서 먼저 익숙해져요.

# ── 1) 타입 힌트: "이 변수에는 이런 값이 들어와요" 라는 표시 ──
# FastAPI 는 이 표시를 보고 들어온 값을 자동으로 바꾸고(변환) 검사해요.
def add(a: int, b: int) -> int:                # a, b 는 정수, 돌려주는 값도 정수
    return a + b

print("1) add(3, 4) =", add(3, 4))

# ── 2) 기본값이 있는 매개변수: 안 보내면 기본값을 써요 ──
# FastAPI 에서는 "기본값이 있으면 선택, 없으면 필수" 쿼리가 돼요.
def greet(name: str, lang: str = "ko") -> str:
    if lang == "en":
        return f"Hello, {name}!"
    return f"{name}님, 안녕하세요!"            # f-문자열: {} 안의 변수 값이 들어가요

print("2)", greet("철수"), "/", greet("Tom", lang="en"))

# ── 3) None 이 될 수 있는 값: str | None ──
# "글자이거나, 아예 없을 수도 있어요" 라는 뜻 (Python 3.10 이상 문법)
def search(keyword: str | None = None) -> str:
    if keyword is None:
        return "전체 목록"
    return f"'{keyword}' 검색 결과"

print("3)", search(), "/", search("마우스"))

# ── 4) 데코레이터 @: 함수 위에 붙이는 "등록 스티커" ──
# FastAPI 의 @app.get("/주소") 는 "이 주소로 요청이 오면 아래 함수를 실행해 줘" 라고 등록하는 스티커예요.
# 아래는 그 원리를 아주 작게 흉내 낸 것이에요. (내부 동작은 몰라도 사용하는 데 문제없어요)
ROUTES = {}                                    # 주소 → 함수 를 적어 두는 장부


def route(path):                               # route("/hello") 가 스티커를 만들어 줘요
    def sticker(func):
        ROUTES[path] = func                    # 장부에 등록
        return func
    return sticker


@route("/hello")                               # ← FastAPI 의 @app.get("/hello") 와 같은 모양
def hello():
    return {"message": "안녕하세요"}


@route("/health")
def health():
    return {"status": "ok"}


print("4) 등록된 주소:", list(ROUTES))
print("   '/hello' 로 요청이 왔다고 치면 →", ROUTES["/hello"]())

# ── 5) import: 다른 파일(모듈)의 것을 가져와 쓰기 ──
# from fastapi import FastAPI  ← "fastapi 라는 설치된 묶음에서 FastAPI 를 가져와요"
from datetime import date
print("5) date 를 가져와서 쓰기:", date(2026, 3, 2).strftime("%Y년 %m월 %d일"))   # 설치 없이 쓰는 표준 모듈
```

▶ 실행 결과

```
1) add(3, 4) = 7
2) 철수님, 안녕하세요! / Hello, Tom!
3) 전체 목록 / '마우스' 검색 결과
4) 등록된 주소: ['/hello', '/health']
   '/hello' 로 요청이 왔다고 치면 → {'message': '안녕하세요'}
5) date 를 가져와서 쓰기: 2026년 03월 02일
```

도움말 | 문법을 다 몰라도 괜찮아요

`route` 함수 안쪽(함수 안의 함수)은 FastAPI 가 대신 해 주는 부분이라 외울 필요가 없어요. **"@app.get("/주소") 아래 함수 = 그 주소로 요청이 오면 실행되는 함수"** 한 문장만 기억하세요.

6. 실습 환경 준비 — 폴더 · 가상환경 · 설치 필수

① 파이썬과 편집기 확인

파이썬 3.10 이상(권장 3.12)과 VS Code 를 설치해요. VS Code 에서 **[파일] → [폴더 열기]** 로 저장소의 `fastapi` 폴더를 열고, **[터미널] → [새 터미널]** 을 누르면 아래쪽에 명령을 입력하는 창(터미널)이 열려요.

터미널: 파이썬 버전 확인

```
python --version
```

② 가상환경 만들기와 켜기

가상환경은 **이 과정 전용 패키지 상자**예요. 다른 과목의 패키지와 섞이지 않게 해 줘요. 처음 한 번만 만들고, 터미널을 새로 열 때마다 **켜기(activate)** 만 하면 돼요. 켜지면 줄 맨 앞에 `(.venv)` 가 보여요.

터미널: 가상환경 만들기 (처음 한 번)

```
python -m venv .venv
```

운영체제	켜는 명령 (터미널을 열 때마다)
Windows (PowerShell)	`.venv\Scripts\Activate.ps1`
Windows (명령 프롬프트)	`.venv\Scripts\activate.bat`
macOS / Linux	`source .venv/bin/activate`
끄기 (공통)	`deactivate`

주의 | PowerShell 에서 "스크립트를 실행할 수 없으므로..." 오류가 나면

Windows 의 보안 설정 때문이에요. 한 번만 `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` 를 입력하고 `Y` 를 누른 뒤 다시 켜 보세요. 또는 VS Code 터미널 오른쪽 위의 ∨ 를 눌러 **Command Prompt** 로 바꾸고 `activate.bat` 을 써도 돼요.

③ 패키지 설치와 점검

터미널: 이 과정 전체 패키지 설치 (가상환경을 켠 상태에서)

```
pip install fastapi uvicorn python-multipart pillow numpy httpx2 pytest
```

설치가 끝나면 점검 프로그램으로 확인해요. `[X]` 가 있으면 안내대로 해결하고 다시 실행해요.

예제 파일 `fastapi/00_웹과_API_첫걸음/check_setup.py`

▶ 터미널: cd 00_웹과_API_첫걸음 → python check_setup.py

```
[OK]  파이썬 3.12.0
[OK]  가상환경 사용 중
[OK]  fastapi           0.162.0    (1장~)
[OK]  uvicorn           0.54.0     (1장~ 서버 실행)
[OK]  httpx2            2.16.0     (1장~ TestClient)
[OK]  python-multipart  0.0.33     (5장 파일 업로드)
[OK]  pillow            13.0.0     (5~6장 이미지)
[OK]  numpy             2.5.4      (5~6장 배열·모델)
[OK]  pytest            9.2.0      (7장 테스트)
[OK]  8000번 포트 비어 있음

준비 완료! 1장으로 가세요.
```

터미널 명령	뜻
`cd 폴더이름`	그 폴더로 들어가기 (예제는 반드시 **그 예제가 있는 폴더**에서 실행해요)
`cd ..`	한 단계 위 폴더로
`dir` (Windows) / `ls` (macOS)	지금 폴더의 파일 목록 보기
`python 파일.py`	파이썬 파일 실행 (`_client.py`, 연습문제 등)
`uvicorn 파일:app --reload`	서버 켜기 (`app.py` 가 아니라 `파일이름:app`)
`Ctrl + C`	실행 중인 서버 끄기
↑ 화살표	방금 입력한 명령 다시 불러오기

7. 첫 서버 켜고 브라우저로 접속하기 필수

이제 진짜 웹 서버를 켜 볼 거예요. 코드는 1장에서 한 줄씩 자세히 배우니, 지금은 **켜고 → 접속하고 → 끄는** 흐름만 익히세요.

예제 파일 `fastapi/00_웹과_API_첫걸음/ex03_first_server.py`

ex03_first_server.py

```python
# 가장 작은 웹 서버 – 브라우저로 접속해 보기 (서버로 실행)
# 실행: 이 폴더에서  uvicorn ex03_first_server:app --reload
# 확인: 브라우저 주소창에  http://127.0.0.1:8000  과  http://127.0.0.1:8000/time  입력
# 끄기: 서버를 켠 터미널에서 Ctrl + C
from datetime import datetime

from fastapi import FastAPI

app = FastAPI()                                  # 서버(앱) 하나 만들기


@app.get("/")                                    # 주소 "/" 로 들어오면
def home():
    return {"message": "제 첫 웹 서버예요!"}     # 이 딕셔너리가 JSON 으로 브라우저에 보여요

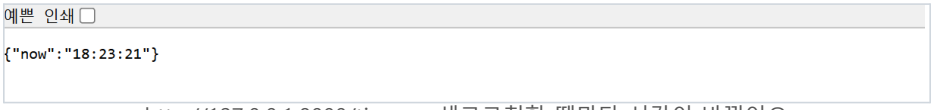

@app.get("/time")                                # 주소 "/time" 으로 들어오면
def now():
    # 요청이 올 때마다 함수가 다시 실행되므로 새로고침할 때마다 시각이 바뀌어요
    return {"now": datetime.now().strftime("%H:%M:%S")}
```

직접 해 보기 | 터미널 두 개로 따라 하기

- **터미널 1** 에서 `cd 00_웹과_API_첫걸음` → `uvicorn ex03_first_server:app --reload` 를 입력해요. `Uvicorn running on http://127.0.0.1:8000` 이 보이면 서버가 켜진 거예요. 이 터미널은 서버가 쓰고 있으니 **그대로 두세요.**
- 브라우저 주소창에 `http://127.0.0.1:8000` 을 입력해요. 아래 그림처럼 JSON 이 보이면 성공이에요.
- 주소를 `http://127.0.0.1:8000/time` 으로 바꿔 보고, 새로고침(F5)을 여러 번 눌러 시각이 바뀌는지 확인해요. 요청이 올 때마다 함수가 다시 실행되기 때문이에요.
- 터미널 1 을 보면 요청이 올 때마다 `"GET / HTTP/1.1" 200 OK` 같은 줄이 찍혀요. 서버가 받은 요청 기록이에요.
- 코드의 메시지를 바꾸고 저장(Ctrl+S)한 뒤 새로고침해 보세요. `--reload` 덕분에 서버가 저절로 다시 켜져서 바뀐 내용이 보여요.
- 다 했으면 터미널 1 에서 `Ctrl + C` 로 서버를 꺼요. 다른 명령을 입력하고 싶으면 VS Code 터미널 오른쪽 위의 **+** 로 **터미널 2** 를 열어요.



브라우저로 http://127.0.0.1:8000 에 접속한 화면 (Edge). "예쁜 인쇄" 를 체크하면 줄을 나눠 보여 줘요



http://127.0.0.1:8000/time — 새로고침할 때마다 시각이 바뀌어요

브라우저 대신 파이썬 코드로도 같은 요청을 보낼 수 있어요. 이 과정의 `..._client.py` 파일들은 모두 이 방법(`TestClient`)으로 결과를 보여 줘요. 서버를 켜지 않아도 돼서 편리해요. 자세한 사용법은 1장 5절에서 배워요.

예제 파일 `fastapi/00_웹과_API_첫걸음/ex03_first_server_client.py`

ex03_first_server_client.py

```python
# 클라이언트 예제: 서버를 켜지 않고 ex03_first_server 의 앱에 요청을 보내 봐요 (python 으로 실행)
# 1장에서 자세히 배우는 TestClient 를 미리 맛보는 예제예요. 브라우저로 확인했다면 건너뛰어도 돼요.
from fastapi.testclient import TestClient

from ex03_first_server import app

client = TestClient(app)                        # 가짜 브라우저 역할

res = client.get("/")                           # 브라우저 주소창에 http://127.0.0.1:8000/ 을 친 것과 같아요
print(res.status_code, res.json())              # 상태 코드(200 = 성공)와 JSON 응답

res = client.get("/time")
print(res.status_code, list(res.json()))        # 시각은 매번 달라서 키 이름만 출력

res = client.get("/없는주소")
print(res.status_code, res.json())              # 등록하지 않은 주소 → 404
```

▶ 실행 결과

```
200 {'message': '제 첫 웹 서버예요!'}
200 ['now']
404 {'detail': 'Not Found'}
```

8. 자주 막히는 곳과 정리

증상	원인	해결
`'python' 은(는) 내부 또는 외부 명령…이 아닙니다`	파이썬이 설치되지 않았거나 PATH 에 없음	설치할 때 **Add python.exe to PATH** 체크, 또는 `py` 명령 사용
`No module named 'fastapi'`	가상환경이 꺼진 터미널에서 실행	줄 앞에 `(.venv)` 가 있는지 확인 → 켜고 다시 실행
`uvicorn` 을 찾을 수 없음	설치 안 됨 / 가상환경 꺼짐	`pip install uvicorn` 또는 `python -m uvicorn 파일:app`
`Could not import module "ex03_first_server"`	다른 폴더에서 서버를 켬	`cd` 로 예제 폴더에 들어가서 실행
`address already in use` / `10048`	8000번 방에 이미 다른 서버가 있음	켜 둔 서버를 `Ctrl+C` 로 끄기, 또는 `--port 8001`
브라우저에 "사이트에 연결할 수 없음"	서버가 꺼져 있음	터미널 1 에 `Uvicorn running…` 이 보이는지 확인

개념	한 줄 정리
요청 · 응답	클라이언트가 요청하고, 서버가 응답한다
URL	호스트 + 포트 + 경로 + 쿼리
메서드	GET 가져오기, POST 보내기, PUT/PATCH 고치기, DELETE 지우기
상태 코드	2xx 성공, 4xx 요청 문제, 5xx 서버 문제
JSON	딕셔너리 ↔ 글자. `json.dumps` / `json.loads`
서버 켜기	예제 폴더에서 `uvicorn 파일:app --reload`, 끄기는 `Ctrl+C`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요.

문제 1. JSON 글자 `'{"city": "서울", "temp": 21.5, "rain": false}'` 를 딕셔너리로 바꾼 뒤 `서울의 기온은 21.5도, 비: 아니요` 처럼 출력하세요. (rain 이 true 면 "예")

문제 2. `ex03_first_server.py` 를 참고해 `GET /about` 에 `{"name": 내 이름, "course": "FastAPI"}` 를 돌려주는 서버를 만들고 브라우저로 확인하세요.

문제 3. 주소 `http://127.0.0.1:8080/books/12?lang=ko` 에서 호스트 · 포트 · 경로 · 쿼리를 각각 적어 보세요.

연습문제 정답 및 해설

문제 1 정답

문제1.py

```python
# 문제 1. JSON 글자 '{"city": "서울", "temp": 21.5, "rain": false}' 를 딕셔너리로 바꾼 뒤,
#         "서울의 기온은 21.5도, 비: 아니요" 처럼 출력하세요. (rain 이 true 면 "예")
import json

text = '{"city": "서울", "temp": 21.5, "rain": false}'
weather = json.loads(text)                    # JSON 글자 → 딕셔너리 (false → False)
rain = "예" if weather["rain"] else "아니요"
print(f"{weather['city']}의 기온은 {weather['temp']}도, 비: {rain}")
```

▶ 실행 결과

```
서울의 기온은 21.5도, 비: 아니요
```

`json.loads` 가 JSON 의 `false` 를 파이썬의 `False` 로 바꿔 줘요.

문제 2 정답

문제2.py

```python
# 문제 2. ex03_first_server.py 를 참고해 GET /about 에 {"name": 내 이름, "course": "FastAPI"} 를
#         돌려주는 서버를 만들고, 브라우저(또는 TestClient)로 확인하세요.
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


@app.get("/about")
def about():
    return {"name": "홍길동", "course": "FastAPI"}


# 서버로 켤 때:  uvicorn 문제2:app --reload  →  브라우저에서 http://127.0.0.1:8000/about
# 여기서는 바로 확인할 수 있게 TestClient 로 요청을 보내요
client = TestClient(app)
print(client.get("/about").json())
```

▶ 실행 결과

```
{'name': '홍길동', 'course': 'FastAPI'}
```

서버로 확인하려면 정답 폴더에서 `uvicorn 문제2:app --reload` 로 켜고 브라우저에서 `http://127.0.0.1:8000/about` 에 접속해요.

문제 3 정답

부분	값
호스트	`127.0.0.1` (내 컴퓨터)
포트	`8080`
경로	`/books/12`
쿼리	`lang=ko` (이름 `lang`, 값 `ko`)

01장

FastAPI 시작하기

웹 API가 무엇인지 알고, 첫 API 서버를 만들어 실행해 봅니다

항목	내용
예제 폴더	`fastapi/01_FastAPI_시작하기`
예제 파일	앱 `ex01_hello.py`, `ex02_path_query.py` / 요청 보내기 `ex01_hello_client.py`, `ex02_path_query_client.py`
준비	`pip install fastapi uvicorn` (Python 과정 8 ~ 11장 권장)
예상 학습 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- API, HTTP 요청·응답, JSON이 무엇인지 설명할 수 있어요.
- FastAPI로 첫 API를 만들고 `uvicorn`으로 서버를 실행할 수 있어요.
- 브라우저와 자동 문서(`/docs`)로 API를 시험할 수 있어요.
- 경로 매개변수(`/items/{id}`)와 쿼리 매개변수(`?q=`)를 쓸 수 있어요.
- 타입 힌트로 값이 자동 변환·검사되는 것을 확인할 수 있어요.
- `TestClient`로 서버 없이 API를 호출해 볼 수 있어요.

도움말 | 이번 장의 구성

- 1. 왜 FastAPI 를 배우나요? — 모델 서빙 **[필수]**
- 2. API · HTTP · JSON **[필수]**
- 3. 설치와 첫 API **[필수]**
- 4. 서버 실행과 자동 문서 **[필수]**
- 5. TestClient 로 요청 보내기 **[필수]**
- 6. 경로 매개변수와 쿼리 매개변수 **[필수]**
- 7. 자주 하는 실수와 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

0장에서 켜 본 서버에 **내가 원하는 주소(창구)를 여러 개** 만들고, 주소에 값을 실어 보내는 두 가지 방법(경로 · 쿼리)을 배워요. 이 장은 모두 [필수] 예요.

- 처음에는 **브라우저와 `/docs` 화면**으로 확인하고, `_client.py` 파일은 "같은 일을 코드로 하는 방법" 으로 가볍게 보세요.
- 0장을 건너뛰었다면 0장 6절(가상환경·설치)과 7절(서버 켜고 끄기)부터 하고 오세요.

1. 왜 FastAPI 를 배우나요? — 모델 서빙 필수

비유로 이해하기 | 식당의 주방과 주문 창구

딥러닝 모델을 열심히 학습시켜도, 그 모델이 **내 컴퓨터의 노트북 파일 안에만** 있으면 다른 사람은 쓸 수 없어요. 식당으로 치면 훌륭한 요리사(모델)는 있는데 **주문 창구**가 없는 셈이죠.

API 서버는 이 주문 창구예요. 웹사이트·앱·다른 프로그램이 "이 사진이 고양이야?"라고 **요청**을 보내면, 서버가 모델에게 물어보고 "고양이일 확률 97%"라고 **응답**해요. 모델을 이렇게 서비스로 제공하는 것을 **모델 서빙(serving)**이라고 해요.

단계	하는 일	이 과정의 장
1. API 기초	요청을 받아 응답하는 서버 만들기	1장
2. 데이터 검사	들어오는 데이터의 형식·범위를 자동 검사	2장
3. 실전 API	CRUD, 라우터, 오류 응답	3장
4. 서버 구조	의존성, 시작할 때 모델 불러오기(lifespan), 미들웨어	4장
5. 파일·이미지	이미지 업로드 → 배열로 전처리	5장
6. 모델 서빙	학습한 모델을 불러와 `/predict` API로	6장
7. 테스트·배포	pytest, 프로젝트 구조, Docker	7장

FastAPI 의 장점	설명
빠름	이름처럼 빠르고, 비동기(async)를 지원해요.
타입 힌트 기반	Python 11장의 타입 힌트만 적으면 **자동으로 변환·검사**해요.
자동 문서	`/docs`에 시험해 볼 수 있는 API 문서가 자동으로 생겨요.
머신러닝 친화	PyTorch, TensorFlow, scikit-learn 모델 서빙에 가장 많이 쓰이는 프레임워크 중 하나예요.

2. API · HTTP · JSON 필수

용어	뜻	예
API	프로그램끼리 정해진 방식으로 요청·응답하는 **창구**	날씨 API, 번역 API, 모델 예측 API
HTTP	웹에서 요청·응답을 주고받는 약속	`http://127.0.0.1:8000/hello`
엔드포인트	API의 주소 하나하나	`/predict`, `/health`
JSON	데이터를 주고받는 글자 형식 (파이썬 딕셔너리와 비슷)	`{"name": "홍길동", "age": 20}`
상태 코드	응답이 성공했는지 알려 주는 숫자	200 성공, 404 없음, 422 입력 오류, 500 서버 오류

HTTP 메서드	뜻	예
`GET`	조회 (데이터 가져오기)	상품 목록 보기
`POST`	생성 / 처리 요청 (데이터 보내기)	회원 가입, **예측 요청**
`PUT` / `PATCH`	수정 (전체 / 일부)	정보 변경
`DELETE`	삭제	글 삭제

3. 설치와 첫 API 필수

터미널에서 설치 (uvicorn: FastAPI 앱을 실행해 주는 서버 프로그램)

```
pip install fastapi uvicorn
```

예제 파일 `fastapi/01_FastAPI_시작하기/ex01_hello.py`

가장 짧은 FastAPI 앱이에요. **실행하는 파일이 아니라** 서버(uvicorn)가 불러가는 파일이에요.

ex01_hello.py

```
# FastAPI: 웹 API 서버를 만드는 도구
from fastapi import FastAPI

app = FastAPI(title="첫 번째 API", version="1.0")   # API 서버(앱) 만들기


@app.get("/")                          # GET 방식으로 "/" 주소에 요청이 오면
def root():                            # 이 함수가 실행되고
    return {"message": "안녕하세요, FastAPI!"}   # 딕셔너리가 JSON 으로 응답돼요


@app.get("/hello/{name}")              # 주소의 일부를 변수로 받기
# {name}: 주소 /hello/홍길동 의 "홍길동" 부분이 name 변수로 들어와요
def hello(name: str):
    return {"greeting": f"{name}님, 반가워요!"}


@app.get("/health")                    # 서버가 살아 있는지 확인용 (서빙에서 꼭 만들어요)
def health():
    return {"status": "ok"}
```

한 줄씩 읽기

줄	코드	설명
2	`from fastapi import FastAPI`	FastAPI 클래스 가져오기
4	`app = FastAPI(title="첫 번째 API", version="1.0") # API 서버(앱) 만들기`	★ 앱 객체. 이름은 관례적으로 `app`
7	`@app.get("/") # GET 방식으로 "/" 주소에 요청이 오면`	★ **데코레이터**: "GET 방식으로 `/`에 요청이 오면 아래 함수를 실행해 줘"라고 등록
9	`return {"message": "안녕하세요, FastAPI!"} # 딕셔너리가 JSON 으로 응답돼요`	딕셔너리를 return 하면 FastAPI가 **JSON으로 바꿔** 응답해요.
12	`@app.get("/hello/{name}") # 주소의 일부를 변수로 받기`	주소 안의 `{name}` 자리가 함수의 `name` 매개변수로 들어와요.
18	`@app.get("/health") # 서버가 살아 있는지 확인용 (서빙에서 꼭 만들어요)`	서버 상태 확인용 엔드포인트. 실제 서비스에서는 이 주소로 서버가 정상인지 주기적으로 검사해요.

4. 서버 실행과 자동 문서 필수

터미널에서 예제 폴더로 이동한 뒤 아래 명령을 입력해요. `ex01_hello:app`은 "`ex01_hello.py` 파일 안의 `app` 객체"라는 뜻이에요.

서버 실행 (터미널 명령)

```
cd fastapi/01_FastAPI_시작하기
uvicorn ex01_hello:app --reload
```

▶ 실제로 실행한 터미널 화면 (서버 실행 → 다른 터미널에서 curl 로 요청. --reload 를 붙이면 감시 안내 줄이 더 나와요)

```
[터미널 1] $ uvicorn ex01_hello:app
INFO:     Started server process [11336]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:7717 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:7718 - "GET /hello/Kim HTTP/1.1" 200 OK
INFO:     127.0.0.1:7719 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:7720 - "GET /docs HTTP/1.1" 200 OK

[터미널 2]
$ curl http://127.0.0.1:8000/
{"message":"안녕하세요, FastAPI!"}
$ curl http://127.0.0.1:8000/hello/Kim
{"greeting":"Kim님, 반가워요!"}
$ curl http://127.0.0.1:8000/health
{"status":"ok"}
$ curl -o /dev/null -w "%{http_code}" http://127.0.0.1:8000/docs
200
```

주소 (브라우저에 입력)	보이는 것
`http://127.0.0.1:8000/`	`{"message":"안녕하세요, FastAPI!"}`
`http://127.0.0.1:8000/hello/홍길동`	인사 JSON
`http://127.0.0.1:8000/docs`	★ **Swagger UI 자동 문서** — 각 API의 `Try it out` 버튼으로 바로 시험
`http://127.0.0.1:8000/redoc`	읽기 좋은 다른 모양의 문서

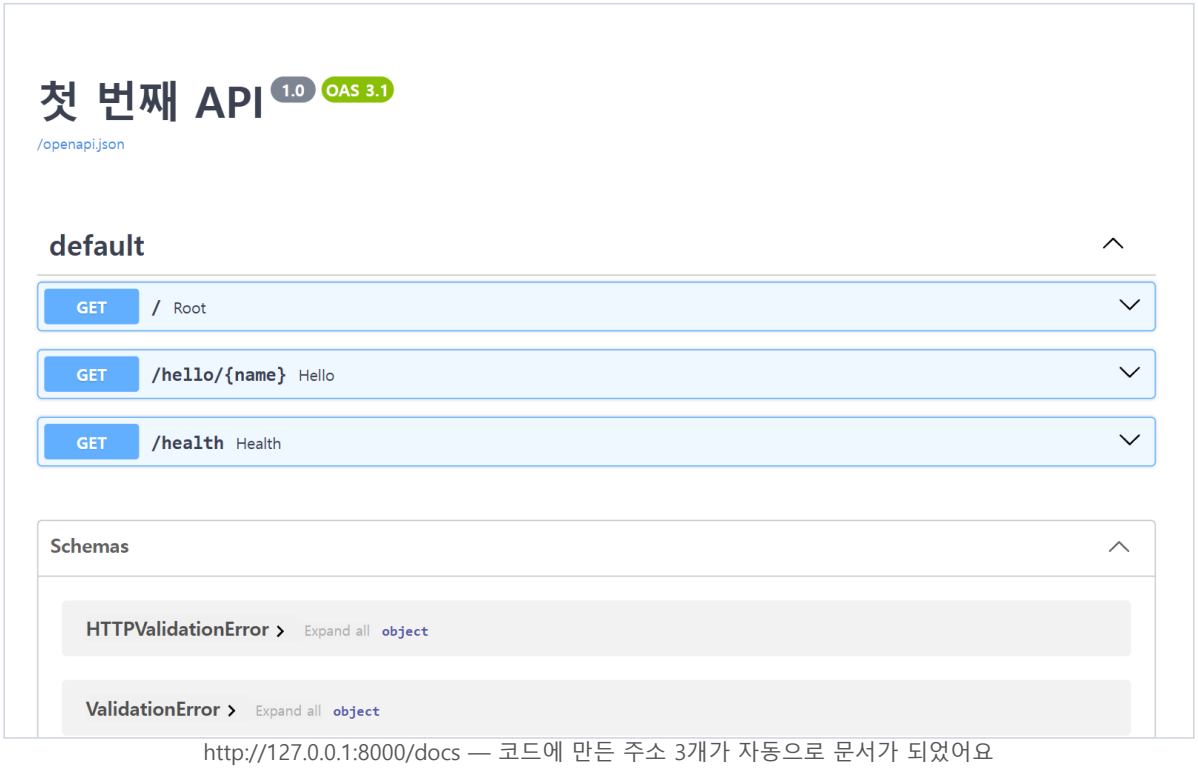
uvicorn 옵션	뜻
`--reload`	코드를 저장하면 서버가 **자동으로 다시 시작** (개발할 때만)
`--port 8080`	포트 번호 바꾸기 (기본 8000)
`--host 0.0.0.0`	같은 네트워크의 다른 컴퓨터에서도 접속 가능하게
`Ctrl + C`	서버 끄기

알아 두기 | 127.0.0.1 과 포트

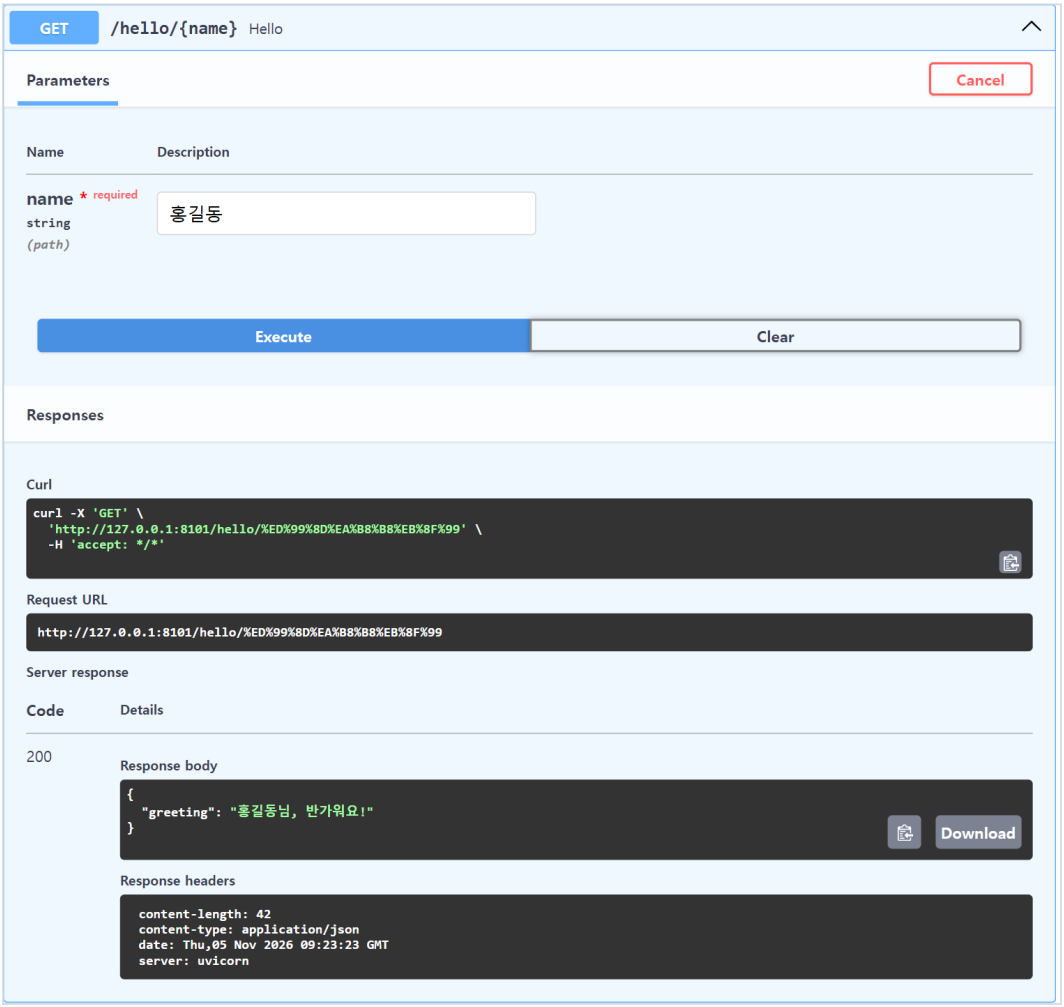
`127.0.0.1`(= `localhost`)은 **내 컴퓨터 자신**을 가리키는 주소예요. `8000`은 포트 번호로, 한 컴퓨터에서 여러 서버를 구분하는 "방 번호"예요.

직접 해 보기 | 화면으로 보는 /docs — Try it out 따라 하기

- `uvicorn ex01_hello:app --reload` 로 서버를 켜고 브라우저에서 `http://127.0.0.1:8000/docs` 에 접속해요.
- 목록에서 **GET /hello/{name}** 을 눌러 펼치고 오른쪽의 **Try it out** 버튼을 눌러요.
- `name` 칸에 이름을 적고 파란 **Execute** 버튼을 눌러요.
- 아래 **Server response** 의 Code(200)와 Response body(JSON)가 실제 응답이에요. **Request URL** 은 브라우저 주소창에 넣어도 똑같은 결과가 나오는 주소예요.



http://127.0.0.1:8000/docs — 코드에 만든 주소 3개가 자동으로 문서가 되었어요



GET /hello/{name} 을 펼쳐 Try it out → name 입력 → Execute 한 결과. 한글 이름은 주소에서 %ED%99... 로 바뀌어 전송돼요

5. TestClient 로 요청 보내기 필수

> **도움말 | TestClient 는 "코드로 만든 가짜 브라우저"**
>
> `client.get("/hello/홍길동")` 한 줄은 브라우저 주소창에 `http://127.0.0.1:8000/hello/홍길동` 을 입력한 것과 같아요. 서버를 켜지 않아도 돼서 결과를 확인하기 편해요. 예제의 `show()` 함수는 **결과를 보기 좋게 출력하려고 만든 도우미**일 뿐이라 안쪽(try/except)을 다 이해하지 못해도 괜찮아요.

서버를 켜고 브라우저로 확인하는 대신, **서버 없이** 코드로 요청을 보내 볼 수 있어요. 이 과정의 `..._client.py` 파일들은 모두 `TestClient`로 앱에 실제 요청을 보내고 응답을 출력해요. 그래서 교안의 실행 결과는 **실제 응답 그대로**예요. (7장에서 이 도구로 자동 테스트도 만들어요.)

예제 파일 `fastapi/01_FastAPI_시작하기/ex01_hello_client.py`

ex01_hello_client.py

```
1  # 클라이언트 예제: 앱(ex01_hello.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
2  import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
3  from fastapi.testclient import TestClient
4  from ex01_hello import app           # 같은 폴더의 앱 파일에서 app 가져오기
5
6  client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구
7
8
9  # show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
10 def show(title, res):
11     print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
12     # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
13     try:
14         print(json.dumps(res.json(), ensure_ascii=False, indent=2))
15     # JSON 이 아니면(글자 응답 등) 그대로 출력
16     except ValueError:
17         print(res.text)
18     print()
19
20 # 여러 주소로 GET 요청을 보내고 결과 보기
21 show("루트", client.get("/"))
22 show("인사", client.get("/hello/홍길동"))
23 show("상태 확인", client.get("/health"))
24 # 없는 주소는 404 Not Found
25 show("없는 주소", client.get("/nothing"))
```

한 줄씩 읽기

줄	코드	설명
3	`from fastapi.testclient import TestClient`	FastAPI가 제공하는 테스트용 클라이언트 (설치: `pip install httpx2`)
4	`from ex01_hello import app # 같은 폴더의 앱 파일에서 app 가져오기`	앱 파일을 모듈로 가져와요 (Python 10장 import)
6	`client = TestClient(app) # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구`	★ `client.get("/주소")`가 실제 HTTP 요청과 똑같이 동작해요.
11	`print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")`	보낸 요청과 **상태 코드** 출력
14	`print(json.dumps(res.json(), ensure_ascii=False, indent=2))`	`res.json()`: 응답 JSON → 딕셔너리. `ensure_ascii=False`로 한글 그대로
25	`show("없는 주소", client.get("/nothing"))`	등록하지 않은 주소 → **404 Not Found**

▶ 실행 결과

```
[루트] GET / → 200
{
  "message": "안녕하세요, FastAPI!"
}

[인사] GET /hello/홍길동 → 200
{
  "greeting": "홍길동님, 반가워요!"
}

[상태 확인] GET /health → 200
{
  "status": "ok"
}

[없는 주소] GET /nothing → 404
{
  "detail": "Not Found"
}
```

도움말 | 실행 방법

앱 파일과 같은 폴더에서 `python ex01_hello_client.py`. TestClient를 쓰려면 `pip install httpx2`(또는 `httpx`)가 필요해요.

6. 경로 매개변수와 쿼리 매개변수 필수

종류	주소 모양	FastAPI 코드	용도
경로 매개변수	`/items/3`	`@app.get("/items/{item_id}")` + `def f(item_id: int)`	**무엇을** (특정 대상)
쿼리 매개변수	`/items?skip=1&limit=2`	`def f(skip: int = 0, limit: int = 3)`	**어떻게** (검색, 정렬, 개수)

예제 파일 `fastapi/01_FastAPI_시작하기/ex02_path_query.py`

ex02_path_query.py

```python
from fastapi import FastAPI

app = FastAPI()

# 실습용 상품 목록 (번호: 이름)
ITEMS = {1: "노트북", 2: "마우스", 3: "키보드", 4: "모니터", 5: "스피커"}


@app.get("/items/{item_id}")                        # 경로 매개변수: 주소 안의 값
def read_item(item_id: int):                        # int 힌트 → "3" 이 3 으로 변환
    # ITEMS.get(번호, 기본값): 없는 번호면 "없는 상품"
    return {"item_id": item_id, "name": ITEMS.get(item_id, "없는 상품")}


@app.get("/items")                                  # 쿼리 매개변수: ?skip=0&limit=2
def list_items(skip: int = 0, limit: int = 3, keyword: str | None = None):
    # 상품 이름만 리스트로 꺼내기
    names = list(ITEMS.values())
    # keyword 가 있으면 그 글자가 들어간 이름만 남기기
    if keyword:
        names = [n for n in names if keyword in n]
    # names[skip: skip + limit]: skip 번째부터 limit 개만 잘라서 돌려주기 (페이지 나누기)
```

ex02_path_query.py

```
    return {"skip": skip, "limit": limit, "keyword": keyword, "items": names[skip: skip + limit]}


@app.get("/calc/add")
def add(a: float, b: float):                        # 기본값이 없으면 "필수" 쿼리
    return {"a": a, "b": b, "result": a + b}
```

한 줄씩 읽기

줄	코드	설명
10	`def read_item(item_id: int): # int 힌트 → "3"이 3 으로 변환`	★ 주소의 "3"은 글자지만, `int` 힌트 덕분에 **자동으로 정수**로 바뀌어요.
16	`def list_items(skip: int = 0, limit: int = 3, keyword: str | None = None):`	경로에 없는 매개변수는 **쿼리 매개변수**. 기본값이 있으면 생략 가능, `str | None`은 없어도 되는 값 (Python 11장)
23	`return {"skip": skip, "limit": limit, "keyword": keyword, "items": names[skip: skip + limit]}`	슬라이싱으로 페이지 나누기 (Python 6장)
27	`def add(a: float, b: float): # 기본값이 없으면 "필수" 쿼리`	기본값이 없으면 **반드시** 보내야 하는 쿼리

예제 파일 `fastapi/01_FastAPI_시작하기/ex02_path_query_client.py`

ex02_path_query_client.py

```
# 클라이언트 예제: 앱(ex02_path_query.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                             # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex02_path_query import app             # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)                # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

show("경로 매개변수", client.get("/items/3"))
show("쿼리 기본값", client.get("/items"))
# params= 로 쿼리를 사전으로 넘기면 ?skip=1&limit=2 로 바뀌어요
show("쿼리 지정", client.get("/items", params={"skip": 1, "limit": 2}))
show("검색", client.get("/items?keyword=마"))
show("계산", client.get("/calc/add?a=3&b=4.5"))
# 아래 두 개는 일부러 틀린 요청: FastAPI 가 422 오류와 이유를 돌려줘요
show("숫자가 아닌 id", client.get("/items/abc"))
show("필수 쿼리 빠짐", client.get("/calc/add?a=3"))
```

한 줄씩 읽기

줄	코드	설명
23	`show("쿼리 지정", client.get("/items", params={"skip": 1, "limit": 2}))`	`params=` 딕셔너리로 쿼리를 보내면 `?skip=1&limit=2`가 붙어요.

줄	코드	설명
24	`show("검색", client.get("/items?keyword=마"))`	주소에 직접 써도 돼요.
27	`show("숫자가 아닌 id", client.get("/items/abc"))`	★ `int`여야 하는데 "abc" → FastAPI가 자동으로 **422 오류**와 이유를 돌려줘요.
28	`show("필수 쿼리 빠짐", client.get("/calc/add?a=3"))`	b가 빠져서 422 — `"type": "missing"`

▶ **실행 결과**

```
[경로 매개변수] GET /items/3 → 200
{
  "item_id": 3,
  "name": "키보드"
}

[쿼리 기본값] GET /items → 200
{
  "skip": 0,
  "limit": 3,
  "keyword": null,
  "items": [
    "노트북",
    "마우스",
    "키보드"
  ]
}

[쿼리 지정] GET /items → 200
{
  "skip": 1,
  "limit": 2,
  "keyword": null,
  "items": [
    "마우스",
    "키보드"
  ]
}

[검색] GET /items → 200
{
  "skip": 0,
  "limit": 3,
  "keyword": "마",
  "items": [
    "마우스"
  ]
}

[계산] GET /calc/add → 200
{
  "a": 3.0,
  "b": 4.5,
  "result": 7.5
}

[숫자가 아닌 id] GET /items/abc → 422
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": [
        "path",
        "item_id"
```

▶ 실행 결과

```
      ],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "abc"
    }
  ]
}

[필수 쿼리 빠짐] GET /calc/add → 422
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "query",
        "b"
      ],
      "msg": "Field required",
      "input": null
    }
  ]
}
```

알아 두기 | 422 오류는 FastAPI 가 대신 검사해 준 것

우리는 `item_id: int`라고 타입 힌트만 썼는데, FastAPI가 "abc는 정수가 아니에요"라고 **자동으로** 검사하고 친절한 오류 응답까지 만들어 줬어요. Python 11장에서 직접 isinstance로 검사하던 일을 대신 해 주는 거예요. 모델 서빙에서 이상한 입력이 모델에 들어가는 것을 막아 주는 첫 번째 방어선이에요.

7. 자주 하는 실수와 정리

증상	원인	해결
`Error loading ASGI app. Could not import module`	파일 이름이 틀렸거나 다른 폴더에서 실행	예제 폴더로 `cd`, `파일명:app` 확인
`Attribute "app" not found`	`:` 뒤 이름이 앱 변수와 다름	`app = FastAPI()` 변수 이름 확인
`[Errno 10048] ... address already in use`	8000번 포트를 이미 사용 중	다른 서버 끄기 또는 `--port 8001`
코드를 바꿨는데 결과가 그대로	`--reload` 없이 실행	`--reload` 또는 서버 재시작
`ModuleNotFoundError: No module named 'httpx'`	TestClient 준비 안 됨	`pip install httpx2`
422 Unprocessable Entity	입력 형식이 힌트와 다름	응답의 `detail`에서 어느 값이 틀렸는지 확인

개념	핵심
앱	`app = FastAPI()`
엔드포인트	`@app.get("/주소")` + 함수 → return 딕셔너리 = JSON
실행	`uvicorn 파일명:app --reload` → `/docs`
경로 매개변수	`/items/{item_id}` + `item_id: int`
쿼리 매개변수	함수 매개변수 (기본값 있으면 선택)
자동 검사	타입 힌트와 다르면 422
테스트	`TestClient(app).get("/주소")`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. `GET /square/{n}`에 정수 n을 받아 `{"n": n, "square": n의 제곱}`을 돌려주는 앱을 만들고, TestClient로 n=7을 요청해 보세요.

▶ **이렇게 출력되면 성공**

```
{'n': 7, 'square': 49}
```

문제 2. `GET /bmi?height=175&weight=70`처럼 키(cm)와 몸무게(kg)를 쿼리로 받아 BMI(소수 1자리)를 돌려주는 API를 만드세요. (BMI = 몸무게 ÷ (키/100)²)

▶ **이렇게 출력되면 성공**

```
{'height': 175.0, 'weight': 70.0, 'bmi': 22.9}
```

문제 3. 문제 2의 API에 `height=abc`를 보내면 상태 코드가 무엇인지 출력하세요.

▶ **이렇게 출력되면 성공**

```
422
```

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


@app.get("/square/{n}")
def square(n: int):
    return {"n": n, "square": n * n}


client = TestClient(app)
print(client.get("/square/7").json())
```

▶ 실행 결과

```
{'n': 7, 'square': 49}
```

연습할 때는 앱과 TestClient를 한 파일에 둬도 돼요. 실제 프로젝트에서는 1장 예제처럼 앱 파일과 테스트 파일을 나눠요.

문제 2 정답

직접 입력해 보세요

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


@app.get("/bmi")
def bmi(height: float, weight: float):
    value = weight / (height / 100) ** 2
    return {"height": height, "weight": weight, "bmi": round(value, 1)}


client = TestClient(app)
print(client.get("/bmi", params={"height": 175, "weight": 70}).json())
```

▶ 실행 결과

```
{'height': 175.0, 'weight': 70.0, 'bmi': 22.9}
```

문제 3 정답

직접 입력해 보세요

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


@app.get("/bmi")
def bmi(height: float, weight: float):
```

직접 입력해 보세요

```
    return {"bmi": round(weight / (height / 100) ** 2, 1)}


client = TestClient(app)
print(client.get("/bmi?height=abc&weight=70").status_code)
```

▶ 실행 결과

```
422
```

타입 힌트 `float`와 맞지 않아서 FastAPI가 422(입력 형식 오류)를 돌려줘요.

02장

Pydantic 으로 요청과 응답 다루기

들어오는 데이터의 모양과 범위를 정하고, 자동으로 검사합니다

항목	내용
예제 폴더	`fastapi/02_Pydantic_요청과_응답`
예제 파일	앱 `ex01_body.py` ~ `ex04_custom_validator.py` + 각각의 `_client.py`
이전 장	01 FastAPI 시작하기 · Python 11장 타입 체크
예상 학습 시간	3 ~ 4시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- POST 요청의 본문(body)을 Pydantic 모델로 받을 수 있어요.
- `Field`로 값의 범위·길이·예시를 정할 수 있어요.
- 리스트·중첩 모델·`Literal`·`Optional` 필드를 쓸 수 있어요.
- `response_model`로 응답 모양을 정하고 민감한 값을 숨길 수 있어요.
- `status_code`와 422 오류 응답의 구조를 이해해요.
- `field_validator`로 나만의 검사 규칙을 만들 수 있어요.

도움말 | 이번 장의 구성

- 1. Pydantic 이란? **[필수]**
- 2. 요청 본문(body) 받기 **[필수]**
- 3. Field 로 범위 정하기 **[필수]**
- 4. 중첩 모델과 리스트 **[필수]**
- 5. 나만의 검사 규칙 **[심화]**
- 6. 자주 하는 실수와 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

1장의 GET 은 주소에 값을 실었지만, 회원 가입처럼 **보낼 값이 많을 때는 POST 요청의 본문(body)에 JSON 을 담아** 보내요. Pydantic 은 그 JSON 의 "모양과 규칙" 을 정해 두고 자동으로 검사해 주는 도구예요.

- [필수] 1~4절 : `BaseModel` 로 모양 정하기, `Field` 로 범위 정하기, 리스트·응답 모양
- [심화] 5절 : 나만의 검사 규칙(`field_validator`) — 처음에는 건너뛰어도 돼요
- POST 는 브라우저 주소창으로 보낼 수 없어요. `/docs` **의 Try it out** 으로 시험해 보세요(2절 끝 그림).

1. Pydantic 이란? 필수

> **비유로 이해하기 | 공항 보안 검색대**
> 비행기에 타기 전에 짐을 검색대에 통과시키듯, API로 들어오는 데이터도 **입구에서 검사**해야 해요. 나이에 "열여덟"이 들어오거나 면적이 -1이면 모델이 엉뚱한 예측을 하거나 서버가 오류로 멈출 수 있어요.
>
> **Pydantic**은 "데이터는 이런 모양이어야 한다"를 클래스로 적으면, 그대로 **검사하고 변환**해 주는 라이브러리예요. FastAPI는 Pydantic을 기본으로 사용해요.

Python 11장에서 직접 하던 것	Pydantic 에서는
`if not isinstance(age, int): raise TypeError`	`age: int` 한 줄
`if age < 0: raise ValueError`	`age: int = Field(ge=0)`
`@dataclass` + `__post_init__`	`class X(BaseModel):` + `field_validator`
오류 메시지 직접 작성	어느 필드가 왜 틀렸는지 **자동으로** 정리된 422 응답

2. 요청 본문(body) 받기 필수

1장의 경로·쿼리 매개변수는 주소에 값을 담았어요. 데이터가 많거나 구조가 복잡하면 **POST 요청의 본문(body)** 에 JSON으로 담아 보내요. 모델 예측 요청도 대부분 이 방식이에요.

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex01_body.py`

ex01_body.py

```python
from fastapi import FastAPI
# BaseModel 을 물려받으면 "이런 모양의 JSON 을 받겠다"고 정할 수 있어요
from pydantic import BaseModel

app = FastAPI()


class Student(BaseModel):          # 요청 데이터의 "모양" 정의 (dataclass 와 비슷)
    # 각 줄 = 필드 이름: 자료형. 자료형이 맞지 않으면 FastAPI 가 422 오류로 돌려보내요
    name: str
    age: int
    score: float
    email: str | None = None       # 없어도 되는 값


@app.post("/students")             # POST: 데이터를 보내는 요청
def create_student(student: Student):          # 본문 JSON → Student 객체로 자동 변환
    # 받은 점수로 합격 여부 계산
    grade = "합격" if student.score >= 60 else "불합격"
    return {"received": student, "result": grade, "name_length": len(student.name)}
```

한 줄씩 읽기

줄	코드	설명
8	`class Student(BaseModel): # 요청 데이터의 "모양" 정의 (dataclass 와 비슷)`	★ `BaseModel`을 상속하고 타입 힌트로 필드를 적어요 (Python 11장의 dataclass와 비슷).
13	`email: str | None = None # 없어도 되는 값`	기본값이 있으면 선택, 없으면 필수
16	`@app.post("/students") # POST: 데이터를 보내는 요청`	POST 엔드포인트
17	`def create_student(student: Student): # 본문 JSON → Student 객체로 자동 변환`	★ 매개변수 타입이 BaseModel이면 FastAPI가 **본문 JSON**을 읽어 검사·변환한 객체를 넣어 줘요.
20	`return {"received": student, "result": grade, "name_length": len(student.name)}`	Pydantic 객체를 그대로 응답에 넣어도 JSON으로 바뀌어요.

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex01_body_client.py`

ex01_body_client.py

```python
# 클라이언트 예제: 앱(ex01_body.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex01_body import app            # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

# json= 에 사전을 넣으면 JSON 본문으로 보내요
show("정상", client.post("/students", json={"name": "홍길동", "age": 20, "score": 85.5}))
# "31" 같은 숫자 글자는 int 로 바꿔 줘요
show("문자열 숫자도 변환", client.post("/students", json={"name": "이순신", "age": "31", "score": "59"}))
# 바꿀 수 없는 값, 빠진 값은 422 오류
show("나이가 글자", client.post("/students", json={"name": "유관순", "age": "열여덟", "score": 90}))
show("필수 값 빠짐", client.post("/students", json={"name": "강감찬"}))
```

한 줄씩 읽기

줄	코드	설명
21	`show("정상", client.post("/students", json={"name": "홍길동", "age": 20, "score": 85.5}))`	`json=` 딕셔너리를 본문 JSON으로 보내요.
23	`show("문자열 숫자도 변환", client.post("/students", json={"name": "이순신", "age": "31", "score": "59"}))`	"31"처럼 숫자로 바꿀 수 있는 글자는 **자동 변환** (응답의 age가 31 정수)
25	`show("나이가 글자", client.post("/students", json={"name": "유관순", "age": "열여덟", "score": 90}))`	바꿀 수 없으면 422 `int_parsing`
26	`show("필수 값 빠짐", client.post("/students", json={"name": "강감찬"}))`	age, score가 없어서 422 `missing` 두 개

▶ 실행 결과

```
[정상] POST /students → 200
{
  "received": {
    "name": "홍길동",
    "age": 20,
    "score": 85.5,
    "email": null
  },
  "result": "합격",
  "name_length": 3
}

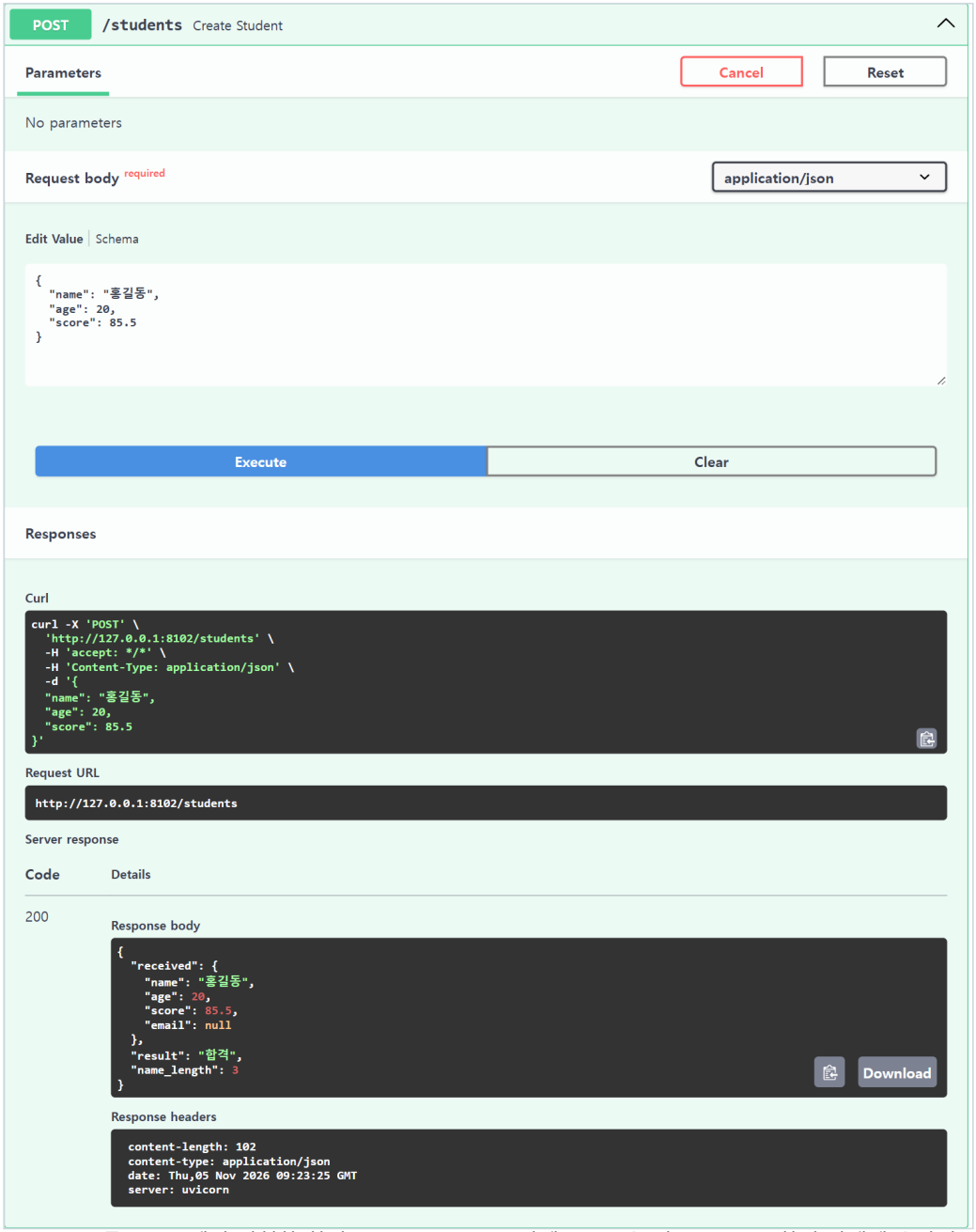
[문자열 숫자도 변환] POST /students → 200
{
  "received": {
```

▶ 실행 결과

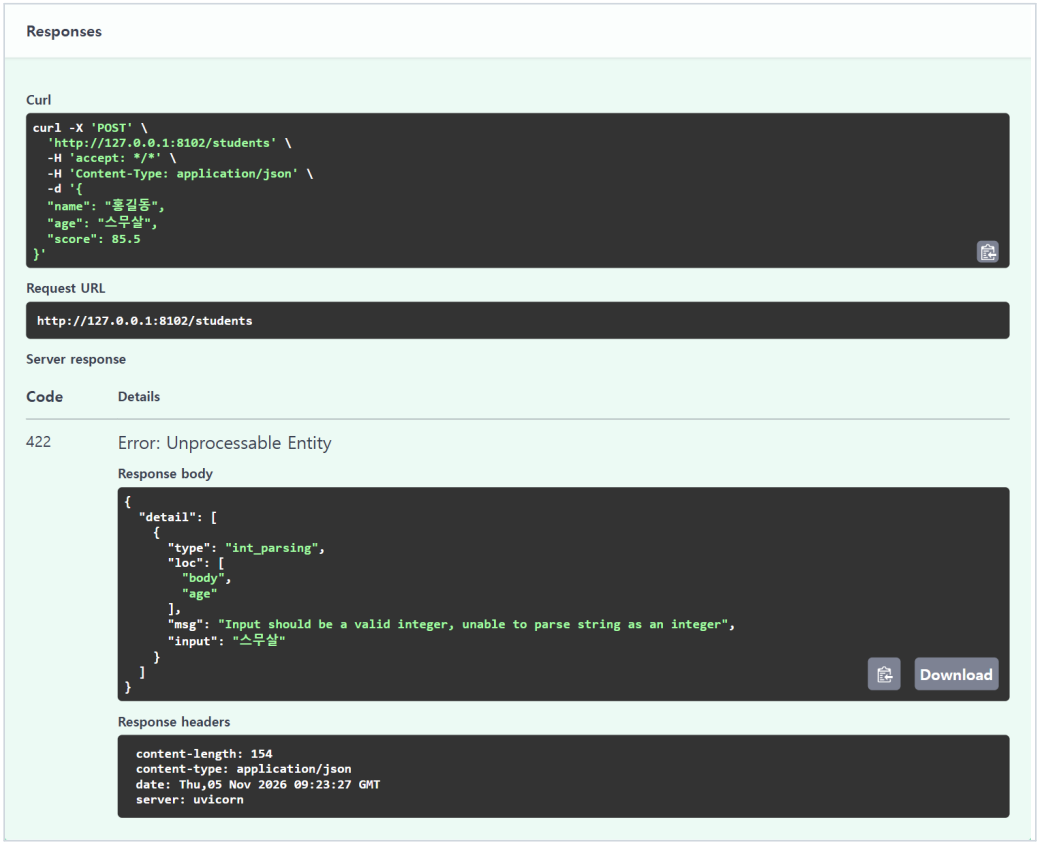
```
    "name": "이순신",
    "age": 31,
    "score": 59.0,
    "email": null
  },
  "result": "불합격",
  "name_length": 3
}

[나이가 글자] POST /students → 422
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": [
        "body",
        "age"
      ],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "열여덟"
    }
  ]
}

[필수 값 빠짐] POST /students → 422
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "age"
      ],
      "msg": "Field required",
      "input": {
        "name": "강감찬"
      }
    },
    {
      "type": "missing",
      "loc": [
        "body",
        "score"
      ],
      "msg": "Field required",
      "input": {
        "name": "강감찬"
      }
    }
  ]
}
```



POST /students 를 /docs 에서 시험한 화면 — Request body 칸에 JSON 을 적고 Execute 하면 아래에 응답이 나와요



age 에 "스무살" 을 보내면 422 — detail 의 loc(어느 칸) · msg(이유)를 읽으면 무엇이 틀렸는지 알 수 있어요

3. Field 로 범위 정하기 필수

Field 옵션	뜻	예
gt, ge	보다 큼 / 크거나 같음	Field(gt=0) 양수만
lt, le	보다 작음 / 작거나 같음	Field(le=500)
min_length, max_length	문자열·리스트 길이	Field(max_length=20)
pattern	정규식 모양	Field(pattern=r"^010-\d{4}-\d{4}$")
default	기본값	Field(default=1)
description, examples	/docs 문서에 보일 설명·예시	

예제 파일 fastapi/02_Pydantic_요청과_응답/ex02_field.py

ex02_field.py

```python
from typing import Literal
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class HouseInput(BaseModel):        # 집값 예측 모델에 넣을 입력이라고 생각해 봐요
    # Field: 값의 범위 · 길이 · 설명을 정하기 (gt 초과, ge 이상, le 이하)
    area: float = Field(gt=0, le=500, description="전용면적(m2)", examples=[84.5])
    rooms: int = Field(ge=1, le=10, description="방 개수")
    built_year: int = Field(ge=1950, le=2026)
    # Literal: 적어 둔 값 중 하나만 허용
    region: Literal["서울", "경기", "부산", "기타"] = "기타"     # 정해진 값 중 하나만
    memo: str = Field(default="", max_length=20)
```

ex02_field.py

```
16
17
18 @app.post("/houses/check")
19 def check(house: HouseInput):
20     # 건물 나이 계산 / model_dump(): 모델 → 사전
21     age = 2026 - house.built_year
22     return {"ok": True, "house_age": age, "input": house.model_dump()}
```

한 줄씩 읽기

줄	코드	설명
8	`class HouseInput(BaseModel): # 집값 예측 모델에 넣을 입력이라고 생각해 봐요`	모델 서빙에서는 **모델 입력 형식**을 이렇게 정의해요.
10	`area: float = Field(gt=0, le=500, description="전용면적(m2)", examples=[84.5])`	★ 0보다 크고 500 이하. 설명과 예시는 `/docs`에 표시돼요.
14	`region: Literal["서울", "경기", "부산", "기타"] = "기타" # 정해진 값 중 하나만`	`Literal`: 정해진 값 중 하나만 허용
22	`return {"ok": True, "house_age": age, "input": house.model_dump()}`	`model_dump()`: Pydantic 객체 → 딕셔너리

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex02_field_client.py`

ex02_field_client.py

```
1  # 클라이언트 예제: 앱(ex02_field.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
2  import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
3  from fastapi.testclient import TestClient
4  from ex02_field import app           # 같은 폴더의 앱 파일에서 app 가져오기
5
6  client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구
7
8
9  # show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
10 def show(title, res):
11     print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
12     # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
13     try:
14         print(json.dumps(res.json(), ensure_ascii=False, indent=2))
15     # JSON 이 아니면(글자 응답 등) 그대로 출력
16     except ValueError:
17         print(res.text)
18     print()
19
20 # 정상 입력을 하나 만들어 두고, {**good, "area": 0} 처럼 일부만 바꿔서 시험
21 good = {"area": 84.5, "rooms": 3, "built_year": 2015, "region": "서울"}
22 show("정상", client.post("/houses/check", json=good))
23 show("면적 0", client.post("/houses/check", json={**good, "area": 0}))
24 show("없는 지역", client.post("/houses/check", json={**good, "region": "제주"}))
25 # 여러 값이 동시에 틀리면 오류 목록에 모두 나와요
26 show("여러 개가 틀림", client.post("/houses/check", json={"area": -1, "rooms": 20, "built_year": 1900}))
```

한 줄씩 읽기

줄	코드	설명
22	`show("정상", client.post("/houses/check", json=good))`	memo는 기본값 ""
23	`show("면적 0", client.post("/houses/check", json={**good, "area": 0}))`	`{**good, "area": 0}`: good을 복사하고 area만 바꾸기. → `greater_than` 오류

줄	코드	설명
24	`show("없는 지역", client.post("/houses/check", json={**good, "region": "제주"}))`	`literal_error`: 허용된 값 목록을 알려 줘요.
26	`show("여러 개가 틀림", client.post("/houses/check", json={"area": -1, "rooms": 20, "built_year": 1900}))`	★ 틀린 곳을 **모두** 한 번에 알려 줘요.

▶ 실행 결과

```
[정상] POST /houses/check → 200
{
  "ok": true,
  "house_age": 11,
  "input": {
    "area": 84.5,
    "rooms": 3,
    "built_year": 2015,
    "region": "서울",
    "memo": ""
  }
}

[면적 0] POST /houses/check → 422
{
  "detail": [
    {
      "type": "greater_than",
      "loc": [
        "body",
        "area"
      ],
      "msg": "Input should be greater than 0",
      "input": 0,
      "ctx": {
        "gt": 0.0
      }
    }
  ]
}

[없는 지역] POST /houses/check → 422
{
  "detail": [
    {
      "type": "literal_error",
      "loc": [
        "body",
        "region"
      ],
      "msg": "Input should be '서울', '경기', '부산' or '기타'",
      "input": "제주",
      "ctx": {
        "expected": "'서울', '경기', '부산' or '기타'"
      }
    }
  ]
}

[여러 개가 틀림] POST /houses/check → 422
{
  "detail": [
    {
      "type": "greater_than",
      "loc": [
```

▶ 실행 결과


```
        "body",
        "area"
      ],
      "msg": "Input should be greater than 0",
      "input": -1,
      "ctx": {
        "gt": 0.0
      }
    },
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "rooms"
      ],
      "msg": "Input should be less than or equal to 10",
      "input": 20,
      "ctx": {
        "le": 10
      }
    },
    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "built_year"
      ],
      "msg": "Input should be greater than or equal to 1950",
      "input": 1900,
      "ctx": {
        "ge": 1950
      }
    }
  ]
}
```


알아 두기 | 422 응답 읽는 법

`detail` 리스트의 각 항목: `type` 오류 종류, `loc` 위치(`["body", "area"]` = 본문의 area), `msg` 설명, `input` 실제로 들어온 값, `ctx` 기준값(예: `{"gt": 0}`)

4. 중첩 모델과 리스트 필수

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex03_nested_response.py`

ex03_nested_response.py

```python
from fastapi import FastAPI, status
from pydantic import BaseModel, Field

app = FastAPI()


# 상품 하나의 모양
class Item(BaseModel):
    name: str
    price: int = Field(gt=0)
    qty: int = Field(default=1, ge=1)


class Order(BaseModel):            # 모델 안에 모델 (중첩)
```

ex03_nested_response.py

```
    customer: str
    items: list[Item] = Field(min_length=1)       # Item 들의 리스트, 최소 1개
    coupon: str | None = None


class UserIn(BaseModel):           # 요청으로 받는 모양 (비밀번호 포함)
    username: str
    password: str = Field(min_length=8)


class UserOut(BaseModel):          # 응답으로 돌려줄 모양 (비밀번호 없음!)
    username: str
    message: str


@app.post("/orders", status_code=status.HTTP_201_CREATED)     # 생성 성공은 201
def create_order(order: Order):
    # 주문 총액 = 가격 × 수량을 모두 더하기, 쿠폰이면 10% 할인
    total = sum(i.price * i.qty for i in order.items)
    if order.coupon == "WELCOME":
        total = int(total * 0.9)
    return {"customer": order.customer, "count": len(order.items), "total": total}


@app.post("/users", response_model=UserOut)    # 응답을 UserOut 모양으로 "걸러서" 보냄
def create_user(user: UserIn):
    # 비밀번호까지 돌려줘도 response_model=UserOut 이 걸러서 응답에서 빠져요
    return {"username": user.username, "password": user.password, "message": "가입 완료"}
```

한 줄씩 읽기

줄	코드	설명
14	`class Order(BaseModel): # 모델 안에 모델 (중첩)`	주문 안에 상품 목록
16	`items: list[Item] = Field(min_length=1) # Item 들의 리스트, 최소 1개`	★ `list[Item]`: 리스트의 **모든 원소**를 Item으로 검사 (Python 11장에서 `all(isinstance(...))`로 하던 것)
30	`@app.post("/orders", status_code=status.HTTP_201_CREATED) # 생성 성공은 201`	성공 상태 코드 지정. 무언가를 **만들었을 때**는 201이 관례
33	`total = sum(i.price * i.qty for i in order.items)`	검사를 통과한 객체라서 안심하고 계산
25	`class UserOut(BaseModel): # 응답으로 돌려줄 모양 (비밀번호 없음!)`	응답용 모델은 따로
39	`@app.post("/users", response_model=UserOut) # 응답을 UserOut 모양으로 "걸러서" 보냄`	★ `response_model`: return 값에 password가 있어도 **UserOut에 없는 필드는 빼고** 응답
42	`return {"username": user.username, "password": user.password, "message": "가입 완료"}`	실수로 비밀번호를 넣어도 응답에는 나가지 않아요.

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex03_nested_response_client.py`

ex03_nested_response_client.py

```
# 클라이언트 예제: 앱(ex03_nested_response.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                            # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex03_nested_response import app           # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)               # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구
```

ex03_nested_response_client.py

```
7
8
9  # show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
10 def show(title, res):
11     print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
12     # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
13     try:
14         print(json.dumps(res.json(), ensure_ascii=False, indent=2))
15     # JSON 이 아니면(글자 응답 등) 그대로 출력
16     except ValueError:
17         print(res.text)
18     print()
19
20 # 주문 안에 상품 목록(중첩 JSON)
21 order = {"customer": "kim", "coupon": "WELCOME",
22          "items": [{"name": "마우스", "price": 15000, "qty": 2}, {"name": "패드", "price": 5000}]}
23 show("중첩 주문", client.post("/orders", json=order))
24 # items 가 비어 있으면 min_length=1 에 걸려 422
25 show("빈 주문", client.post("/orders", json={"customer": "lee", "items": []}))
26 show("가입 (비밀번호는 응답에서 빠짐)", client.post("/users", json={"username": "hong", "password":
   "secret1234"}))
27 show("짧은 비밀번호", client.post("/users", json={"username": "hong", "password": "123"}))
```

한 줄씩 읽기

줄	코드	설명
22	`"items": [{"name": "마우스", "price": 15000, "qty": 2}, {"name": "패드", "price": 5000}]}`	패드의 qty는 기본값 1. 합계 35000 → 쿠폰 10% 할인 31500
25	`show("빈 주문", client.post("/orders", json={"customer": "lee", "items": []}))`	`min_length=1` → `too_short`
26	`show("가입 (비밀번호는 응답에서 빠짐)", client.post("/users", json={"username": "hong", "password": "secret1234"}))`	응답에 password가 **없어요**.

▶ 실행 결과

```
[중첩 주문] POST /orders → 201
{
  "customer": "kim",
  "count": 2,
  "total": 31500
}

[빈 주문] POST /orders → 422
{
  "detail": [
    {
      "type": "too_short",
      "loc": [
        "body",
        "items"
      ],
      "msg": "List should have at least 1 item after validation, not 0",
      "input": [],
      "ctx": {
        "field_type": "List",
        "min_length": 1,
        "actual_length": 0
      }
    }
  ]
```

▶ 실행 결과

```
}

[가입 (비밀번호는 응답에서 빠짐)] POST /users → 200
{
  "username": "hong",
  "message": "가입 완료"
}

[짧은 비밀번호] POST /users → 422
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "password"
      ],
      "msg": "String should have at least 8 characters",
      "input": "123",
      "ctx": {
        "min_length": 8
      }
    }
  ]
}
```

> **도움말 | 모델 서빙에서는 입력·출력 모델을 꼭 나눠요**
>
> `PredictRequest`(입력)와 `PredictResponse`(출력: 예측값, 확률, 모델 버전)를 따로 정의하면, 문서가 깔끔해지고 내부 값이 실수로 새어 나가지 않아요. 6장에서 이렇게 만들어요.

5. 나만의 검사 규칙 심화

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex04_custom_validator.py`

ex04_custom_validator.py

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field, field_validator, model_validator

app = FastAPI()


class Features(BaseModel):          # 모델 입력: 숫자 4개 (예: 꽃잎·꽃받침 길이)
    values: list[float] = Field(min_length=4, max_length=4)

    # field_validator: 필드 하나를 직접 검사하는 함수 (ValueError 를 내면 422 오류)
    @field_validator("values")
    @classmethod
    def no_negative(cls, v: list[float]) -> list[float]:
        if any(x < 0 for x in v):
            raise ValueError("음수 값은 넣을 수 없어요")
        return [round(x, 2) for x in v]           # 검사 + 정리(반올림)까지


# 두 필드를 비교하려면 model_validator (모든 값이 들어온 뒤 실행)
class DateRange(BaseModel):
    start: int
    end: int

    @model_validator(mode="after")             # 여러 필드를 함께 검사
```

ex04_custom_validator.py

```
    def check_order(self):
        if self.start > self.end:
            raise ValueError("start 는 end 보다 클 수 없어요")
        return self


@app.post("/features")
def features(f: Features):
    return {"values": f.values, "sum": round(sum(f.values), 2)}


@app.post("/range")
def date_range(r: DateRange):
    return {"days": r.end - r.start}
```

한 줄씩 읽기

줄	코드	설명
8	`values: list[float] = Field(min_length=4, max_length=4)`	정확히 4개
11	`@field_validator("values")`	★ `values` 필드를 검사하는 함수 등록
12	`@classmethod`	Pydantic 검사 함수는 classmethod로 (첫 인자 `cls`)
15	`raise ValueError("음수 값은 넣을 수 없어요")`	ValueError를 일으키면 422 응답의 `msg`가 돼요.
16	`return [round(x, 2) for x in v] # 검사 + 정리(반올림)까지`	return 값이 실제 저장되는 값 → **정리**까지 할 수 있어요.
24	`@model_validator(mode="after") # 여러 필드를 함께 검사`	필드끼리의 관계(시작 ≤ 끝) 검사

예제 파일 `fastapi/02_Pydantic_요청과_응답/ex04_custom_validator_client.py`

ex04_custom_validator_client.py

```
# 클라이언트 예제: 앱(ex04_custom_validator.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex04_custom_validator import app          # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

show("정상", client.post("/features", json={"values": [5.123, 3.5, 1.4, 0.2]}))
# 직접 만든 검사 규칙에 걸리는 요청들
show("음수 포함", client.post("/features", json={"values": [5.1, -3.5, 1.4, 0.2]}))
show("개수 부족", client.post("/features", json={"values": [5.1, 3.5]}))
show("범위 정상", client.post("/range", json={"start": 3, "end": 10}))
show("순서 반대", client.post("/range", json={"start": 10, "end": 3}))
```

한 줄씩 읽기

줄	코드	설명
20	`show("정상", client.post("/features", json={"values": [5.123, 3.5, 1.4, 0.2]}))`	5.123 → 5.12로 정리돼서 응답
22	`show("음수 포함", client.post("/features", json={"values": [5.1, -3.5, 1.4, 0.2]}))`	우리가 만든 메시지 "음수 값은 ..."
25	`show("순서 반대", client.post("/range", json={"start": 10, "end": 3}))`	model_validator의 오류

▶ 실행 결과

```
[정상] POST /features → 200
{
  "values": [
    5.12,
    3.5,
    1.4,
    0.2
  ],
  "sum": 10.22
}

[음수 포함] POST /features → 422
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "values"
      ],
      "msg": "Value error, 음수 값은 넣을 수 없어요",
      "input": [
        5.1,
        -3.5,
        1.4,
        0.2
      ],
      "ctx": {
        "error": {}
      }
    }
  ]
}

[개수 부족] POST /features → 422
{
  "detail": [
    {
      "type": "too_short",
      "loc": [
        "body",
        "values"
      ],
      "msg": "List should have at least 4 items after validation, not 2",
      "input": [
        5.1,
        3.5
      ],
      "ctx": {
        "field_type": "List",
        "min_length": 4,
        "actual_length": 2
```

▶ 실행 결과

```
      }
    }
  ]
}

[범위 정상] POST /range → 200
{
  "days": 7
}

[순서 반대] POST /range → 422
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body"
      ],
      "msg": "Value error, start 는 end 보다 클 수 없어요",
      "input": {
        "start": 10,
        "end": 3
      },
      "ctx": {
        "error": {}
      }
    }
  ]
}
```

6. 자주 하는 실수와 정리

실수	결과	해결
GET 요청에 본문 모델 사용	브라우저·문서에서 보내기 어려움	본문은 POST/PUT/PATCH로
`client.post(..., data=...)`로 JSON 전송	422	`json=`
`Field(gt=0)`를 기본값 자리에 안 씀	검사 안 됨	`x: int = Field(gt=0)`
응답에 비밀번호·내부 값이 나감	정보 유출	`response_model`로 출력 모델 지정
validator에 `@classmethod` 빠뜨림	오류	`@field_validator` 아래 `@classmethod`
Pydantic v1 문법 (`@validator`, `.dict()`)	경고 또는 오류	v2: `@field_validator`, `.model_dump()`

개념	핵심
요청 본문	`class X(BaseModel)` + `def f(x: X)`
범위	`Field(gt, ge, lt, le, min_length, max_length)`
선택지	`Literal["a", "b"]`
중첩	`items: list[Item]`
응답	`response_model=`, `status_code=`
사용자 규칙	`@field_validator`, `@model_validator`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. `Review(BaseModel)`: `product: str`, `stars: int`(1 ~ 5), `comment: str`(최대 50자)를 정의하고 `POST /reviews`가 `{"product": ..., "stars": ...}`를 돌려주게 하세요. stars=7을 보내 상태 코드를 확인하세요.

▶ 이렇게 출력되면 성공

```
200 {'product': '마우스', 'stars': 5}
422
```

문제 2. `PredictRequest`(값 3개짜리 `features: list[float]`)를 받아 `PredictResponse`(`total: float`, `label: str`)로 응답하는 `POST /predict`를 만드세요. 합이 10 이상이면 label "high", 아니면 "low".

▶ 이렇게 출력되면 성공

```
{'total': 12.0, 'label': 'high'}
```

문제 3. `Signup` 모델에 `password`와 `password_confirm`을 받고, 두 값이 다르면 오류가 나도록 `model_validator`를 만드세요. 다른 값을 보냈을 때 오류 메시지(msg)를 출력하세요.

▶ 이렇게 출력되면 성공

```
Value error, 비밀번호가 서로 달라요
```

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

app = FastAPI()


class Review(BaseModel):
    product: str
    stars: int = Field(ge=1, le=5)
    comment: str = Field(default="", max_length=50)


@app.post("/reviews")
def create_review(r: Review):
    return {"product": r.product, "stars": r.stars}


client = TestClient(app)
res = client.post("/reviews", json={"product": "마우스", "stars": 5, "comment": "좋아요"})
print(res.status_code, res.json())
print(client.post("/reviews", json={"product": "마우스", "stars": 7}).status_code)
```

▶ 실행 결과

```
200 {'product': '마우스', 'stars': 5}
422
```

문제 2 정답

직접 입력해 보세요

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

app = FastAPI()


class PredictRequest(BaseModel):
    features: list[float] = Field(min_length=3, max_length=3)


class PredictResponse(BaseModel):
    total: float
    label: str


@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    total = sum(req.features)
    return {"total": total, "label": "high" if total >= 10 else "low"}


client = TestClient(app)
print(client.post("/predict", json={"features": [3, 4, 5]}).json())
```

▶ 실행 결과

```
{'total': 12.0, 'label': 'high'}
```

6장의 진짜 모델 서빙도 이 모양 그대로예요. "합 ≥ 10" 자리에 학습한 모델의 예측이 들어가요.

문제 3 정답

직접 입력해 보세요

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, model_validator

app = FastAPI()


class Signup(BaseModel):
    username: str
    password: str
    password_confirm: str

    @model_validator(mode="after")
    def same_password(self):
        if self.password != self.password_confirm:
            raise ValueError("비밀번호가 서로 달라요")
        return self


@app.post("/signup")
def signup(s: Signup):
    return {"username": s.username}


client = TestClient(app)
res = client.post("/signup", json={"username": "kim", "password": "abc12345", "password_confirm": "abc99999"})
print(res.json()["detail"][0]["msg"])
```

▶ 실행 결과

```
Value error, 비밀번호가 서로 달라요
```

Pydantic은 우리가 만든 메시지 앞에 "Value error, "를 붙여서 알려 줘요.

03장

CRUD API 만들기

만들기 · 읽기 · 수정 · 삭제를 갖춘 할 일(Todo) API와 라우터로 파일 나누기

항목	내용
예제 폴더	`fastapi/03_CRUD_API_만들기`
예제 파일	`ex01_todo_crud.py` + `_client.py`, `ex02_main.py` + `routers/items.py`, `routers/models.py` + `ex02_main_client.py`, `ex03_todo_web.py` + `todo_page.html` + `ex03_todo_web_client.py` (새로 추가)
이전 장	02 Pydantic 요청과 응답
예상 학습 시간	3 ~ 4시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- CRUD(Create · Read · Update · Delete)와 HTTP 메서드를 짝지을 수 있어요.
- `HTTPException`으로 404 같은 오류 응답을 보낼 수 있어요.
- PUT(전체 수정)과 PATCH(일부 수정)를 구분해 만들 수 있어요.
- 쿼리로 필터·페이지 나누기를 할 수 있어요.
- `APIRouter`로 API를 여러 파일로 나눌 수 있어요.
- `tags`, `summary`, `prefix`로 `/docs` 문서를 정리할 수 있어요.

도움말 | 이번 장의 구성

- 1. CRUD 와 HTTP 메서드 **[필수]**
- 2. 할 일 API 만들기 **[필수]**
- 3. HTTPException — 오류 응답 **[필수]**
- 4. PUT 과 PATCH **[필수]**
- 5. APIRouter 로 파일 나누기 **[심화]**
- 6. 웹 페이지에서 할 일 API 쓰기 **[필수]**
- 7. 자주 하는 실수와 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

게시판·쇼핑몰·할 일 앱처럼 **데이터를 만들고(Create) · 읽고(Read) · 고치고(Update) · 지우는(Delete)** 기능을 API 로 만들어요. 이 네 가지를 CRUD 라고 하고, 거의 모든 웹 서비스의 뼈대예요.

- [필수] 1~4절 : CRUD API, 404 오류, PUT 과 PATCH
- [필수] **6절(새로 추가)** : 만든 API 를 **웹 페이지 화면**에서 버튼으로 써 보기 — 웹 초보에게 가장 재미있는 절이에요
- [심화] 5절 : `APIRouter` 로 파일 나누기 — 프로젝트가 커질 때 필요해요

1. CRUD 와 HTTP 메서드 필수

> **비유로 이해하기 | 도서관 대출 시스템**
> 도서관 시스템은 책을 **등록**하고(Create), **찾아보고**(Read), 정보를 **고치고**(Update), 폐기하면 **지워요**(Delete). 거의 모든 서비스는 이 네 가지 동작의 조합이에요. 이것을 줄여서 **CRUD**라고 해요.

동작	HTTP 메서드	주소 예	성공 코드
Create 만들기	`POST`	`/todos`	201 Created
Read 목록	`GET`	`/todos?done=true`	200 OK
Read 하나	`GET`	`/todos/3`	200 OK
Update 전체	`PUT`	`/todos/3`	200 OK
Update 일부	`PATCH`	`/todos/3`	200 OK
Delete 삭제	`DELETE`	`/todos/3`	204 No Content

> **도움말 | 주소는 명사, 동작은 메서드로**
> `/createTodo`, `/deleteTodo/3`처럼 주소에 동작을 넣지 말고, `/todos`(명사, 복수형) + 메서드(POST, DELETE)로 표현하는 것이 REST API의 관례예요.

2. 할 일 API 만들기 필수

예제 파일 `fastapi/03_CRUD_API_만들기/ex01_todo_crud.py`
할 일(Todo) 목록을 관리하는 CRUD API예요.

ex01_todo_crud.py

```python
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="Todo API")


class TodoCreate(BaseModel):                # 만들 때 받는 값
    title: str = Field(min_length=1, max_length=50)
    priority: int = Field(default=3, ge=1, le=5)


class TodoUpdate(BaseModel):                # 일부 수정 (모두 선택)
    title: str | None = Field(default=None, min_length=1, max_length=50)
    priority: int | None = Field(default=None, ge=1, le=5)
    done: bool | None = None


class Todo(TodoCreate):                     # 저장·응답 모양 (상속으로 필드 재사용)
    id: int
    done: bool = False


# 실제 서비스는 데이터베이스를 쓰지만, 여기서는 사전에 저장해요 (서버를 끄면 사라져요)
DB: dict[int, Todo] = {}                    # 실습용 "메모리 데이터베이스"
next_id = 1


# 번호로 할 일을 찾고, 없으면 404 오류를 내는 도우미 함수 (여러 곳에서 재사용)
def get_or_404(todo_id: int) -> Todo:
```

ex01_todo_crud.py

```
    if todo_id not in DB:
        raise HTTPException(status_code=404, detail=f"{todo_id}번 할 일이 없어요")
    return DB[todo_id]


@app.post("/todos", response_model=Todo, status_code=status.HTTP_201_CREATED, tags=["todos"])
def create_todo(data: TodoCreate):           # C: 만들기
    # 다음 번호를 바꿔야 하니 global 로 함수 밖의 변수를 쓰겠다고 알리기
    global next_id
    # 받은 값 + 새 번호로 Todo 만들기 → 저장 → 번호 1 증가
    todo = Todo(id=next_id, **data.model_dump())
    DB[next_id] = todo
    next_id += 1
    return todo


@app.get("/todos", response_model=list[Todo], tags=["todos"])
def list_todos(done: bool | None = None, skip: int = 0, limit: int = 10):   # R: 목록 (필터 + 페이지)
    items = list(DB.values())
    # done 쿼리가 있으면 완료 여부로 걸러 내기
    if done is not None:
        items = [t for t in items if t.done == done]
    return items[skip: skip + limit]


@app.get("/todos/{todo_id}", response_model=Todo, tags=["todos"])
def read_todo(todo_id: int):                 # R: 하나
    return get_or_404(todo_id)


@app.put("/todos/{todo_id}", response_model=Todo, tags=["todos"])
def replace_todo(todo_id: int, data: TodoCreate):    # U: 전체 바꾸기
    # 기존 done 값은 유지하고 나머지를 새 값으로 통째로 바꾸기
    old = get_or_404(todo_id)
    DB[todo_id] = Todo(id=todo_id, done=old.done, **data.model_dump())
    return DB[todo_id]


@app.patch("/todos/{todo_id}", response_model=Todo, tags=["todos"])
def update_todo(todo_id: int, data: TodoUpdate):     # U: 보낸 값만 바꾸기
    old = get_or_404(todo_id)
    changes = data.model_dump(exclude_unset=True)    # 실제로 보낸 필드만
    DB[todo_id] = old.model_copy(update=changes)
    return DB[todo_id]


@app.delete("/todos/{todo_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["todos"])
def delete_todo(todo_id: int):               # D: 삭제
    # 204 No Content: 삭제 성공, 돌려줄 본문은 없어요
    get_or_404(todo_id)
    del DB[todo_id]
```

한 줄씩 읽기

줄	코드	설명
7	`class TodoCreate(BaseModel): # 만들 때 받는 값`	★ 용도별로 모델을 나눠요: 만들 때(TodoCreate), 일부 수정(TodoUpdate), 응답(Todo)
18	`class Todo(TodoCreate): # 저장·응답 모양 (상속으로 필드 재사용)`	Python 9장의 상속으로 title, priority를 물려받고 id, done 추가

줄	코드	설명
24	`DB: dict[int, Todo] = {} # 실습용 "메모리 데이터베이스"`	실습용 저장소. 서버를 끄면 사라져요 (실무에서는 진짜 데이터베이스)
31	`raise HTTPException(status_code=404, detail=f"{todo_id}번 할 일이 없어요")`	★ 3절: 원하는 상태 코드와 메시지로 오류 응답
40	`todo = Todo(id=next_id, **data.model_dump())`	**딕셔너리: 키=값 인자로 풀어 넣기
47	`def list_todos(done: bool | None = None, skip: int = 0, limit: int = 10): # R: 목록 (필터 + 페이지)`	쿼리로 필터(done)와 페이지(skip, limit)
71	`changes = data.model_dump(exclude_unset=True) # 실제로 보낸 필드만`	★ exclude_unset=True: **요청에 실제로 들어 있던** 필드만 → PATCH의 핵심
72	`DB[todo_id] = old.model_copy(update=changes)`	기존 객체를 복사하면서 바뀐 값만 덮어쓰기
76	`@app.delete("/todos/{todo_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["todos"])`	204: 성공했지만 돌려줄 내용이 없음
35	`@app.post("/todos", response_model=Todo, status_code=status.HTTP_201_CREATED, tags=["todos"])`	tags: /docs에서 묶어 보여 줄 그룹 이름

예제 파일 `fastapi/03_CRUD_API_만들기/ex01_todo_crud_client.py`

ex01_todo_crud_client.py

```python
# 클라이언트 예제: 앱(ex01_todo_crud.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                             # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex01_todo_crud import app            # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)                # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

# C(만들기) → R(읽기) → U(수정) → D(삭제) 순서로 시험
show("만들기 1", client.post("/todos", json={"title": "파이썬 복습", "priority": 2}))
show("만들기 2", client.post("/todos", json={"title": "판다스 과제"}))
show("만들기 3", client.post("/todos", json={"title": "모델 학습"}))
show("하나 읽기", client.get("/todos/2"))
show("일부 수정 (PATCH)", client.patch("/todos/2", json={"done": True}))
show("전체 수정 (PUT)", client.put("/todos/3", json={"title": "모델 학습 + 평가", "priority": 1}))
show("완료된 것만", client.get("/todos", params={"done": True}))
# DELETE 는 본문이 없어서 show 대신 상태 코드만 출력
res = client.delete("/todos/1")
print(f"[삭제] DELETE /todos/1 → {res.status_code} (본문 없음: {res.text!r})\n")
# 지운 것을 다시 읽으면 404
show("삭제한 것 읽기", client.get("/todos/1"))
show("전체 목록", client.get("/todos"))
```

한 줄씩 읽기

줄	코드	설명
22	`show("만들기 2", client.post("/todos", json={"title": "판다스 과제"}))`	priority를 안 보내면 기본값 3
25	`show("일부 수정 (PATCH)", client.patch("/todos/2", json={"done": True}))`	done만 바뀌고 title, priority는 그대로
26	`show("전체 수정 (PUT)", client.put("/todos/3", json={"title": "모델 학습 + 평가", "priority": 1}))`	title, priority를 통째로 교체
30	`print(f"[삭제] DELETE /todos/1 → {res.status_code} (본문 없음: {res.text!r})\n")`	204는 본문이 비어 있어서 `res.json()` 대신 `res.text`
32	`show("삭제한 것 읽기", client.get("/todos/1"))`	우리가 만든 404 메시지

▶ 실행 결과

```
[만들기 1] POST /todos → 201
{
  "title": "파이썬 복습",
  "priority": 2,
  "id": 1,
  "done": false
}

[만들기 2] POST /todos → 201
{
  "title": "판다스 과제",
  "priority": 3,
  "id": 2,
  "done": false
}

[만들기 3] POST /todos → 201
{
  "title": "모델 학습",
  "priority": 3,
  "id": 3,
  "done": false
}

[하나 읽기] GET /todos/2 → 200
{
  "title": "판다스 과제",
  "priority": 3,
  "id": 2,
  "done": false
}

[일부 수정 (PATCH)] PATCH /todos/2 → 200
{
  "title": "판다스 과제",
  "priority": 3,
  "id": 2,
  "done": true
}

[전체 수정 (PUT)] PUT /todos/3 → 200
{
  "title": "모델 학습 + 평가",
  "priority": 1,
  "id": 3,
  "done": false
}
```

▶ 실행 결과

```
[완료된 것만] GET /todos → 200
[
  {
    "title": "판다스 과제",
    "priority": 3,
    "id": 2,
    "done": true
  }
]

[삭제] DELETE /todos/1 → 204 (본문 없음: '')

[삭제한 것 읽기] GET /todos/1 → 404
{
  "detail": "1번 할 일이 없어요"
}

[전체 목록] GET /todos → 200
[
  {
    "title": "판다스 과제",
    "priority": 3,
    "id": 2,
    "done": true
  },
  {
    "title": "모델 학습 + 평가",
    "priority": 1,
    "id": 3,
    "done": false
  }
]
```

3. HTTPException — 오류 응답 필수

상태 코드	뜻	언제
400 Bad Request	요청이 잘못됨	규칙 위반 (재고보다 많이 주문)
401 Unauthorized	인증 필요	API 키·로그인 없음 (4장)
403 Forbidden	권한 없음	관리자만 가능
404 Not Found	없음	없는 id
409 Conflict	충돌	이미 있는 아이디로 가입
422 Unprocessable Entity	형식 오류	Pydantic 검사 실패 (자동)
500 Internal Server Error	서버 내부 오류	코드의 예상 못한 예외

HTTPException 사용법

```python
from fastapi import HTTPException

if item_id not in DB:
    raise HTTPException(status_code=404, detail="상품이 없어요")   # 함수가 여기서 멈추고 오류 응답
```

> **알아 두기 | return 이 아니라 raise**
>
> HTTPException은 Python 10장의 예외예요. `raise`하는 순간 함수가 멈추고 FastAPI가 `{"detail": "..."}` 응답으로 바꿔 줘요. 그래서 `get_or_404`처럼 **검사 함수 안에서** raise 해도 돼요.

4. PUT 과 PATCH 필수

	PUT	PATCH
뜻	**전체**를 새 값으로 바꾸기	보낸 **일부**만 바꾸기
요청 모델	모든 필드 필수 (`TodoCreate`)	모든 필드 선택 (`TodoUpdate`, 기본값 None)
핵심 코드	새 객체로 교체	`model_dump(exclude_unset=True)` + `model_copy(update=...)`
예	프로필 전체 저장	"완료" 체크만

> **주의 | PATCH 에서 exclude_unset 을 빠뜨리면**
> `model_dump()`만 쓰면 보내지 않은 필드도 `None`으로 들어가서 **기존 값이 None으로 지워져요.** 반드시 `exclude_unset=True`를 쓰세요.

5. APIRouter 로 파일 나누기 심화

API가 많아지면 한 파일이 너무 길어져요. `APIRouter`로 기능별 파일을 나누고, 메인 파일에서 모아요. 모델 서빙 프로젝트에서도 `/predict`, `/models`, `/health`를 파일별로 나눠요.

폴더 구조

```
03_CRUD_API_만들기/
├── ex02_main.py          ← app 을 만들고 라우터를 모으는 메인 파일
└── routers/
    ├── __init__.py       ← 폴더를 패키지로 (빈 파일)
    ├── items.py          ← /items 관련 API
    └── models.py         ← /models 관련 API
```

예제 파일 `fastapi/03_CRUD_API_만들기/routers/items.py`

items.py

```python
from fastapi import APIRouter, HTTPException

router = APIRouter(prefix="/items", tags=["items"])     # 이 파일의 주소는 모두 /items 로 시작

# 라우터에 붙인 함수들은 ex02_main.py 의 app 에 include_router 로 연결돼요
ITEMS = {1: {"name": "노트북", "price": 1200000}, 2: {"name": "마우스", "price": 15000}}


# "" = prefix 그대로 → GET /items
@router.get("", summary="상품 목록")
def list_items():
    return [{"id": k, **v} for k, v in ITEMS.items()]


@router.get("/{item_id}", summary="상품 하나")
def get_item(item_id: int):
    if item_id not in ITEMS:
        raise HTTPException(404, "상품이 없어요")
    return {"id": item_id, **ITEMS[item_id]}
```

한 줄씩 읽기

줄	코드	설명
3	`router = APIRouter(prefix="/items", tags=["items"])  # 이 파일의 주소는 모두 /items 로 시작`	★ `app` 대신 `router`. `prefix`가 모든 주소 앞에 붙어요.

줄	코드	설명
10	`@router.get("", summary="상품 목록")`	"" + prefix → /items. summary는 /docs에 보일 짧은 설명
18	`raise HTTPException(404, "상품이 없어요")`	상태 코드와 detail을 순서대로 넣어도 돼요.

예제 파일 fastapi/03_CRUD_API_만들기/routers/models.py

models.py

```python
from fastapi import APIRouter

# 모델 관련 주소를 모은 라우터: 모든 주소가 /models 로 시작
router = APIRouter(prefix="/models", tags=["models"])

# 서빙 중인 모델 목록 (실습용 고정 값)
MODELS = [{"name": "house-price", "version": "1.2.0"}, {"name": "iris-classifier", "version": "0.9.1"}]


# GET /models → 목록 그대로 JSON 으로
@router.get("", summary="서빙 중인 모델 목록")
def list_models():
    return MODELS
```

한 줄씩 읽기

줄	코드	설명
4	`router = APIRouter(prefix="/models", tags=["models"])`	서빙 중인 모델 정보 API (6장에서 실제로 사용)

예제 파일 fastapi/03_CRUD_API_만들기/ex02_main.py

ex02_main.py

```python
from fastapi import FastAPI
from routers import items, models          # routers 폴더의 두 파일

app = FastAPI(title="라우터로 나눈 API")
app.include_router(items.router)            # /items ...
app.include_router(models.router)           # /models ...


@app.get("/")
def root():
    return {"routes": sorted(app.openapi()["paths"])}   # 문서(OpenAPI)에 등록된 주소 목록
```

한 줄씩 읽기

줄	코드	설명
2	`from routers import items, models # routers 폴더의 두 파일`	패키지(폴더)에서 모듈 가져오기
5	`app.include_router(items.router) # /items ...`	★ 라우터를 앱에 등록
11	`return {"routes": sorted(app.openapi()["paths"])} # 문서(OpenAPI)에 등록된 주소 목록`	/docs를 만드는 OpenAPI 문서에서 등록된 주소 목록 읽기

예제 파일 `fastapi/03_CRUD_API_만들기/ex02_main_client.py`

ex02_main_client.py

```python
# 클라이언트 예제: 앱(ex02_main.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex02_main import app            # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

# 라우터 두 개의 주소가 모두 하나의 app 에서 동작하는지 확인
show("등록된 주소", client.get("/"))
show("상품 목록", client.get("/items"))
show("상품 하나", client.get("/items/2"))
show("없는 상품", client.get("/items/9"))
show("모델 목록", client.get("/models"))
```

한 줄씩 읽기

줄	코드	설명
21	`show("등록된 주소", client.get("/"))`	두 파일의 주소가 모두 등록됐어요.
24	`show("없는 상품", client.get("/items/9"))`	라우터 안의 HTTPException도 똑같이 동작

▶ 실행 결과

```
[등록된 주소] GET / → 200
{
  "routes": [
    "/",
    "/items",
    "/items/{item_id}",
    "/models"
  ]
}

[상품 목록] GET /items → 200
[
  {
    "id": 1,
    "name": "노트북",
    "price": 1200000
  },
  {
    "id": 2,
    "name": "마우스",
    "price": 15000
  }
]

[상품 하나] GET /items/2 → 200
{
```

▶ 실행 결과

```
  "id": 2,
  "name": "마우스",
  "price": 15000
}

[없는 상품] GET /items/9 → 404
{
  "detail": "상품이 없어요"
}

[모델 목록] GET /models → 200
[
  {
    "name": "house-price",
    "version": "1.2.0"
  },
  {
    "name": "iris-classifier",
    "version": "0.9.1"
  }
]
```

6. 웹 페이지에서 할 일 API 쓰기 필수

지금까지는 `/docs` 와 `TestClient` 로 API 를 불렀어요. 실제 서비스에서는 **웹 페이지(HTML + 자바스크립트)** 가 API 를 불러 화면을 그려요. 1절의 할 일 API 를 그대로 두고, 화면 파일(`todo_page.html`) 하나만 붙여 볼게요.

예제 파일 `fastapi/03_CRUD_API_만들기/ex03_todo_web.py`

ex03_todo_web.py

```python
# 웹 페이지에서 할 일 API 쓰기 — ex01_todo_crud 의 API 에 화면(HTML)을 붙여요 (서버로 실행)
# 실행: 이 폴더에서  uvicorn ex03_todo_web:app --reload
# 확인: 브라우저에서  http://127.0.0.1:8000  → 할 일을 추가·완료·삭제해 보세요
from pathlib import Path

from fastapi.responses import FileResponse

# ex01_todo_crud.py 의 앱(할 일 API 가 모두 들어 있는)을 그대로 가져와요
from ex01_todo_crud import app

HERE = Path(__file__).parent                    # 이 파일이 있는 폴더


# 주소 "/" 로 들어오면 JSON 대신 HTML 파일(웹 페이지)을 돌려줘요
# include_in_schema=False: /docs 문서에는 보이지 않게 (API 가 아니라 화면이니까요)
@app.get("/", include_in_schema=False)
def todo_page():
    return FileResponse(HERE / "todo_page.html")
```

줄	설명
`from ex01_todo_crud import app`	1절에서 만든 할 일 API 가 들어 있는 앱을 그대로 가져와요. API 코드는 한 줄도 고치지 않아요
`@app.get("/", include_in_schema=False)`	주소 `/` 에 화면용 창구를 하나 더 만들어요. `/docs` 문서에는 숨겨요
`return FileResponse(...)`	JSON 대신 **HTML 파일을 그대로** 돌려줘요. 브라우저는 이것을 웹 페이지로 그려요

예제 파일 fastapi/03_CRUD_API_만들기/todo_page.html

todo_page.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>할 일 목록</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 520px; margin: 40px auto; padding: 0 16px; background: #f5f7fb; }
    h1 { font-size: 24px; }
    form { display: flex; gap: 8px; margin-bottom: 16px; }
    input[type=text] { flex: 1; padding: 8px; font-size: 15px; }
    select, button { padding: 8px 12px; font-size: 15px; }
    ul { list-style: none; padding: 0; }
    li { display: flex; align-items: center; gap: 8px; background: white; padding: 10px; margin-bottom: 6px; border-radius: 6px; }
    li.done span { text-decoration: line-through; color: #999; }
    li span { flex: 1; }
    .pri { font-size: 12px; color: #c55a11; }
    #msg { color: #c00000; min-height: 20px; }
  </style>
</head>
<body>
  <h1>할 일 목록</h1>

  <!-- 입력 칸: 제목과 중요도를 적고 [추가] 를 누르면 POST /todos 요청 -->
  <form id="add-form">
    <input type="text" id="title" placeholder="할 일을 입력하세요">
    <select id="priority">
      <option value="1">중요도 1</option><option value="2">중요도 2</option>
      <option value="3" selected>중요도 3</option><option value="4">중요도 4</option>
      <option value="5">중요도 5</option>
    </select>
    <button type="submit">추가</button>
  </form>
  <p id="msg"></p>

  <!-- 할 일 목록이 여기에 그려져요 -->
  <ul id="list"></ul>

  <script>
    // fetch(주소, 옵션): 브라우저가 서버에 HTTP 요청을 보내는 함수 (TestClient 의 client.get / post 와 같은 역할)

    // R: 목록 가져오기 → GET /todos
    async function loadTodos() {
      const res = await fetch("/todos?limit=50");
      const todos = await res.json();              // 응답 JSON → 자바스크립트 배열
      const list = document.getElementById("list");
      list.innerHTML = "";                          // 화면을 비우고 다시 그리기
      for (const t of todos) {
        const li = document.createElement("li");
        if (t.done) li.className = "done";
        li.innerHTML = `
          <input type="checkbox" ${t.done ? "checked" : ""}>
          <span></span>
          <small class="pri">중요도 ${t.priority}</small>
          <button>삭제</button>`;
        li.querySelector("span").textContent = t.title;   // textContent: 글자를 그대로 넣기 (안전)
        li.querySelector("input").onchange = () => toggleDone(t);
```

todo_page.html

```
        li.querySelector("button").onclick = () => deleteTodo(t.id);
        list.appendChild(li);
      }
    }

    // C: 만들기 → POST /todos (본문에 JSON)
    document.getElementById("add-form").onsubmit = async (event) => {
      event.preventDefault();                        // 폼의 기본 동작(페이지 새로고침) 막기
      const res = await fetch("/todos", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          title: document.getElementById("title").value,
          priority: Number(document.getElementById("priority").value),
        }),
      });
      const msg = document.getElementById("msg");
      if (res.status === 201) {
        msg.textContent = "";
        document.getElementById("title").value = "";
        loadTodos();
      } else {
        // 422: 제목이 비었거나 너무 길 때 — 서버(Pydantic)가 알려 준 이유를 보여 줘요
        const err = await res.json();
        msg.textContent = `추가 실패(${res.status}): ${err.detail[0].msg}`;
      }
    };

    // U: 완료 표시 바꾸기 → PATCH /todos/{id} (보낸 값만 수정)
    async function toggleDone(todo) {
      await fetch(`/todos/${todo.id}`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ done: !todo.done }),
      });
      loadTodos();
    }

    // D: 삭제 → DELETE /todos/{id}
    async function deleteTodo(id) {
      await fetch(`/todos/${id}`, { method: "DELETE" });
      loadTodos();
    }

    loadTodos();                                     // 페이지가 열리면 목록부터 불러오기
  </script>
</body>
</html>
```

화면에서 하는 일	자바스크립트	보내는 HTTP 요청	CRUD
페이지 열기 · 목록 다시 그리기	`fetch("/todos?limit=50")`	`GET /todos`	Read
[추가] 버튼	`fetch("/todos", {method: "POST", body: ...})`	`POST /todos` + JSON 본문	Create
체크박스	`fetch("/todos/" + id, {method: "PATCH"})`	`PATCH /todos/1`	Update
[삭제] 버튼	`fetch("/todos/" + id, {method: "DELETE"})`	`DELETE /todos/2`	Delete

알아 두기 | fetch 는 자바스크립트의 TestClient

`fetch(주소, 옵션)` 은 브라우저가 서버에 HTTP 요청을 보내는 함수예요. 파이썬의 `client.post("/todos", json=...)` 와 같은 일을 해요. `await res.json()` 은 응답 JSON 을 자바스크립트 값으로 바꾸는 것으로, 파이썬의 `res.json()` 과 같아요. HTML·자바스크립트 문법은 HTML·CSS 과정에서 자세히 다뤄요.

직접 해 보기 | 서버를 켜고 화면에서 써 보기

- 이 폴더에서 `uvicorn ex03_todo_web:app --reload` 로 서버를 켜고 브라우저에서 `http://127.0.0.1:8000` 에 접속해요.
- 할 일을 몇 개 추가하고, 체크박스를 눌러 보고, 삭제해 보세요.
- 제목을 비운 채 [추가] 를 누르면 Pydantic 이 막아서 422 가 오고, 화면에 이유가 표시돼요.
- 다른 탭에서 `http://127.0.0.1:8000/todos` 를 열면 화면에 보이는 목록이 JSON 그대로 보여요. 화면은 이 JSON 을 그려 준 것뿐이에요.
- 서버를 껐다 켜면 목록이 사라져요. 메모리(딕셔너리)에 저장했기 때문이에요. 실제 서비스는 데이터베이스에 저장해요.



http://127.0.0.1:8000 — 할 일 3개를 추가하고 첫 번째를 완료 표시한 뒤, 빈 제목으로 [추가] 를 눌러 422 메시지가 나온 화면

웹 페이지가 보내는 요청을 코드로 똑같이 보내 보면, 화면 뒤에서 무슨 일이 일어나는지 한눈에 보여요.

예제 파일 `fastapi/03_CRUD_API_만들기/ex03_todo_web_client.py`

ex03_todo_web_client.py

```
# 클라이언트 예제: 웹 페이지가 하는 요청을 코드로 똑같이 보내 봐요 (python 으로 실행)
# 브라우저의 fetch(...) 한 줄 = TestClient 의 client.get/post/patch/delete 한 줄
from fastapi.testclient import TestClient

from ex03_todo_web import app

client = TestClient(app)

# 1) 페이지 열기: "/" 는 JSON 이 아니라 HTML 을 돌려줘요
res = client.get("/")
print("페이지:", res.status_code, res.headers["content-type"], "| 제목 포함?", "<title>할 일 목록</title>"
in res.text)

# 2) [추가] 버튼 → POST /todos
```

ex03_todo_web_client.py

```
print("추가:", client.post("/todos", json={"title": "FastAPI 복습", "priority": 2}).status_code)
print("추가:", client.post("/todos", json={"title": "웹 페이지 만들기"}).status_code)
print("빈 제목:", client.post("/todos", json={"title": ""}).status_code, "→ 화면에 '추가 실패' 표시")

# 3) 체크박스 → PATCH /todos/1
print("완료 표시:", client.patch("/todos/1", json={"done": True}).json())

# 4) [삭제] 버튼 → DELETE /todos/2
print("삭제:", client.delete("/todos/2").status_code)

# 5) 페이지가 다시 그리는 목록 → GET /todos
print("현재 목록:", client.get("/todos").json())
```

▶ 실행 결과

```
페이지: 200 text/html; charset=utf-8 | 제목 포함? True
추가: 201
추가: 201
빈 제목: 422 → 화면에 '추가 실패' 표시
완료 표시: {'title': 'FastAPI 복습', 'priority': 2, 'id': 1, 'done': True}
삭제: 204
현재 목록: [{'title': 'FastAPI 복습', 'priority': 2, 'id': 1, 'done': True}]
```

도움말 | 같은 서버에서 화면을 주면 CORS 걱정이 없어요

화면(HTML)과 API 가 같은 주소(`127.0.0.1:8000`)에서 나오면 브라우저가 그대로 허용해요. 화면을 다른 서버(예: `localhost:3000`)에서 열면 브라우저가 보안상 막는데, 이때 필요한 설정이 4장의 **CORS** 예요.

7. 자주 하는 실수와 정리

실수	결과	해결
`/todos/{id}` 보다 `/todos/done` 을 아래에 정의	"done"을 id로 해석해 422	고정 주소를 **먼저** 정의
PATCH에서 `model_dump()`만 사용	안 보낸 값이 None으로 덮어씀	`exclude_unset=True`
204 응답에 return 값	본문이 무시되거나 오류	204는 return 없이
`global` 빠뜨리고 next_id 증가	`UnboundLocalError`	`global next_id` (Python 8장)
routers 폴더 import 실패	`ModuleNotFoundError`	메인 파일이 있는 폴더에서 실행, `__init__.py` 확인

개념	핵심
CRUD	POST / GET / PUT·PATCH / DELETE
오류	`raise HTTPException(status_code, detail)`
모델 나누기	Create / Update(전부 선택) / 응답 모델
PATCH	`model_dump(exclude_unset=True)` + `model_copy(update=)`
파일 나누기	`APIRouter(prefix, tags)` + `app.include_router()`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. 메모리 딕셔너리로 **회원(name, email)** 을 저장하는 `POST /members`(201)와 `GET /members/{id}`(없으면 404)를 만들고, 만든 뒤 1번과 99번을 조회해 상태 코드를 출력하세요.

▶ 이렇게 출력되면 성공

```
201
200 {'id': 1, 'name': 'kim', 'email': 'kim@test.com'}
404 {'detail': '회원이 없어요'}
```

문제 2. 재고가 `{"apple": 5}`인 상점에서 `POST /buy?item=apple&qty=8`처럼 재고보다 많이 사면 **400** 과 "재고 부족" 메시지를 돌려주는 API를 만드세요.

▶ 이렇게 출력되면 성공

```
200 {'item': 'apple', 'left': 2}
400 {'detail': '재고 부족 (남은 수량 2)'}
```

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel

app = FastAPI()
MEMBERS: dict[int, dict] = {}


class MemberIn(BaseModel):
    name: str
    email: str


@app.post("/members", status_code=201)
def create(m: MemberIn):
    new_id = len(MEMBERS) + 1
    MEMBERS[new_id] = {"id": new_id, **m.model_dump()}
    return MEMBERS[new_id]


@app.get("/members/{member_id}")
def read(member_id: int):
    if member_id not in MEMBERS:
        raise HTTPException(404, "회원이 없어요")
    return MEMBERS[member_id]


client = TestClient(app)
print(client.post("/members", json={"name": "kim", "email": "kim@test.com"}).status_code)
r = client.get("/members/1"); print(r.status_code, r.json())
r = client.get("/members/99"); print(r.status_code, r.json())
```

▶ 실행 결과

```
201
200 {'id': 1, 'name': 'kim', 'email': 'kim@test.com'}
404 {'detail': '회원이 없어요'}
```

문제 2 정답

직접 입력해 보세요

```python
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient

app = FastAPI()
STOCK = {"apple": 5}


@app.post("/buy")
def buy(item: str, qty: int):
    if item not in STOCK:
        raise HTTPException(404, "없는 상품")
    if qty > STOCK[item]:
        raise HTTPException(400, f"재고 부족 (남은 수량 {STOCK[item]})")
    STOCK[item] -= qty
```

직접 입력해 보세요

```
    return {"item": item, "left": STOCK[item]}


client = TestClient(app)
r = client.post("/buy?item=apple&qty=3"); print(r.status_code, r.json())
r = client.post("/buy?item=apple&qty=8"); print(r.status_code, r.json())
```

▶ 실행 결과

```
200 {'item': 'apple', 'left': 2}
400 {'detail': '재고 부족 (남은 수량 2)'}
```

입력 형식은 맞지만(그래서 422가 아님) **업무 규칙**에 어긋나면 400을 쓰는 것이 일반적이에요.

04장

의존성 · 비동기 · 수명주기 · 미들웨어

공통 기능을 재사용하고, 서버가 켜질 때 모델을 한 번만 불러오는 구조 만들기

항목	내용
예제 폴더	`fastapi/04_의존성_비동기_수명주기`
예제 파일	`ex01_depends.py` ~ `ex05_settings.py` + 각각의 `_client.py`
이전 장	03 CRUD API 만들기
예상 학습 시간	4시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- `Depends`로 공통 매개변수·검사 로직을 재사용할 수 있어요.
- 헤더의 API 키로 간단한 인증을 만들 수 있어요.
- `async def`와 `await`의 차이, 언제 쓰는지 알아요.
- `lifespan`으로 서버 시작 시 **모델을 한 번만** 불러오고 종료 시 정리할 수 있어요.
- 미들웨어로 모든 요청의 처리 시간을 기록할 수 있어요.
- 환경 변수로 설정을 바꿀 수 있고, CORS가 무엇인지 알아요.

도움말 | 이번 장의 구성

- 1. Depends — 의존성 주입 **[필수]**
- 2. API 키 인증 **[필수]**
- 3. async def 와 await **[심화]**
- 4. lifespan — 시작할 때 모델 불러오기 **[필수]**
- 5. 미들웨어와 CORS · 6. 설정과 환경 변수 **[심화]**
- 7. 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

API 가 많아지면 "여러 API 가 똑같이 하는 준비 작업" 이 생겨요. 이 장은 그런 공통 작업을 깔끔하게 정리하는 도구 모음이에요. 내용이 많으니 **[필수] 절만 먼저** 하세요.

- [필수] 1절 `Depends` — 공통 준비 작업을 함수 하나로 만들어 여러 API 에서 쓰기
- [필수] 2절 API 키 — 헤더로 "출입증" 검사하기 (1절의 응용)
- [필수] 4절 `lifespan` — 서버가 켜질 때 **모델을 한 번만** 불러오기 (6장 모델 서빙의 핵심)
- [심화] 3절 `async`/`await`, 5·6절 미들웨어 · CORS · 환경 변수 — 개념만 읽고 넘어가도 6장까지 따라갈 수 있어요

1. Depends — 의존성 주입 필수

비유로 이해하기 | 식당의 공용 주방 도구

여러 요리에 같은 도마와 칼을 쓰듯, 여러 API가 같은 준비 작업(페이지 계산, 로그인 확인, DB 연결, 모델 꺼내기)을 해야 할 때가 많아요. 이 준비 작업을 함수로 만들어 두고 "이 API는 이 준비가 필요해요"라고 **선언만** 하면 FastAPI가 대신 실행해 결과를 넣어 줘요. 이것을 **의존성 주입(Dependency Injection)** 이라고 해요.

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex01_depends.py`

ex01_depends.py

```
1  from typing import Annotated
2  # Depends: "이 함수를 먼저 실행해서 그 결과를 넣어 줘" 라는 표시 / Query: 쿼리 값의 범위 정하기
3  from fastapi import Depends, FastAPI, Query
4
5  app = FastAPI()
6
7
8  # skip: 건너뛸 개수(0 이상), limit: 가져올 개수(1 ~ 50)
9  def pagination(skip: int = Query(0, ge=0), limit: int = Query(5, ge=1, le=50)):
10     return {"skip": skip, "limit": limit}        # 여러 API 가 함께 쓰는 쿼리 묶음
11
12
13 Page = Annotated[dict, Depends(pagination)]      # 별명을 붙여 짧게 쓰기
14
15 # 1 ~ 100 숫자 목록 (실습용 데이터)
16 NUMBERS = list(range(1, 101))
17
18
19 @app.get("/numbers")
20 def numbers(page: Page):                          # FastAPI 가 pagination() 을 먼저 실행해 결과를 넣어 줌
21     # 의존성 결과에서 skip, limit 꺼내기
22     s, l = page["skip"], page["limit"]
23     return {"page": page, "data": NUMBERS[s: s + l]}
24
25
26 @app.get("/squares")
27 def squares(page: Page):                          # 같은 의존성 재사용
28     s, l = page["skip"], page["limit"]
29     return {"page": page, "data": [n * n for n in NUMBERS[s: s + l]]}
```

한 줄씩 읽기

줄	코드	설명
9	`def pagination(skip: int = Query(0, ge=0), limit: int = Query(5, ge=1, le=50)):`	여러 API가 함께 쓸 쿼리 매개변수 묶음. `Query(기본값, 검사)`로 범위 지정
13	`Page = Annotated[dict, Depends(pagination)] # 별명을 붙여 짧게 쓰기`	★ `Annotated[타입, Depends(함수)]`: "이 매개변수는 pagination()의 결과로 채워 줘"
20	`def numbers(page: Page): # FastAPI 가 pagination() 을 먼저 실행해 결과를 넣어 줌`	요청이 오면 ① pagination 실행(쿼리 검사 포함) ② 결과를 page에 넣고 ③ numbers 실행
27	`def squares(page: Page): # 같은 의존성 재사용`	한 번 만든 의존성을 여러 API에서 재사용

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex01_depends_client.py`

ex01_depends_client.py

```python
# 클라이언트 예제: 앱(ex01_depends.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                             # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex01_depends import app            # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)                # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

show("기본값", client.get("/numbers"))
show("페이지 지정", client.get("/squares?skip=10&limit=3"))
# limit=100 은 le=50 을 넘어서 422 오류
show("limit 범위 초과", client.get("/numbers?limit=100"))
```

한 줄씩 읽기

줄	코드	설명
23	`show("limit 범위 초과", client.get("/numbers?limit=100"))`	의존성 안의 검사(`le=50`)도 422로 동작

▶ 실행 결과

```
[기본값] GET /numbers → 200
{
  "page": {
    "skip": 0,
    "limit": 5
  },
  "data": [
    1,
    2,
    3,
    4,
    5
  ]
}

[페이지 지정] GET /squares → 200
{
  "page": {
    "skip": 10,
    "limit": 3
  },
  "data": [
    121,
    144,
    169
  ]
}

[limit 범위 초과] GET /numbers → 422
```

▶ 실행 결과

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "query",
        "limit"
      ],
      "msg": "Input should be less than or equal to 50",
      "input": "100",
      "ctx": {
        "le": 50
      }
    }
  ]
}
```

2. API 키 인증 필수

모델 API를 아무나 쓰면 서버 비용이 늘고 악용될 수 있어요. 가장 간단한 보호 방법은 **API 키**를 요청 **헤더**에 담아 보내게 하는 것이에요.

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex02_api_key.py`

ex02_api_key.py

```python
from typing import Annotated
# Header(): 요청 헤더에서 값 읽기 (이름 x_api_key → 헤더 X-API-Key)
from fastapi import Depends, FastAPI, Header, HTTPException

app = FastAPI()
API_KEYS = {"team-a-123": "팀A", "team-b-456": "팀B"}     # 실제로는 환경 변수·DB 에 보관


def verify_key(x_api_key: Annotated[str | None, Header()] = None) -> str:   # 헤더 X-API-Key 읽기
    # 키가 없으면 401(인증 필요), 틀리면 403(권한 없음)
    if x_api_key is None:
        raise HTTPException(status_code=401, detail="X-API-Key 헤더가 필요해요")
    if x_api_key not in API_KEYS:
        raise HTTPException(status_code=403, detail="유효하지 않은 API 키예요")
    # 통과하면 팀 이름을 돌려줘요 → predict 의 team 변수로 들어가요
    return API_KEYS[x_api_key]


# 키 검사가 없는 공개 API
@app.get("/public")
def public():
    return {"message": "누구나 볼 수 있어요"}


@app.post("/predict")
def predict(team: Annotated[str, Depends(verify_key)]):     # 키 검사를 통과해야 실행
    return {"team": team, "prediction": "OK"}
```

한 줄씩 읽기

줄	코드	설명
9	`def verify_key(x_api_key: Annotated[str | None, Header()] = None) -> str: # 헤더 X-API-Key 읽기`	★ `Header()`: 요청 헤더에서 읽기. 매개변수 `x_api_key`는 헤더 `X-API-Key`와 자동으로 짝지어져요 (밑줄 ↔ 하이픈).

줄	코드	설명
12	`raise HTTPException(status_code=401, detail="X-API-Key 헤더가 필요해요")`	키가 없으면 401 (인증 필요)
14	`raise HTTPException(status_code=403, detail="유효하지 않은 API 키예요")`	틀린 키면 403 (권한 없음)
26	`def predict(team: Annotated[str, Depends(verify_key)]): # 키 검사를 통과해야 실행`	의존성이 예외를 일으키면 **API 함수는 실행조차 안 돼요.**

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex02_api_key_client.py`

ex02_api_key_client.py

```python
# 클라이언트 예제: 앱(ex02_api_key.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex02_api_key import app            # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

show("공개 API", client.get("/public"))
show("키 없음", client.post("/predict"))
# headers= 로 요청 헤더를 붙여 보내기
show("틀린 키", client.post("/predict", headers={"X-API-Key": "wrong"}))
show("올바른 키", client.post("/predict", headers={"X-API-Key": "team-a-123"}))
```

한 줄씩 읽기

줄	코드	설명
24	`show("올바른 키", client.post("/predict", headers={"X-API-Key": "team-a-123"}))`	`headers=`로 헤더 보내기

▶ 실행 결과

```
[공개 API] GET /public → 200
{
  "message": "누구나 볼 수 있어요"
}

[키 없음] POST /predict → 401
{
  "detail": "X-API-Key 헤더가 필요해요"
}

[틀린 키] POST /predict → 403
{
  "detail": "유효하지 않은 API 키예요"
}

[올바른 키] POST /predict → 200
{
```

▶ 실행 결과

```
  "team": "팀A",
  "prediction": "OK"
}
```

> **주의 | API 키를 코드에 적지 마세요**
> 예제는 이해를 돕기 위해 코드에 키를 적었지만, 실제로는 **환경 변수**(6절)나 비밀 관리 도구에 보관하고, GitHub에 올리지 않아요. 더 강한 인증이 필요하면 OAuth2/JWT를 사용해요.

3. async def 와 await 심화

> **심화 · 선택 | 처음 배우는 중이라면 이 절은 건너뛰어도 돼요**
> 결론만 기억하세요: **잘 모르겠으면 그냥 `def` 로 쓴다.** FastAPI 가 알아서 동시에 여러 요청을 처리해 줘요. `async def` 는 외부 API·데이터베이스처럼 "기다리는 일" 이 많을 때 성능을 더 끌어올리는 방법이에요.

> **비유로 이해하기 | 카페 직원 한 명**
> 직원이 커피 머신이 다 내릴 때까지 **멍하니 기다리면**(동기), 그동안 다음 손님 주문을 못 받아요. **기다리는 동안 다른 주문을 받는 것**(비동기)이 `async`/`await`예요. 핵심은 "기다리는 일"이 많을 때 효과가 있다는 거예요.

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex03_async.py`

ex03_async.py

```python
# time.sleep: 기다리는 동안 그 일꾼(스레드)이 멈춰요 / asyncio.sleep: 기다리는 동안 다른 요청을 처리할 수 있어요
import asyncio
import time
from fastapi import FastAPI

app = FastAPI()


@app.get("/sync")
def sync_job():                         # 일반 함수: FastAPI 가 별도 스레드에서 실행
    time.sleep(0.2)                     # 계산·모델 예측처럼 CPU 를 쓰는 일이라고 생각
    return {"type": "sync", "slept": 0.2}


@app.get("/async")
async def async_job():                  # 비동기 함수
    await asyncio.sleep(0.2)            # 다른 서버 응답·파일·DB 를 "기다리는" 일
    return {"type": "async", "slept": 0.2}


async def fetch(name: str, sec: float) -> str:
    await asyncio.sleep(sec)            # 외부 API 를 기다린다고 가정
    return f"{name} 완료"


@app.get("/gather")
async def gather():
    # 걸린 시간 재기 시작
    start = time.perf_counter()
    results = await asyncio.gather(fetch("날씨", 0.3), fetch("환율", 0.3), fetch("뉴스", 0.3))  # 동시에 기다리기
    # 세 일을 하나씩 하면 0.9초, 동시에 기다리면 약 0.3초
    took = time.perf_counter() - start
    return {"results": results, "under_0_5s": took < 0.5}
```

한 줄씩 읽기

줄	코드	설명
10	`def sync_job(): # 일반 함수: FastAPI 가 별도 스레드에서 실행`	일반 `def`도 괜찮아요. FastAPI가 따로 실행해 줘서 서버 전체가 멈추지는 않아요.
16	`async def async_job(): # 비동기 함수`	`async def`: 비동기 함수
17	`await asyncio.sleep(0.2) # 다른 서버 응답·파일·DB 를 "기다리는" 일`	★ `await`: "기다리는 동안 다른 요청을 처리해도 돼"
30	`results = await asyncio.gather(fetch("날씨", 0.3), fetch("환율", 0.3), fetch("뉴스", 0.3)) # 동시에 기다리기`	세 개를 **동시에** 기다려서 0.9초가 아니라 약 0.3초
33	`return {"results": results, "under_0_5s": took < 0.5}`	실행 시간은 컴퓨터마다 달라서 "0.5초 미만인가?"만 확인

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex03_async_client.py`

ex03_async_client.py

```python
# 클라이언트 예제: 앱(ex03_async.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex03_async import app           # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

show("일반 함수", client.get("/sync"))
show("비동기 함수", client.get("/async"))
show("세 가지를 동시에 기다리기", client.get("/gather"))
```

한 줄씩 읽기

줄	코드	설명
22	`show("세 가지를 동시에 기다리기", client.get("/gather"))`	`under_0_5s: true` — 동시에 기다렸다는 증거

▶ 실행 결과

```
[일반 함수] GET /sync → 200
{
  "type": "sync",
  "slept": 0.2
}

[비동기 함수] GET /async → 200
{
  "type": "async",
  "slept": 0.2
}
```

▶ 실행 결과

```
[세 가지를 동시에 기다리기] GET /gather → 200
{
  "results": [
    "날씨 완료",
    "환율 완료",
    "뉴스 완료"
  ],
  "under_0_5s": true
}
```

상황	추천
외부 API 호출, DB, 파일처럼 **기다리는** 일 (비동기 라이브러리 사용)	`async def` + `await`
모델 예측, 이미지 처리처럼 **CPU 를 쓰는** 계산	그냥 `def` (FastAPI가 스레드에서 실행)
잘 모르겠으면	`def` — 안전한 기본값

> **주의 | async def 안에서 time.sleep 은 금지**
>
> `async def` 안에서 `time.sleep()`이나 무거운 계산을 하면 **서버 전체가 멈춰요.** async 함수 안에서는 `await`할 수 있는 것만 쓰거나, 무거운 일은 일반 `def` 함수로 만드세요.

4. lifespan — 시작할 때 모델 불러오기 필수

딥러닝 모델 파일은 수백 MB ~ 수 GB라서 불러오는 데 몇 초씩 걸려요. 요청마다 불러오면 매우 느리겠죠. **서버가 켜질 때 한 번만** 불러와 두고 모든 요청이 함께 쓰는 것이 모델 서빙의 기본 구조예요.

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex04_lifespan.py`

ex04_lifespan.py

```python
import time
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request

# 서버가 켜지고 꺼질 때 일어난 일을 기록할 목록
LOG: list[str] = []


def load_model():
    time.sleep(0.3)                                  # 큰 모델 파일을 읽는 데 시간이 걸린다고 가정
    LOG.append("모델 불러옴")
    # 가짜 모델: predict 는 x 를 받아 2배를 돌려주는 함수(lambda)
    return {"name": "demo-model", "version": "1.0", "predict": lambda x: x * 2}


# lifespan: 서버 시작 · 종료 때 할 일을 한 함수에 (yield 앞 = 시작, 뒤 = 종료)
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = load_model()                   # ① 서버 시작: 한 번만 실행
    LOG.append("서버 시작")
    yield                                            # ② 이 동안 요청을 처리
    app.state.model = None                           # ③ 서버 종료: 정리
    LOG.append("서버 종료 (모델 해제)")


app = FastAPI(lifespan=lifespan)


@app.get("/predict")
```

ex04_lifespan.py

```
30 def predict(x: float, request: Request):
31     model = request.app.state.model                    # 미리 불러 둔 모델 사용 (매번 불러오지 않음)
32     return {"model": model["name"], "x": x, "y": model["predict"](x)}
33
34
35 # 지금까지의 기록 보기
36 @app.get("/log")
37 def log():
38     return LOG
```

한 줄씩 읽기

줄	코드	설명
9	`def load_model():`	모델 불러오기를 흉내 내는 함수 (6장에서 진짜 모델로 바꿔요)
17	`@asynccontextmanager`	`yield` 앞뒤로 시작/종료 코드를 나누는 도구
19	`app.state.model = load_model() # ① 서버 시작: 한 번만 실행`	★ `app.state`: 앱 전체가 공유하는 보관함에 모델 저장
21	`yield # ② 이 동안 요청을 처리`	서버가 요청을 받는 동안
22	`app.state.model = None # ③ 서버 종료: 정리`	종료할 때 자원 정리 (GPU 메모리 해제 등)
26	`app = FastAPI(lifespan=lifespan)`	lifespan 등록
31	`model = request.app.state.model # 미리 불러 둔 모델 사용 (매번 불러오지 않음)`	`Request` 객체로 앱의 state에 접근

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex04_lifespan_client.py`

ex04_lifespan_client.py

```
1  import json
2  from fastapi.testclient import TestClient
3  # 이 장의 앱과 LOG 목록을 함께 불러오기
4  import ex04_lifespan
5  from ex04_lifespan import app
6
7  with TestClient(app) as client:                # with 블록 = 서버가 켜져 있는 동안
8      # 숫자 3개로 예측 요청 → 모델은 서버 시작 때 한 번만 불러와요
9      for x in [1, 2.5, 10]:
10         print(client.get("/predict", params={"x": x}).json())
11     print("요청 3번 후 로그:", client.get("/log").json())
12
13 # with 블록을 나오면 서버가 꺼지고, lifespan 의 yield 뒤가 실행돼요
14 print("서버 종료 후 로그:", ex04_lifespan.LOG)
```

한 줄씩 읽기

줄	코드	설명
7	`with TestClient(app) as client: # with 블록 = 서버가 켜져 있는 동안`	★ TestClient를 `with`로 쓰면 lifespan의 시작·종료가 실제로 실행돼요.
11	`print("요청 3번 후 로그:", client.get("/log").json())`	요청을 3번 했지만 "모델 불러옴"은 **한 번**
14	`print("서버 종료 후 로그:", ex04_lifespan.LOG)`	with 블록이 끝나면 종료 코드 실행

▶ 실행 결과

```
{'model': 'demo-model', 'x': 1.0, 'y': 2.0}
{'model': 'demo-model', 'x': 2.5, 'y': 5.0}
{'model': 'demo-model', 'x': 10.0, 'y': 20.0}
요청 3번 후 로그: ['모델 불러옴', '서버 시작']
서버 종료 후 로그: ['모델 불러옴', '서버 시작', '서버 종료 (모델 해제)']
```

알아 두기 | 옛날 방식 @app.on_event

인터넷 예제의 `@app.on_event("startup")`은 예전 방식이에요. 지금은 `lifespan`을 쓰는 것이 권장돼요.

5. 미들웨어와 CORS · 6. 설정과 환경 변수 심화

심화 · 선택 | 처음 배우는 중이라면 이 절은 개념만 읽어도 돼요

미들웨어·CORS·환경 변수는 서비스를 **배포하고 운영할 때** 필요한 설정이에요. 아래 표의 뜻만 알고 넘어가도 5~7장을 따라가는 데 문제없어요. 3장 6절처럼 화면과 API 를 같은 서버에서 주면 CORS 설정 없이도 동작해요.

기능	설명
미들웨어	**모든 요청과 응답**이 지나가는 관문. 처리 시간 기록, 로그, 공통 헤더 추가
CORS	브라우저는 보안상 **다른 주소**의 웹페이지가 API를 부르는 것을 막아요. 프론트엔드(예: `localhost:3000`)에서 호출하려면 허용 목록에 넣어야 해요.
환경 변수	코드를 고치지 않고 설정(모델 버전, 키, 허용 주소)을 바꾸는 방법. `os.getenv("이름", "기본값")`

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex05_middleware_settings.py`

ex05_middleware_settings.py

```python
import os
import time
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware

# os.getenv(이름, 기본값): 환경 변수 읽기 → 코드를 고치지 않고 설정 바꾸기
APP_NAME = os.getenv("APP_NAME", "my-model-server")            # 환경 변수, 없으면 기본값
MODEL_VERSION = os.getenv("MODEL_VERSION", "1.0.0")
# ALLOWED_ORIGINS="주소1,주소2" 처럼 쉼표로 여러 개
ALLOWED = os.getenv("ALLOWED_ORIGINS", "http://localhost:3000").split(",")

app = FastAPI(title=APP_NAME)

app.add_middleware(CORSMiddleware, allow_origins=ALLOWED,       # 다른 주소의 웹페이지에서 호출 허용
                   allow_methods=["*"], allow_headers=["*"])


@app.middleware("http")
async def add_process_time(request: Request, call_next):       # 모든 요청을 지나가는 관문
    start = time.perf_counter()
    response = await call_next(request)                        # 실제 API 실행
    # 걸린 시간(ms)과 모델 버전을 응답 헤더에 붙이기
    ms = (time.perf_counter() - start) * 1000
    response.headers["X-Process-Time-ms"] = f"{ms:.1f}"
    response.headers["X-Model-Version"] = MODEL_VERSION
    return response


@app.get("/info")
def info():
```

ex05_middleware_settings.py

```
    return {"app": APP_NAME, "model_version": MODEL_VERSION, "allowed_origins": ALLOWED}


# 일부러 50ms 기다리는 API (처리 시간 헤더 확인용)
@app.get("/slow")
def slow():
    time.sleep(0.05)
    return {"done": True}
```

한 줄씩 읽기

줄	코드	설명
8	`MODEL_VERSION = os.getenv("MODEL_VERSION", "1.0.0")`	★ 환경 변수가 있으면 그 값, 없으면 기본값
14	`app.add_middleware(CORSMiddleware, allow_origins=ALLOWED, # 다른 주소의 웹페이지에서 호출 허용`	허용할 출처(웹페이지 주소) 목록
18	`@app.middleware("http")`	★ 직접 만드는 미들웨어
21	`response = await call_next(request) # 실제 API 실행`	앞쪽은 요청 전, 뒤쪽은 응답 후
24	`response.headers["X-Process-Time-ms"] = f"{ms:.1f}"`	모든 응답에 처리 시간 헤더 추가 — 모델이 얼마나 빨리 응답하는지 측정할 때 유용

예제 파일 `fastapi/04_의존성_비동기_수명주기/ex05_middleware_settings_client.py`

ex05_middleware_settings_client.py

```
import os
os.environ["MODEL_VERSION"] = "2.1.0"          # 앱을 불러오기 "전에" 환경 변수 설정 (터미널의 set/export 와 같음)

from fastapi.testclient import TestClient
from ex05_middleware_settings import app

client = TestClient(app)
# 응답 본문과 미들웨어가 붙인 헤더 확인
res = client.get("/info")
print(res.json())
print("헤더 X-Model-Version:", res.headers["X-Model-Version"])

res = client.get("/slow")
print("처리 시간 50ms 이상?", float(res.headers["X-Process-Time-ms"]) >= 50)

# OPTIONS 요청: 브라우저가 "이 출처에서 불러도 되나요?" 하고 먼저 물어보는 요청(CORS 사전 요청)
pre = client.options("/info", headers={"Origin": "http://localhost:3000", "Access-Control-Request-Method": "GET"})
print("허용된 출처 CORS:", pre.status_code, pre.headers.get("access-control-allow-origin"))
bad = client.options("/info", headers={"Origin": "http://evil.com", "Access-Control-Request-Method": "GET"})
print("허용 안 된 출처 CORS:", bad.status_code, bad.headers.get("access-control-allow-origin"))
```

한 줄씩 읽기

줄	코드	설명
2	`os.environ["MODEL_VERSION"] = "2.1.0" # 앱을 불러오기 "전에" 환경 변수 설정 (터미널의 set/export 와 같음)`	앱 코드는 그대로인데 버전이 2.1.0으로 바뀌어요.
14	`print("처리 시간 50ms 이상?", float(res.headers["X-Process-Time-ms"]) >= 50)`	미들웨어가 붙인 헤더 확인

줄	코드	설명
17	`pre = client.options("/info", headers={"Origin": "http://localhost:3000", "Access-Control-Request-Method": "GET"})`	브라우저가 보내는 사전 확인(preflight) 요청을 흉내
19	`bad = client.options("/info", headers={"Origin": "http://evil.com", "Access-Control-Request-Method": "GET"})`	허용 목록에 없는 출처 → 허용 헤더가 없어요(400).

▶ 실행 결과

```
{'app': 'my-model-server', 'model_version': '2.1.0', 'allowed_origins': ['http://localhost:3000']}
헤더 X-Model-Version: 2.1.0
처리 시간 50ms 이상? True
허용된 출처 CORS: 200 http://localhost:3000
허용 안 된 출처 CORS: 400 None
```

터미널에서 환경 변수 주고 실행하기

```
# Windows (명령 프롬프트)
set MODEL_VERSION=2.1.0
uvicorn ex05_middleware_settings:app

# macOS / Linux
MODEL_VERSION=2.1.0 uvicorn ex05_middleware_settings:app
```

7. 정리

기능	코드
의존성	`Annotated[타입, Depends(함수)]`
헤더	`Annotated[str | None, Header()]` → `X-Api-Key` 헤더
비동기	`async def` + `await`, 동시에: `asyncio.gather()`
수명주기	`@asynccontextmanager async def lifespan(app): ... yield ...` → `FastAPI(lifespan=lifespan)`
공유 객체	`app.state.model`, `request.app.state.model`
미들웨어	`@app.middleware("http")`, `app.add_middleware(CORSMiddleware, ...)`
설정	`os.getenv("이름", "기본값")`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. 쿼리 `lang`(기본 "ko")을 읽는 의존성 `get_lang`을 만들고, `/hello`와 `/bye` 두 API가 이를 함께 써서 언어별 인사를 돌려주게 하세요.

▶ 이렇게 출력되면 성공

```
{'msg': '안녕하세요'}
{'msg': 'Goodbye'}
```

문제 2. lifespan에서 `app.state.counter = 0`으로 시작하고, `/visit`를 부를 때마다 1씩 늘려 돌려주세요. 세 번 호출한 결과를 출력하세요.

▶ 이렇게 출력되면 성공

```
1
2
```

▶ 이렇게 출력되면 성공

```
3
```

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
from typing import Annotated
from fastapi import Depends, FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


def get_lang(lang: str = "ko") -> str:
    return lang if lang in ("ko", "en") else "ko"


Lang = Annotated[str, Depends(get_lang)]


@app.get("/hello")
def hello(lang: Lang):
    return {"msg": "안녕하세요" if lang == "ko" else "Hello"}


@app.get("/bye")
def bye(lang: Lang):
    return {"msg": "안녕히 가세요" if lang == "ko" else "Goodbye"}


client = TestClient(app)
print(client.get("/hello").json())
print(client.get("/bye?lang=en").json())
```

▶ 실행 결과

```
{'msg': '안녕하세요'}
{'msg': 'Goodbye'}
```

문제 2 정답

직접 입력해 보세요

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
from fastapi.testclient import TestClient


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.counter = 0
    yield


app = FastAPI(lifespan=lifespan)


@app.get("/visit")
def visit(request: Request):
    request.app.state.counter += 1
    return {"count": request.app.state.counter}

```

직접 입력해 보세요

```
20
21 with TestClient(app) as client:
22     for _ in range(3):
23         print(client.get("/visit").json()["count"])
```

▶ 실행 결과

```
1
2
3
```

05장

파일 업로드와 이미지 처리

이미지를 받아 검사하고, 딥러닝 모델이 먹을 수 있는 배열로 바꿉니다

항목	내용
예제 폴더	`fastapi/05_파일업로드와_이미지`
예제 파일	`ex00_make_samples.py`, 앱 `ex01_upload.py` ~ `ex03_preprocess.py` + 각각의 `_client.py`
준비	`pip install python-multipart pillow numpy`
이전 장	04 의존성 · 비동기 · 수명주기
예상 학습 시간	3 ~ 4시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- `UploadFile`로 파일 하나·여러 개를 받을 수 있어요.
- 파일 종류(content type)와 크기를 검사하고 알맞은 오류를 돌려줄 수 있어요.
- Pillow로 업로드한 이미지의 크기·모드를 확인하고 바꿀 수 있어요.
- 처리한 이미지를 응답으로 돌려줄 수 있어요.
- 이미지를 **딥러닝 입력 배열**(크기 조정 → 정규화 → `(1, H, W, C)`)로 바꿀 수 있어요.
- TestClient로 파일 업로드 요청을 보낼 수 있어요.

도움말 | 이번 장의 구성

- 1. 왜 파일 업로드가 필요할까? **[필수]**
- 2. 실습 이미지 만들기 **[필수]**
- 3. UploadFile 기본 **[필수]**
- 4. 이미지 검사와 처리 **[필수]**
- 5. 딥러닝 입력으로 전처리 **[심화]**
- 6. 자주 하는 실수와 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

지금까지는 글자(JSON)만 주고받았어요. 이 장에서는 **사진 같은 파일을 서버로 올리고(업로드)**, 서버에서 크기·종류를 검사하고 바꿔 보는 방법을 배워요. 6장의 "사진을 보내면 AI 가 맞히는 API" 의 준비 단계예요.

- [필수] 1~4절 : 파일 업로드, 종류·크기 검사, 이미지 처리 후 돌려주기
- [심화] 5절 : 이미지를 딥러닝 모델 입력(숫자 배열)로 바꾸기 — NumPy 를 배웠다면 도전해 보세요
- `/docs` 에서 파일 업로드도 시험할 수 있어요: Try it out 을 누르면 **[파일 선택]** 버튼이 생겨요.

1. 왜 파일 업로드가 필요할까? 필수

비유로 이해하기 | 사진을 보여 주며 묻기

"이 사진 속 동물이 뭐예요?"라고 물으려면 사진을 **건네줘야** 하죠. 이미지 분류, 얼굴 인식, 문서 OCR처럼 딥러닝 서비스의 대부분은 **파일(이미지·음성·문서)을 받아서** 처리해요.

단계	하는 일	이 장의 예제
① 받기	`UploadFile`로 파일 받기	ex01
② 검사	종류·크기·진짜 이미지인지	ex02
③ 처리	Pillow로 크기·색 바꾸기	ex02
④ 전처리	모델 입력 배열로 (크기 조정, 정규화, 차원 추가)	ex03
⑤ 예측	모델에 넣고 결과 응답	6장

설치 (python-multipart: 파일 업로드 처리에 필요)

```
pip install python-multipart pillow numpy
```

2. 실습 이미지 만들기 필수

예제 파일 `fastapi/05_파일업로드와_이미지/ex00_make_samples.py`
저장소에 이미 `samples/` 폴더가 있으니 다시 실행하지 않아도 돼요.

ex00_make_samples.py

```
# 실습용 이미지 만들기 (samples 폴더)
from pathlib import Path
from PIL import Image, ImageDraw

# 이미지를 저장할 samples 폴더
OUT = Path(__file__).parent / "samples"
OUT.mkdir(exist_ok=True)

# 빨강 · 초록 · 파랑 원 그림 3장
for name, color in [("red", (220, 40, 40)), ("green", (40, 180, 70)), ("blue", (40, 80, 220))]:
    # 흰 바탕 320×240 그림 + 그리기 도구
    img = Image.new("RGB", (320, 240), "white")
    d = ImageDraw.Draw(img)
    d.ellipse((60, 30, 260, 210), fill=color)          # 색이 칠해진 원
    img.save(OUT / f"{name}_circle.png")

# 흑백 숫자 그림 (MNIST 처럼 검은 바탕 흰 글씨)
digit = Image.new("L", (200, 200), 0)                  # 검은 바탕 흑백(L) 이미지
ImageDraw.Draw(digit).rectangle((85, 30, 115, 170), fill=255)   # 숫자 1 모양
digit.save(OUT / "digit_one.png")

# 이미지가 아닌 파일도 하나 (오류 시험용)
(OUT / "note.txt").write_text("이미지가 아닌 파일\n두 번째 줄\n", encoding="utf-8")
# 만든 파일과 크기 출력
for p in sorted(OUT.iterdir()):
    print(f"{p.name:18} {p.stat().st_size:>6} bytes")
```

한 줄씩 읽기

줄	코드	설명
12	`img = Image.new("RGB", (320, 240), "white")`	Pillow로 320×240 흰 바탕 컬러 이미지

줄	코드	설명
14	`d.ellipse((60, 30, 260, 210), fill=color) # 색이 칠해진 원`	원 그리기 (왼쪽 위, 오른쪽 아래 좌표)
18	`digit = Image.new("L", (200, 200), 0) # 검은 바탕 흑백(L) 이미지`	`L`: 흑백(밝기 0 ~ 255) 모드 — 손글씨 숫자 인식(MNIST) 모델의 입력과 비슷
23	`(OUT / "note.txt").write_text("이미지가 아닌 파일\n두 번째 줄\n", encoding="utf-8")`	검사용 텍스트 파일

▶ 실행 결과

```
blue_circle.png      1566 bytes
digit_one.png         326 bytes
green_circle.png     1566 bytes
note.txt               44 bytes
red_circle.png       1562 bytes
```

3. UploadFile 기본 필수

예제 파일 `fastapi/05_파일업로드와_이미지/ex01_upload.py`

ex01_upload.py

```python
# UploadFile: 업로드된 파일 (이름 · 종류 · 내용)
from fastapi import FastAPI, File, UploadFile

app = FastAPI()


@app.post("/upload")
async def upload(file: UploadFile):                  # 파일 하나
    data = await file.read()                         # 파일 내용 (bytes)
    # 받은 파일의 이름 · 종류 · 크기를 돌려주기
    return {"filename": file.filename, "content_type": file.content_type, "size": len(data)}


@app.post("/upload/text-lines")
async def count_lines(file: UploadFile):
    text = (await file.read()).decode("utf-8")        # bytes → 문자열
    # 줄 단위로 나누기
    lines = text.splitlines()
    return {"filename": file.filename, "lines": len(lines), "first_line": lines[0] if lines else None}


@app.post("/upload/many")
async def upload_many(files: list[UploadFile] = File(...)):   # 여러 파일
    return [{"filename": f.filename, "size": len(await f.read())} for f in files]
```

한 줄씩 읽기

줄	코드	설명
8	`async def upload(file: UploadFile): # 파일 하나`	★ 매개변수 타입을 `UploadFile`로 하면 업로드 파일을 받아요 (multipart/form-data 형식).
9	`data = await file.read() # 파일 내용 (bytes)`	`await file.read()`: 내용을 bytes로. UploadFile의 read는 비동기라 `async def` + `await`
11	`return {"filename": file.filename, "content_type": file.content_type, "size": len(data)}`	파일 이름, 종류(MIME), 크기

줄	코드	설명
16	`text = (await file.read()).decode("utf-8") # bytes → 문자열`	텍스트 파일은 decode로 문자열
23	`async def upload_many(files: list[UploadFile] = File(...)): # 여러 파일`	여러 파일은 `list[UploadFile]`

예제 파일 `fastapi/05_파일업로드와_이미지/ex01_upload_client.py`

ex01_upload_client.py

```
# 클라이언트 예제: 앱(ex01_upload.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex01_upload import app           # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)              # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

from pathlib import Path
# 실습 파일이 있는 samples 폴더
S = Path(__file__).parent / "samples"

# files= {"폼 이름": (파일 이름, 내용, 종류)} 로 업로드
with open(S / "red_circle.png", "rb") as f:                    # "rb": 바이너리로 읽기
    show("파일 하나", client.post("/upload", files={"file": ("red_circle.png", f, "image/png")}))

show("텍스트 줄 수", client.post("/upload/text-lines",
     files={"file": ("note.txt", (S / "note.txt").read_bytes(), "text/plain")}))

# 여러 파일: 같은 이름("files")으로 여러 개를 리스트로
many = [("files", (p.name, p.read_bytes(), "image/png")) for p in sorted(S.glob("*_circle.png"))]
show("여러 파일", client.post("/upload/many", files=many))
# 파일 없이 보내면 422
show("파일 없이 요청", client.post("/upload"))
```

한 줄씩 읽기

줄	코드	설명
25	`with open(S / "red_circle.png", "rb") as f: # "rb": 바이너리로 읽기`	이미지는 글자가 아니라서 바이너리 모드로
26	`show("파일 하나", client.post("/upload", files={"file": ("red_circle.png", f, "image/png")}))`	★ `files={"매개변수이름": (파일이름, 내용, 종류)}`
32	`many = [("files", (p.name, p.read_bytes(), "image/png")) for p in sorted(S.glob("*_circle.png"))]`	여러 파일은 (이름, 튜플) 쌍의 **리스트**
35	`show("파일 없이 요청", client.post("/upload"))`	필수 파일이 없으면 422

▶ 실행 결과

```
[파일 하나] POST /upload → 200
{
  "filename": "red_circle.png",
  "content_type": "image/png",
  "size": 1562
}

[텍스트 줄 수] POST /upload/text-lines → 200
{
  "filename": "note.txt",
  "lines": 2,
  "first_line": "이미지가 아닌 파일"
}

[여러 파일] POST /upload/many → 200
[
  {
    "filename": "blue_circle.png",
    "size": 1566
  },
  {
    "filename": "green_circle.png",
    "size": 1566
  },
  {
    "filename": "red_circle.png",
    "size": 1562
  }
]

[파일 없이 요청] POST /upload → 422
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "file"
      ],
      "msg": "Field required",
      "input": null
    }
  ]
}
```

도움말 | curl 로 업로드하기

서버를 켜고 터미널에서 `curl -F "file=@samples/red_circle.png" http://127.0.0.1:8000/upload` — `-F`는 폼(파일) 전송, `@`는 파일 경로라는 뜻이에요. `/docs`에서도 파일 선택 버튼으로 시험할 수 있어요.

4. 이미지 검사와 처리 필수

검사	방법	실패 시 코드
파일 종류	`file.content_type in {"image/png", "image/jpeg"}`	415 Unsupported Media Type
파일 크기	`len(data) > MAX_BYTES`	413 Content Too Large
진짜 이미지인지	`Image.open()` 실패 → `UnidentifiedImageError`	400 Bad Request

예제 파일 fastapi/05_파일업로드와_이미지/ex02_image.py

ex02_image.py

```python
import io
from fastapi import FastAPI, HTTPException, UploadFile
from fastapi.responses import Response
from PIL import Image, UnidentifiedImageError

app = FastAPI()
# 허용하는 이미지 종류와 최대 크기
ALLOWED = {"image/png", "image/jpeg"}
MAX_BYTES = 2 * 1024 * 1024                                 # 2MB


# 업로드 파일 → PIL 이미지 (세 가지 검사를 차례로)
async def read_image(file: UploadFile) -> Image.Image:
    if file.content_type not in ALLOWED:                    # ① 종류 검사
        raise HTTPException(415, f"PNG/JPEG 만 가능해요 (받은 종류: {file.content_type})")
    data = await file.read()
    if len(data) > MAX_BYTES:                               # ② 크기 검사
        raise HTTPException(413, "파일이 너무 커요 (최대 2MB)")
    try:
        return Image.open(io.BytesIO(data))                 # ③ 진짜 이미지인지
    except UnidentifiedImageError:
        raise HTTPException(400, "이미지 파일을 읽을 수 없어요")


# 이미지 정보: 형식 · 색 모드 · 크기
@app.post("/image/info")
async def image_info(file: UploadFile):
    img = await read_image(file)
    return {"filename": file.filename, "format": img.format, "mode": img.mode, "width": img.width,
"height": img.height}


# 썸네일 만들기 → PNG 바이트로 저장 → 이미지 그대로 응답
@app.post("/image/thumbnail")
async def thumbnail(file: UploadFile, size: int = 64, gray: bool = False):
    img = await read_image(file)
    img.thumbnail((size, size))                             # 비율을 지키며 줄이기
    if gray:
        img = img.convert("L")                              # 흑백
    buf = io.BytesIO()
    img.save(buf, format="PNG")
    return Response(content=buf.getvalue(), media_type="image/png")   # 이미지 자체를 응답
```

한 줄씩 읽기

줄	코드	설명
13	`async def read_image(file: UploadFile) -> Image.Image:`	★ 검사를 함수 하나로 모아 여러 API에서 재사용
15	`raise HTTPException(415, f"PNG/JPEG 만 가능해요 (받은 종류: {file.content_type})")`	종류 검사
20	`return Image.open(io.BytesIO(data)) # ③ 진짜 이미지인지`	`io.BytesIO`: bytes를 파일처럼 다루게 해 주는 상자
21	`except UnidentifiedImageError:`	확장자만 .png인 가짜 파일 막기 — content_type은 보내는 쪽이 마음대로 정할 수 있어서 **내용까지** 확인해야 안전해요.
36	`img.thumbnail((size, size)) # 비율을 지키며 줄이기`	가로세로 비율을 유지하며 size 안에 맞게

줄	코드	설명
41	`return Response(content=buf.getvalue(), media_type="image/png") # 이미지 자체를 응답`	★ JSON 대신 **이미지 bytes**를 응답 (브라우저에서 바로 보여요)

예제 파일 `fastapi/05_파일업로드와_이미지/ex02_image_client.py`

ex02_image_client.py

```python
# 클라이언트 예제: 앱(ex02_image.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                          # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex02_image import app           # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)             # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

from pathlib import Path
from PIL import Image
import io
S = Path(__file__).parent / "samples"
# png("파일 이름"): 업로드용 files 사전을 만들어 주는 짧은 함수
png = lambda name: {"file": (name, (S / name).read_bytes(), "image/png")}

show("이미지 정보", client.post("/image/info", files=png("blue_circle.png")))
show("흑백 이미지 정보", client.post("/image/info", files=png("digit_one.png")))
# 오류 경우: 이미지가 아닌 종류(415), 이름만 png 인 가짜 파일(400)
show("텍스트 파일", client.post("/image/info", files={"file": ("note.txt", b"hello", "text/plain")}))
show("가짜 PNG", client.post("/image/info", files={"file": ("fake.png", b"not image", "image/png")}))

# 썸네일은 JSON 이 아니라 이미지 자체가 와요 → 바이트를 다시 이미지로 열어 확인 · 저장
res = client.post("/image/thumbnail?size=80&gray=true", files=png("green_circle.png"))
print("썸네일 응답:", res.status_code, res.headers["content-type"], len(res.content), "bytes")
thumb = Image.open(io.BytesIO(res.content))
print("받은 이미지:", thumb.size, thumb.mode)
thumb.save(Path(__file__).parent / "images" / "thumb_green.png")
```

한 줄씩 읽기

줄	코드	설명
31	`show("가짜 PNG", client.post("/image/info", files={"file": ("fake.png", b"not image", "image/png")}))`	종류는 image/png라고 속였지만 내용 검사에서 400
34	`res = client.post("/image/thumbnail?size=80&gray=true", files=png("green_circle.png"))`	파일 + 쿼리 매개변수를 함께
36	`thumb = Image.open(io.BytesIO(res.content))`	응답 이미지 bytes(`res.content`)를 다시 이미지로
38	`thumb.save(Path(__file__).parent / "images" / "thumb_green.png")`	받은 썸네일을 저장 — 아래 그림이 실제 응답 이미지예요.

▶ 실행 결과

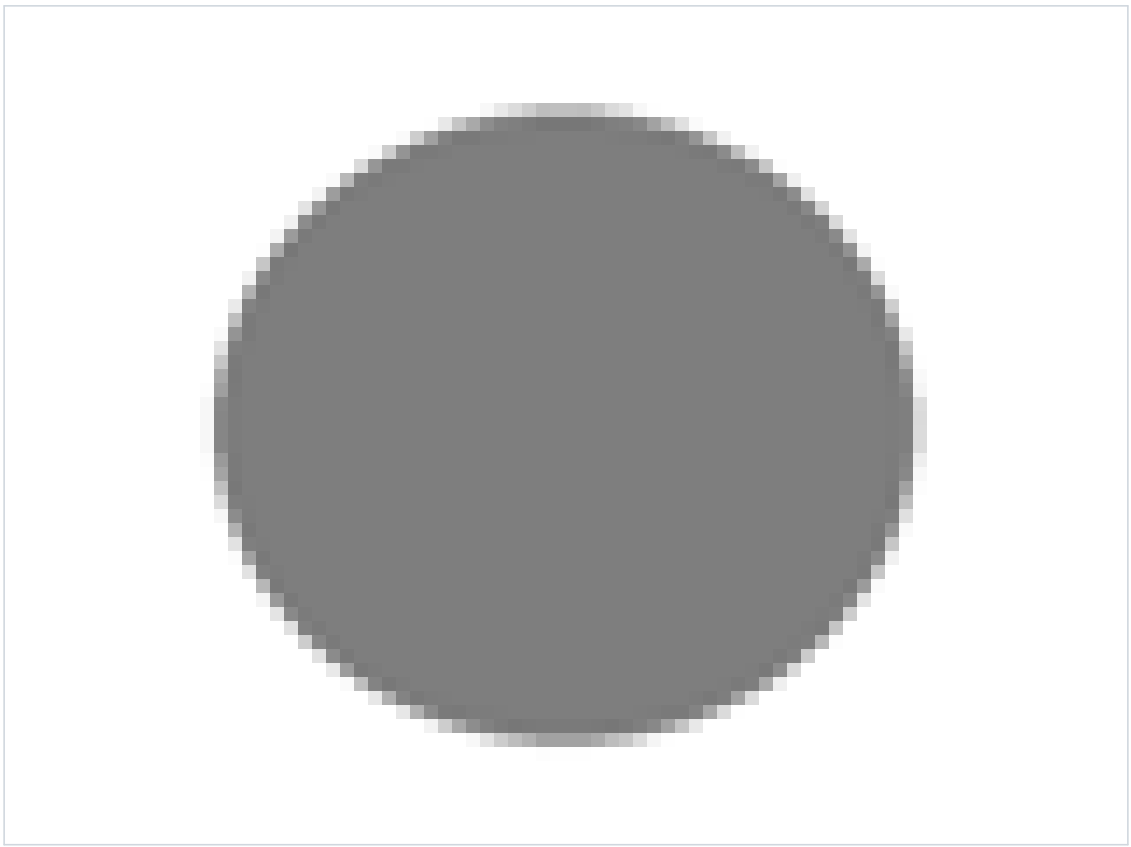
```
[이미지 정보] POST /image/info → 200
{
  "filename": "blue_circle.png",
  "format": "PNG",
  "mode": "RGB",
  "width": 320,
  "height": 240
}

[흑백 이미지 정보] POST /image/info → 200
{
  "filename": "digit_one.png",
  "format": "PNG",
  "mode": "L",
  "width": 200,
  "height": 200
}

[텍스트 파일] POST /image/info → 415
{
  "detail": "PNG/JPEG 만 가능해요 (받은 종류: text/plain)"
}

[가짜 PNG] POST /image/info → 400
{
  "detail": "이미지 파일을 읽을 수 없어요"
}

썸네일 응답: 200 image/png 700 bytes
받은 이미지: (80, 60) L
```



▲ 실행 결과 그래프: images/thumb_green.png

5. 딥러닝 입력으로 전처리 심화

> **심화 · 선택 | 심화 · NumPy 배열을 배운 뒤에 보세요**
> 이 절은 이미지를 `(1, 28, 28, 1)` 같은 숫자 배열로 바꾸는 내용이에요. NumPy 과정을 아직 하지 않았다면 6장의 [필수] 절(1~5절)로 바로 넘어가도 돼요.

딥러닝 모델은 학습할 때와 **똑같은 모양과 범위**의 입력을 받아야 해요. 예를 들어 손글씨 숫자 모델(MNIST)은 28×28 흑백, 값 0 ~ 1, 모양 `(배치, 28, 28, 1)`을 기대해요. 업로드된 이미지는 크기도 색도 제각각이라서 **전처리**가 꼭 필요해요.

단계	코드	모양 변화 (예: 200×200 흑백)
① 색 모드	`img.convert("L")`	컬러/흑백 → 흑백
② 크기	`img.resize((28, 28))`	200×200 → 28×28
③ 배열 + 정규화	`np.asarray(img, dtype=np.float32) / 255.0`	`(28, 28)`, 값 0 ~ 1
④ 채널 차원	`arr[..., np.newaxis]`	`(28, 28, 1)`
⑤ 배치 차원	`arr[np.newaxis, ...]`	`(1, 28, 28, 1)`

예제 파일 `fastapi/05_파일업로드와_이미지/ex03_preprocess.py`

ex03_preprocess.py

```python
import io
import numpy as np
from fastapi import FastAPI, HTTPException, UploadFile
from PIL import Image

app = FastAPI()


def to_model_input(img: Image.Image, size=(28, 28), gray=True) -> np.ndarray:
    """딥러닝 모델 입력 모양으로 바꾸기: (1, 높이, 너비, 채널), 값 0 ~ 1"""
    img = img.convert("L" if gray else "RGB")     # ① 색 모드 통일
    img = img.resize(size)                        # ② 모델이 학습한 크기로
    arr = np.asarray(img, dtype=np.float32) / 255.0   # ③ 0 ~ 255 → 0 ~ 1 (정규화)
    if gray:
        arr = arr[..., np.newaxis]                # (28, 28) → (28, 28, 1)
    return arr[np.newaxis, ...]                   # ④ 배치 차원 추가 → (1, 28, 28, 1)


@app.post("/preprocess")
async def preprocess(file: UploadFile, size: int = 28, gray: bool = True):
    # 업로드 → PIL 이미지 (읽을 수 없으면 400)
    try:
        img = Image.open(io.BytesIO(await file.read()))
    except Exception:
        raise HTTPException(400, "이미지를 읽을 수 없어요")
    # 모델 입력으로 바꾸기
    x = to_model_input(img, (size, size), gray)
    # 바뀐 결과를 숫자로 확인: 모양 · 자료형 · 최소/최대/평균 · 가운데 줄의 값 일부
    return {
        "original": [img.width, img.height, img.mode],
        "input_shape": list(x.shape),
        "dtype": str(x.dtype),
        "min": round(float(x.min()), 3), "max": round(float(x.max()), 3),
        "mean": round(float(x.mean()), 4),
        "center_row": [round(float(v), 2) for v in x[0, size // 2, ::4, 0]] if gray else None,
    }
```

한 줄씩 읽기

줄	코드	설명
9	`def to_model_input(img: Image.Image, size=(28, 28), gray=True) -> np.ndarray:`	★ 학습 때 쓴 전처리와 **똑같아야** 해요. 그래서 함수로 만들어 학습 코드와 서빙 코드가 함께 쓰는 것이 좋아요.
13	`arr = np.asarray(img, dtype=np.float32) / 255.0 # ③ 0 ~ 255 → 0 ~ 1 (정규화)`	Pillow 이미지 → NumPy 배열 (NumPy 1장), 255로 나눠 정규화
15	`arr = arr[..., np.newaxis] # (28, 28) → (28, 28, 1)`	`np.newaxis`: 크기 1인 차원 추가 (NumPy 4장 reshape와 같은 효과)
16	`return arr[np.newaxis, ...] # ④ 배치 차원 추가 → (1, 28, 28, 1)`	모델은 여러 장을 한 번에 받도록 만들어져서, 1장이어도 배치 차원이 필요해요.
35	`"center_row": [round(float(v), 2) for v in x[0, size // 2, ::4, 0]] if gray else None,`	가운데 줄의 값을 4칸마다 — 숫자 1의 흰 막대 부분이 1.0 근처로 보여요.
33	`"min": round(float(x.min()), 3), "max": round(float(x.max()), 3),`	NumPy 값은 `float()`로 바꿔야 JSON으로 보낼 수 있어요.

예제 파일 `fastapi/05_파일업로드와_이미지/ex03_preprocess_client.py`

ex03_preprocess_client.py

```python
# 클라이언트 예제: 앱(ex03_preprocess.py)에 실제로 요청을 보내고 응답을 출력해요 (python 으로 실행)
import json                             # 응답(JSON)을 보기 좋게 출력할 때 사용
from fastapi.testclient import TestClient
from ex03_preprocess import app             # 같은 폴더의 앱 파일에서 app 가져오기

client = TestClient(app)                # 서버를 띄우지 않고 요청을 보내 볼 수 있는 도구


# show(제목, 응답): 요청 방식 · 주소 · 상태 코드와 응답 내용을 출력하는 도우미 함수
def show(title, res):
    print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
    # 응답 본문이 JSON 이면 들여쓰기해서 출력 (ensure_ascii=False: 한글을 그대로)
    try:
        print(json.dumps(res.json(), ensure_ascii=False, indent=2))
    # JSON 이 아니면(글자 응답 등) 그대로 출력
    except ValueError:
        print(res.text)
    print()

from pathlib import Path
S = Path(__file__).parent / "samples"
png = lambda name: {"file": (name, (S / name).read_bytes(), "image/png")}

# 흑백 숫자 그림은 MNIST 처럼 28×28 흑백, 컬러 그림은 224×224 RGB 로
show("숫자 이미지 → 28x28 흑백", client.post("/preprocess", files=png("digit_one.png")))
show("컬러 이미지 → 224x224 RGB", client.post("/preprocess?size=224&gray=false",
files=png("red_circle.png")))
```

한 줄씩 읽기

줄	코드	설명
25	`show("숫자 이미지 → 28x28 흑백", client.post("/preprocess", files=png("digit_one.png")))`	`input_shape: [1, 28, 28, 1]`, 값 범위 0 ~ 1
26	`show("컬러 이미지 → 224x224 RGB", client.post("/preprocess?size=224&gray=false", files=png("red_circle.png")))`	ResNet 같은 이미지 분류 모델이 쓰는 224×224×3 모양

▶ 실행 결과

```
[숫자 이미지 → 28x28 흑백] POST /preprocess → 200
{
  "original": [
    200,
    200,
    "L"
  ],
  "input_shape": [
    1,
    28,
    28,
    1
  ],
  "dtype": "float32",
  "min": 0.0,
  "max": 1.0,
  "mean": 0.1097,
  "center_row": [
    0.0,
    0.0,
    0.0,
    0.97,
    0.25,
    0.0,
    0.0
  ]
}

[컬러 이미지 → 224x224 RGB] POST /preprocess → 200
{
  "original": [
    320,
    240,
    "RGB"
  ],
  "input_shape": [
    1,
    224,
    224,
    3
  ],
  "dtype": "float32",
  "min": 0.071,
  "max": 1.0,
  "mean": 0.7739,
  "center_row": null
}
```

주의 | PyTorch 는 채널 순서가 달라요

TensorFlow/Keras는 보통 `(배치, 높이, 너비, 채널)`(NHWC), PyTorch는 `(배치, 채널, 높이, 너비)`(NCHW)를 써요. PyTorch 모델이라면 `x.transpose(0, 3, 1, 2)`로 순서를 바꿔야 해요. 6장 참고 코드에서 다시 봐요.

6. 자주 하는 실수와 정리

실수	결과	해결
python-multipart 미설치	`Form data requires "python-multipart"` 오류	`pip install python-multipart`
`file.read()`에 await 빠뜨림	코루틴 객체가 나옴	`await file.read()` (async def 안에서)
content_type만 믿고 처리	가짜 파일로 서버 오류	`Image.open`으로 내용 확인
NumPy 값을 그대로 return	`TypeError: Object of type float32 is not JSON serializable`	`float()`, `.tolist()`
학습과 다른 전처리	예측이 엉망	전처리 함수를 학습·서빙에서 공유
클라이언트에서 `json=`으로 파일 전송	422	`files=`

개념	핵심
파일 받기	`file: UploadFile` → `await file.read()`
여러 파일	`files: list[UploadFile] = File(...)`
검사	content_type → 크기 → `Image.open`
이미지 응답	`Response(content=bytes, media_type="image/png")`
전처리	convert → resize → `/255` → `np.newaxis`로 `(1, H, W, C)`
테스트	`client.post(url, files={"file": (이름, bytes, 종류)})`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. 업로드한 이미지의 **평균 밝기**(흑백으로 바꾼 뒤 0 ~ 255 평균, 소수 1자리)를 돌려주는 `POST /brightness`를 만들고 `samples/digit_one.png`로 시험하세요.

▶ 이렇게 출력되면 성공

```
{'filename': 'digit_one.png', 'brightness': 27.9}
```

문제 2. 이미지를 받아 **좌우로 뒤집은** 이미지를 PNG로 돌려주는 `POST /flip`을 만들고, 응답의 content-type과 이미지 크기를 출력하세요.

힌트: `img.transpose(Image.Transpose.FLIP_LEFT_RIGHT)`

▶ 이렇게 출력되면 성공

```
image/png (320, 240)
```

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
import io
import numpy as np
from fastapi import FastAPI, UploadFile
from fastapi.testclient import TestClient
from PIL import Image

app = FastAPI()


@app.post("/brightness")
async def brightness(file: UploadFile):
    img = Image.open(io.BytesIO(await file.read())).convert("L")
    return {"filename": file.filename, "brightness": round(float(np.asarray(img).mean()), 1)}


client = TestClient(app)
data = open("samples/digit_one.png", "rb").read()
print(client.post("/brightness", files={"file": ("digit_one.png", data, "image/png")}).json())
```

▶ 실행 결과

```
{'filename': 'digit_one.png', 'brightness': 27.9}
```

검은 바탕(0)에 흰 막대(255)가 조금 있어서 평균이 낮게 나와요. 밝은 사진일수록 값이 커져요.

문제 2 정답

직접 입력해 보세요

```python
import io
from fastapi import FastAPI, UploadFile
from fastapi.responses import Response
from fastapi.testclient import TestClient
from PIL import Image

app = FastAPI()


@app.post("/flip")
async def flip(file: UploadFile):
    img = Image.open(io.BytesIO(await file.read()))
    img = img.transpose(Image.Transpose.FLIP_LEFT_RIGHT)
    buf = io.BytesIO()
    img.save(buf, format="PNG")
    return Response(buf.getvalue(), media_type="image/png")


client = TestClient(app)
data = open("samples/blue_circle.png", "rb").read()
res = client.post("/flip", files={"file": ("blue_circle.png", data, "image/png")})
print(res.headers["content-type"], Image.open(io.BytesIO(res.content)).size)
```

▶ 실행 결과

```
image/png (320, 240)
```

06장

머신러닝 모델 서빙

학습한 모델을 저장하고, 서버 시작 때 불러와, /predict API로 예측을 제공합니다

항목	내용
예제 폴더	`fastapi/06_머신러닝_모델_서빙`
예제 파일	`ex00_train_model.py`(학습·저장), `model_core.py`(모델 클래스), `ex01_serving_app.py`(서빙 앱), `ex02_image_app.py`(이미지 분류 앱) + 요청 파일들
저장되는 모델	`model/pass_model.npz`, `model/model_card.json`, `model/color_model.npz`
이전 장	01 ~ 05장 전체, 시각화 6장의 EDA
예상 학습 시간	5 ~ 6시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- 모델 서빙의 전체 흐름(학습 → 저장 → 불러오기 → 예측 API)을 설명할 수 있어요.
- 학습한 모델의 가중치와 **전처리 정보**를 함께 저장할 수 있어요.
- `lifespan`에서 모델을 한 번만 불러오고, `Depends`로 꺼내 쓸 수 있어요.
- 입력·출력 스키마를 Pydantic으로 정의한 `/predict`, `/predict/batch`를 만들 수 있어요.
- `/health`, `/model/info`로 서버·모델 상태를 알려 줄 수 있어요.
- 모델이 없을 때 503을 돌려주는 등 실패 상황을 처리할 수 있어요.
- 이미지 분류 API를 만들고, PyTorch/Keras 모델로 바꿔 끼우는 방법을 알아요.

도움말 | 이번 장의 구성

- 1. 모델 서빙의 전체 흐름 **[필수]**
- 2. 모델 학습하고 저장하기 **[필수]**
- 3. 모델 클래스 만들기 **[필수]**
- 4. 예측 API 서버 **[필수]**
- 5. 실제 서버로 예측 요청하기 **[필수]**
- 6. 모델이 없을 때 **[심화]**
- 7. 이미지 분류 API **[심화]**
- 8. 딥러닝 모델로 바꿔 끼우기 (참고 코드) **[심화]**
- 9. 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

이 과정의 목표 지점이에요. 미리 학습해 둔 **"합격 확률 예측 모델"** 을 서버가 켜질 때 불러오고, `POST /predict` 로 공부·수면·휴대폰 시간을 보내면 합격 확률을 돌려주는 API 를 만들어요. 모델의 수학 내용은 몰라도 돼요. **"숫자를 넣으면 확률이 나오는 함수"** 라고 생각하세요.

- [필수] 1~5절 : 전체 흐름, 모델 저장·불러오기, `/predict` API, 실제 서버로 요청
- [심화] 6절 모델이 없을 때, 7절 이미지 분류 API, 8절 딥러닝 모델로 바꾸기(참고 코드)
- 2절의 학습 코드(경사하강법)는 **실행만 하고 넘어가도** 돼요. 이 장의 주인공은 학습이 아니라 서빙이에요.

1. 모델 서빙의 전체 흐름 필수

비유로 이해하기 | 레시피를 책으로 만들어 식당에 넘기기

연구실(노트북)에서 맛있는 요리법(모델)을 개발했다면, ① 레시피를 **책으로 정리**하고(저장), ② 식당이 문을 열 때 **주방에 비치**하고(불러오기), ③ 손님 주문이 오면 **레시피대로 요리**해서(예측) 내보내요. 이 장에서 이 과정을 처음부터 끝까지 만들어 봐요.

단계	하는 일	파일
① 학습	데이터로 모델을 학습하고 평가	`ex00_train_model.py`
② 저장	가중치 + **전처리 정보** + 설명서(model card)	`model/pass_model.npz`, `model_card.json`
③ 모델 클래스	불러오기·예측을 담당하는 코드	`model_core.py`
④ 서빙 앱	lifespan으로 불러오고 `/predict` 제공	`ex01_serving_app.py`
⑤ 요청	웹·앱·다른 서버가 JSON으로 요청	`ex01_serving_client.py`, curl

알아 두기 | 이 장의 모델은 연습용이에요

딥러닝 프레임워크(PyTorch, TensorFlow)는 설치가 크고 무거워서, 이 장은 **NumPy로 직접 학습한 작은 모델**(로지스틱 회귀)로 서빙 구조를 익혀요. 서빙 앱은 모델 클래스의 `predict_proba()`만 부르기 때문에, 나중에 딥러닝 모델로 바꿀 때는 **모델 클래스만** 바꾸면 돼요 (8절).

2. 모델 학습하고 저장하기 필수

시각화 6장의 학생 생활 데이터와 같은 규칙으로 400명을 만들고, 공부·수면·휴대폰 시간으로 **합격(점수 60 이상) 여부**를 예측하는 모델을 학습해요.

예제 파일 `fastapi/06_머신러닝_모델_서빙/ex00_train_model.py`

ex00_train_model.py

```python
# 1) 데이터 만들기 → 2) 학습 → 3) 평가 → 4) 저장
import json
from pathlib import Path
import numpy as np
from PIL import Image, ImageDraw

# 모델 · 샘플 이미지를 저장할 폴더 만들기
HERE = Path(__file__).parent
(HERE / "model").mkdir(exist_ok=True)
(HERE / "samples").mkdir(exist_ok=True)

# ── 1) 학생 생활 데이터 (시각화 6장과 같은 규칙) ──
rng = np.random.default_rng(seed=2026)
n = 400
# 공부 · 수면 · 휴대폰 시간으로 점수를 만드는 규칙 (공부 ↑ 점수 ↑, 휴대폰 ↑ 점수 ↓) + 잡음
study = rng.uniform(0, 6, n)
sleep = 8.5 - study * 0.35 + rng.normal(0, 0.6, n)
phone = (5 - study * 0.5 + rng.normal(0, 1, n)).clip(0.5)
score = (50 + study * 7 - phone * 2 + rng.normal(0, 6, n)).clip(0, 100)
X = np.column_stack([study, sleep, phone])          # 입력 3개
y = (score >= 60).astype(float)                     # 정답: 합격 1, 불합격 0
FEATURES = ["study_h", "sleep_h", "phone_h"]

# ── 2) 학습: 표준화 + 로지스틱 회귀 (경사하강법) ──
idx = rng.permutation(n)
train, test = idx[:320], idx[320:]                  # 80% 학습 / 20% 평가
mean, std = X[train].mean(axis=0), X[train].std(axis=0)
```

ex00_train_model.py

```
Z = (X - mean) / std                                  # 표준화 (서빙 때도 똑같이 해야 함!)

# 경사하강법 3000번: 예측 확률과 정답의 차이(grad)만큼 가중치를 조금씩 고쳐요
w, b = np.zeros(3), 0.0
for step in range(3000):
    p = 1 / (1 + np.exp(-(Z[train] @ w + b)))       # 시그모이드 → 합격 확률
    grad = p - y[train]
    w -= 0.1 * Z[train].T @ grad / len(train)
    b -= 0.1 * grad.mean()

# ── 3) 평가 ──
# 평가 데이터로 정확도 계산: 확률 0.5 이상이면 합격으로 예측
p_test = 1 / (1 + np.exp(-(Z[test] @ w + b)))
acc = float(((p_test >= 0.5) == y[test]).mean())
print("가중치:", np.round(w, 3), "절편:", round(b, 3))
print(f"평가 정확도: {acc:.1%}  (평가 데이터 {len(test)}명)")

# ── 4) 저장: 가중치 + 전처리 정보 + 설명서 ──
np.savez(HERE / "model" / "pass_model.npz", w=w, b=b, mean=mean, std=std, features=np.array(FEATURES))
card = {"name": "pass-predictor", "version": "1.0.0", "features": FEATURES,
        "algorithm": "logistic regression (numpy)", "test_accuracy": round(acc, 4), "trained_on": 320}
(HERE / "model" / "model_card.json").write_text(json.dumps(card, ensure_ascii=False, indent=2),
encoding="utf-8")
print("저장:", sorted(p.name for p in (HERE / "model").iterdir()))

# ── (이미지 분류용) 색 분류 모델과 샘플 이미지 ──
# 클래스별 대표 색(0 ~ 1 의 RGB)
CLASSES = ["red", "green", "blue"]
proto = np.array([[0.86, 0.16, 0.16], [0.16, 0.71, 0.27], [0.16, 0.31, 0.86]])   # 클래스별 대표 색
np.savez(HERE / "model" / "color_model.npz", proto=proto, classes=np.array(CLASSES))
# 샘플 이미지 4장 (보라색은 학습에 없는 색 → 모델이 어떻게 답하는지 보기)
for name, color in [("red", (220, 40, 40)), ("green", (40, 180, 70)), ("blue", (40, 80, 220)), ("purple",
(140, 60, 200))]:
    img = Image.new("RGB", (160, 120), color)
    ImageDraw.Draw(img).rectangle((0, 0, 159, 20), fill="white")    # 위쪽에 흰 띠
    img.save(HERE / "samples" / f"{name}.png")
print("샘플 이미지:", sorted(p.name for p in (HERE / "samples").iterdir()))
```

한 줄씩 읽기

줄	코드	설명
20	`X = np.column_stack([study, sleep, phone]) # 입력 3개`	입력(특성) 3개를 열로 묶은 배열 (400, 3)
21	`y = (score >= 60).astype(float) # 정답: 합격 1, 불합격 0`	정답(레이블)
26	`train, test = idx[:320], idx[320:] # 80% 학습 / 20% 평가`	학습에 안 쓴 데이터로 평가해야 진짜 실력을 알 수 있어요.
28	`Z = (X - mean) / std # 표준화 (서빙 때도 똑같이 해야 함!)`	★ 평균 0, 표준편차 1로 맞추기. 이 **mean, std도 함께 저장**해야 서빙 때 똑같이 변환할 수 있어요.
33	`p = 1 / (1 + np.exp(-(Z[train] @ w + b))) # 시그모이드 → 합격 확률`	로지스틱 회귀: 가중합(NumPy 3장의 @)을 시그모이드로 0 ~ 1 확률로
35	`w -= 0.1 * Z[train].T @ grad / len(train)`	경사하강법: 틀린 만큼 가중치를 조금씩 고치기 (딥러닝 학습과 같은 원리)
46	`np.savez(HERE / "model" / "pass_model.npz", w=w, b=b, mean=mean, std=std, features=np.array(FEATURES))`	★ 가중치와 전처리 정보, 입력 순서를 한 파일에

줄	코드	설명
47	`card = {"name": "pass-predictor", "version": "1.0.0", "features": FEATURES,`	**model card**: 모델 이름·버전·입력·정확도를 적은 설명서. `/model/info`로 보여 줘요.

▶ 실행 결과

```
가중치: [ 3.951 -0.19  -0.776] 절편: 1.637
평가 정확도: 91.2%  (평가 데이터 80명)
저장: ['color_model.npz', 'model_card.json', 'pass_model.npz']
샘플 이미지: ['blue.png', 'green.png', 'purple.png', 'red.png']
```

주의 | 전처리 정보를 저장하지 않으면

모델 가중치만 저장하고 mean/std를 잊으면, 서빙할 때 입력을 똑같이 변환할 수 없어서 **엉뚱한 예측**이 나와요. 딥러닝에서도 "이미지 크기, 정규화 값, 입력 순서, 클래스 이름"을 꼭 함께 저장하세요.

3. 모델 클래스 만들기 필수

예제 파일 `fastapi/06_머신러닝_모델_서빙/model_core.py`

모델 코드와 API 코드를 **나눠 두는 것**이 서빙 프로젝트의 핵심 구조예요.

model_core.py

```python
# 모델 불러오기·예측을 담당하는 클래스 (서빙 앱은 이 클래스만 알면 됨)
import json
from pathlib import Path
import numpy as np


# 학습한 값(가중치 · 절편 · 평균 · 표준편차)과 설명서(card)를 들고 있는 모델 클래스
class PassModel:
    def __init__(self, w, b, mean, std, features, card):
        self.w, self.b, self.mean, self.std = w, b, mean, std
        self.features = list(features)
        self.card = card

    # classmethod: 객체 없이 PassModel.load(경로) 로 불러서 새 객체를 만드는 함수
    @classmethod
    def load(cls, path: str | Path) -> "PassModel":
        path = Path(path)
        data = np.load(path)                                    # FileNotFoundError 가능
        # 같은 폴더의 model_card.json 이 있으면 읽고, 없으면 빈 사전
        card_path = path.with_name("model_card.json")
        card = json.loads(card_path.read_text(encoding="utf-8")) if card_path.exists() else {}
        return cls(data["w"], float(data["b"]), data["mean"], data["std"], data["features"], card)

    # property: 함수지만 model.version 처럼 괄호 없이 읽어요
    @property
    def version(self) -> str:
        return self.card.get("version", "unknown")

    # 표준화 → 가중합 → 시그모이드 = 합격 확률
    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        Z = (X - self.mean) / self.std                          # 학습 때와 같은 전처리
        return 1 / (1 + np.exp(-(Z @ self.w + self.b)))


# 색 분류 모델: 이미지 평균 색이 어느 대표 색과 가장 가까운지로 분류
class ColorModel:
    def __init__(self, path: str | Path):
```

model_core.py

```
        data = np.load(path)
        self.proto, self.classes = data["proto"], list(data["classes"])
        self.version = "color-0.1"

    def predict_proba(self, x: np.ndarray) -> np.ndarray:      # x: (N, H, W, 3), 0 ~ 1
        mean_rgb = x.mean(axis=(1, 2))                          # 이미지마다 평균 색 (N, 3)
        dist = ((mean_rgb[:, None, :] - self.proto[None]) ** 2).sum(axis=2)   # 대표 색과의 거리
        # 거리가 가까울수록 점수가 크도록 음수로 → softmax 로 확률
        logits = -dist * 20
        e = np.exp(logits - logits.max(axis=1, keepdims=True))
        return e / e.sum(axis=1, keepdims=True)                 # softmax → 확률
```

한 줄씩 읽기

줄	코드	설명
8	`class PassModel:`	★ 모델의 불러오기·예측을 이 클래스에 모아요. 서빙 앱은 이 클래스의 메서드만 알면 돼요.
16	`def load(cls, path: str | Path) -> "PassModel":`	`@classmethod` 생성 함수: `PassModel.load("경로")` (Python 9, 11장)
18	`data = np.load(path) # FileNotFoundError 가능`	파일이 없으면 예외 → 6절에서 처리
26	`def version(self) -> str:`	`@property`로 버전을 속성처럼
31	`Z = (X - self.mean) / self.std # 학습 때와 같은 전처리`	★ 저장해 둔 mean/std로 똑같이 표준화
36	`class ColorModel:`	7절 이미지 분류용 간단한 모델 (평균 색과 대표 색의 거리 → softmax 확률)

4. 예측 API 서버 필수

예제 파일 `fastapi/06_머신러닝_모델_서빙/ex01_serving_app.py`

ex01_serving_app.py

```
import os
from contextlib import asynccontextmanager
from typing import Annotated

import numpy as np
from fastapi import Depends, FastAPI, HTTPException, Request
from pydantic import BaseModel, Field

from model_core import PassModel

# 모델 파일 위치: 환경 변수 MODEL_PATH 가 있으면 그것, 없으면 이 폴더의 model/pass_model.npz
MODEL_PATH = os.getenv("MODEL_PATH", os.path.join(os.path.dirname(__file__), "model", "pass_model.npz"))


# ── 입력·출력 스키마 ──
class StudentFeatures(BaseModel):
    study_h: float = Field(ge=0, le=16, description="하루 공부 시간", examples=[4.5])
    sleep_h: float = Field(ge=0, le=16, description="하루 수면 시간", examples=[7.0])
    phone_h: float = Field(ge=0, le=24, description="하루 휴대폰 사용 시간", examples=[2.0])


# 응답 하나의 모양: 합격 확률 · 결과 · 모델 버전
class Prediction(BaseModel):
    pass_probability: float
    label: str
```

ex01_serving_app.py

```
    model_version: str


# 여러 명을 한 번에 (1 ~ 100명)
class BatchRequest(BaseModel):
    students: list[StudentFeatures] = Field(min_length=1, max_length=100)


class BatchResponse(BaseModel):
    count: int
    predictions: list[Prediction]


# ── 서버 시작 시 모델 한 번 불러오기 ──
@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        app.state.model = PassModel.load(MODEL_PATH)
        app.state.model_error = None
    except Exception as e:                        # 모델이 없어도 서버는 켜지게
        app.state.model = None
        app.state.model_error = f"{type(e).__name__}: {e}"
    # 예측 횟수 세기용 변수
    app.state.n_predictions = 0
    yield
    app.state.model = None


app = FastAPI(title="합격 예측 API", version="1.0.0", lifespan=lifespan)


def get_model(request: Request) -> PassModel:   # 의존성: 모델 꺼내기 (없으면 503)
    model = request.app.state.model
    if model is None:
        raise HTTPException(status_code=503, detail="모델이 준비되지 않았어요")
    return model


# 다른 함수에서 model: Model 로 쓰면 get_model 을 거쳐 모델이 들어와요
Model = Annotated[PassModel, Depends(get_model)]


def to_array(items: list[StudentFeatures]) -> np.ndarray:
    return np.array([[s.study_h, s.sleep_h, s.phone_h] for s in items])   # 학습 때와 같은 열 순서


def make_prediction(p: float, version: str) -> Prediction:
    return Prediction(pass_probability=round(float(p), 4), label="합격" if p >= 0.5 else "불합격",
                      model_version=version)


# 상태 확인: 모델이 불러와졌는지, 실패했다면 이유
@app.get("/health", tags=["status"])
def health(request: Request):
    return {"status": "ok", "model_loaded": request.app.state.model is not None,
            "model_error": request.app.state.model_error}


@app.get("/model/info", tags=["status"])
def model_info(model: Model, request: Request):
    return {**model.card, "predictions_served": request.app.state.n_predictions}

```

ex01_serving_app.py

```
88
89  # 한 명 예측: 입력 → 배열 → 확률 → 응답 모양
90  @app.post("/predict", response_model=Prediction, tags=["predict"])
91  def predict(student: StudentFeatures, model: Model, request: Request):
92      p = model.predict_proba(to_array([student]))[0]
93      request.app.state.n_predictions += 1
94      return make_prediction(p, model.version)
95
96
97  # 여러 명 예측
98  @app.post("/predict/batch", response_model=BatchResponse, tags=["predict"])
99  def predict_batch(req: BatchRequest, model: Model, request: Request):
100     probs = model.predict_proba(to_array(req.students))       # 한 번에 계산 (빠름)
101     request.app.state.n_predictions += len(probs)
102     return BatchResponse(count=len(probs), predictions=[make_prediction(p, model.version) for p in probs])
```

한 줄씩 읽기

줄	코드	설명
12	`MODEL_PATH = os.getenv("MODEL_PATH", os.path.join(os.path.dirname(__file__), "model", "pass_model.npz"))`	모델 경로를 환경 변수로 바꿀 수 있게 (4장) — 새 버전 모델로 바꿀 때 코드를 고치지 않아도 돼요.
16	`class StudentFeatures(BaseModel):`	★ 입력 스키마: 범위 밖 값(공부 -1시간)은 모델에 닿기 전에 422로 차단 (2장)
23	`class Prediction(BaseModel):`	출력 스키마: 확률, 레이블, **모델 버전** — 어떤 버전이 답했는지 꼭 남겨요.
31	`students: list[StudentFeatures] = Field(min_length=1, max_length=100)`	배치 요청은 최대 100명 (서버 과부하 방지)
43	`app.state.model = PassModel.load(MODEL_PATH)`	★ lifespan: 서버 시작 때 **한 번만** 불러오기 (4장)
45	`except Exception as e: # 모델이 없어도 서버는 켜지게`	모델이 없어도 서버는 켜져서 `/health`로 원인을 알려줘요 (6절).
57	`def get_model(request: Request) -> PassModel: # 의존성: 모델 꺼내기 (없으면 503)`	★ 의존성(4장): 모든 예측 API가 이 함수로 모델을 받아요.
69	`return np.array([[s.study_h, s.sleep_h, s.phone_h] for s in items]) # 학습 때와 같은 열 순서`	입력 순서를 학습 때와 **똑같이**
100	`probs = model.predict_proba(to_array(req.students)) # 한 번에 계산 (빠름)`	배치는 하나씩 반복하지 않고 배열로 한 번에 계산 (NumPy 3장 벡터 연산)

예제 파일 `fastapi/06_머신러닝_모델_서빙/ex01_serving_client.py`

ex01_serving_client.py

```
1  import json
2  from fastapi.testclient import TestClient
3  from ex01_serving_app import app
4
5
6  # 요청 · 응답을 출력하는 도우미
7  def show(title, res):
8      print(f"[{title}] {res.request.method} {res.request.url.path} → {res.status_code}")
9      print(json.dumps(res.json(), ensure_ascii=False, indent=2))
10     print()
11
12
```

ex01_serving_client.py

```
13 with TestClient(app) as client:                          # lifespan 실행 (모델 불러오기)
14     show("상태", client.get("/health"))
15     show("모델 정보", client.get("/model/info"))
16     show("열심히 하는 학생", client.post("/predict", json={"study_h": 5, "sleep_h": 6.5, "phone_h": 1}))
17     show("휴대폰을 많이 하는 학생", client.post("/predict", json={"study_h": 0.5, "sleep_h": 8.5, "phone_h":
   6}))
18     # -1 과 "많이" 는 검사에 걸려 422
19     show("잘못된 입력", client.post("/predict", json={"study_h": -1, "sleep_h": 7, "phone_h": "많이"}))
20     # 공부 시간만 1 ~ 4 시간으로 바꾼 학생 4명
21     batch = {"students": [{"study_h": h, "sleep_h": 7, "phone_h": 2} for h in [1, 2, 3, 4]]}
22     show("여러 명 한 번에", client.post("/predict/batch", json=batch))
23     print("지금까지 예측 수:", client.get("/model/info").json()["predictions_served"])
```

한 줄씩 읽기

줄	코드	설명
13	`with TestClient(app) as client: # lifespan 실행 (모델 불러오기)`	with로 써야 lifespan이 실행돼 모델이 불러와져요.
16	`show("열심히 하는 학생", client.post("/predict", json={"study_h": 5, "sleep_h": 6.5, "phone_h": 1}))`	합격 확률이 높게
17	`show("휴대폰을 많이 하는 학생", client.post("/predict", json={"study_h": 0.5, "sleep_h": 8.5, "phone_h": 6}))`	합격 확률이 낮게
19	`show("잘못된 입력", client.post("/predict", json={"study_h": -1, "sleep_h": 7, "phone_h": "많이"}))`	모델에 닿기 전에 422로 차단
23	`print("지금까지 예측 수:", client.get("/model/info").json()["predictions_served"])`	2 + 4 = 6건 — 서버가 몇 번 예측했는지 세는 간단한 모니터링

▶ 실행 결과

```
[상태] GET /health → 200
{
  "status": "ok",
  "model_loaded": true,
  "model_error": null
}

[모델 정보] GET /model/info → 200
{
  "name": "pass-predictor",
  "version": "1.0.0",
  "features": [
    "study_h",
    "sleep_h",
    "phone_h"
  ],
  "algorithm": "logistic regression (numpy)",
  "test_accuracy": 0.9125,
  "trained_on": 320,
  "predictions_served": 0
}

[열심히 하는 학생] POST /predict → 200
{
  "pass_probability": 0.9997,
  "label": "합격",
  "model_version": "1.0.0"
}

[휴대폰을 많이 하는 학생] POST /predict → 200
{
```

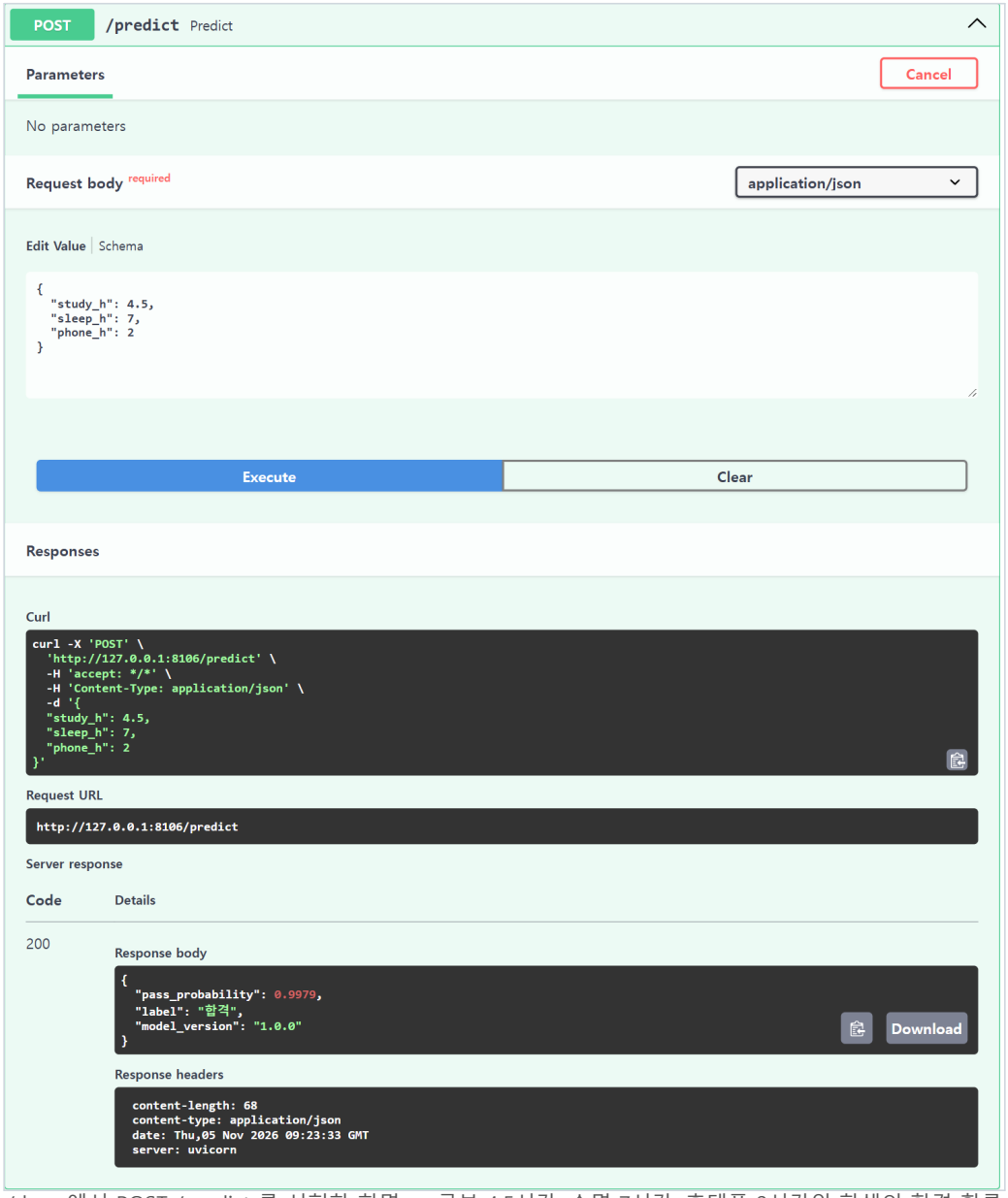
▶ 실행 결과

```
  "pass_probability": 0.0021,
  "label": "불합격",
  "model_version": "1.0.0"
}

[잘못된 입력] POST /predict → 422
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "study_h"
      ],
      "msg": "Input should be greater than or equal to 0",
      "input": -1,
      "ctx": {
        "ge": 0.0
      }
    },
    {
      "type": "float_parsing",
      "loc": [
        "body",
        "phone_h"
      ],
      "msg": "Input should be a valid number, unable to parse string as a number",
      "input": "많이"
    }
  ]
}

[여러 명 한 번에] POST /predict/batch → 200
{
  "count": 4,
  "predictions": [
    {
      "pass_probability": 0.1096,
      "label": "불합격",
      "model_version": "1.0.0"
    },
    {
      "pass_probability": 0.5657,
      "label": "합격",
      "model_version": "1.0.0"
    },
    {
      "pass_probability": 0.9323,
      "label": "합격",
      "model_version": "1.0.0"
    },
    {
      "pass_probability": 0.9932,
      "label": "합격",
      "model_version": "1.0.0"
    }
  ]
}

지금까지 예측 수: 6
```



/docs 에서 POST /predict 를 시험한 화면 — 공부 4.5시간, 수면 7시간, 휴대폰 2시간인 학생의 합격 확률

5. 실제 서버로 예측 요청하기 필수

터미널 명령

```
cd fastapi/06_머신러닝_모델_서빙
python ex00_train_model.py            # 처음 한 번: 모델 학습·저장
uvicorn ex01_serving_app:app          # 서버 실행 → http://127.0.0.1:8000/docs
```

▶ 실제로 실행한 터미널 화면 (서버 실행 → 다른 터미널에서 curl 로 POST 요청)

```
[터미널 1] $ uvicorn ex01_serving_app:app
INFO:     Started server process [2272]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:10608 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:10609 - "POST /predict HTTP/1.1" 200 OK
INFO:     127.0.0.1:10610 - "POST /predict HTTP/1.1" 422 Unprocessable Content
INFO:     127.0.0.1:10611 - "GET /model/info HTTP/1.1" 200 OK

[터미널 2]
```

▶ 실제로 실행한 터미널 화면 (서버 실행 → 다른 터미널에서 curl 로 POST 요청)

```
$ curl http://127.0.0.1:8000/health
{"status":"ok","model_loaded":true,"model_error":null}
$ curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d '{"study_h": 4, "sleep_h": 7, "phone_h": 2}'
{"pass_probability":0.9932,"label":"합격","model_version":"1.0.0"}
$ curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d '{"study_h": 30, "sleep_h": 7, "phone_h": 2}'
{"detail":[{"type":"less_than_equal","loc":["body","study_h"],"msg":"Input should be less than or equal to 16","input":30,"ctx":{"le":16.0}}]}
$ curl http://127.0.0.1:8000/model/info
{"name":"pass-predictor","version":"1.0.0","features":["study_h","sleep_h","phone_h"],"algorithm":"logistic regression (numpy)","test_accuracy":0.9125,"trained_on":320,"predictions_served":1}
```

도움말 | 다른 프로그램에서 호출하기

파이썬에서는 `requests.post("http://127.0.0.1:8000/predict", json={...}).json()`, 자바스크립트에서는 `fetch(url, {method: "POST", headers: {"Content-Type": "application/json"}, body: JSON.stringify({...})})`로 호출해요. 웹페이지에서 부를 때는 4장의 CORS 설정이 필요해요.

6. 모델이 없을 때 심화

심화 · 선택 | 심화 · 6~8절은 시간이 있을 때

여기부터는 실제 서비스에서 만나는 상황(모델 파일이 없을 때, 이미지로 예측하기, 딥러닝 모델로 교체하기)이에요. 기본 과정에서는 9절 정리로 넘어가도 돼요.

예제 파일 `fastapi/06_머신러닝_모델_서빙/ex02_missing_model_client.py`

ex02_missing_model_client.py

```python
import os
os.environ["MODEL_PATH"] = "model/없는_모델.npz"      # 일부러 없는 경로 (앱을 불러오기 전에!)

from fastapi.testclient import TestClient
from ex01_serving_app import app

# 모델이 없으면: /health 는 model_loaded False, /predict 는 503
with TestClient(app) as client:
    print(client.get("/health").json())
    res = client.post("/predict", json={"study_h": 3, "sleep_h": 7, "phone_h": 2})
    print(res.status_code, res.json())
```

한 줄씩 읽기

줄	코드	설명
2	`os.environ["MODEL_PATH"] = "model/없는_모델.npz" # 일부러 없는 경로 (앱을 불러오기 전에!)`	모델 파일을 못 찾는 상황 만들기
9	`print(client.get("/health").json())`	서버는 켜져 있고, `model_loaded: false`와 **원인**을 알려 줘요.
11	`print(res.status_code, res.json())`	예측 요청은 **503 Service Unavailable** — "잠시 서비스할 수 없음"

▶ 실행 결과

```
{'status': 'ok', 'model_loaded': False, 'model_error': "FileNotFoundError: [Errno 2] No such file or directory: 'model\\\\없는_모델.npz'"}
503 {'detail': '모델이 준비되지 않았어요'}
```

> **알아 두기 | 왜 서버를 끄지 않고 503 을 줄까?**
> 실제 서비스에서는 쿠버네티스 같은 관리 도구가 `/health`를 주기적으로 확인해서 문제가 있는 서버로는 요청을 보내지 않고, 운영자에게 알려요. 서버가 아예 꺼져 버리면 **왜** 문제인지 알기 어려워요.

7. 이미지 분류 API 심화

5장의 이미지 전처리와 이 장의 모델 서빙을 합쳐, 이미지를 올리면 클래스별 확률을 돌려주는 API를 만들어요. 여기서는 간단한 "색 분류 모델"을 쓰지만, 구조는 실제 CNN 이미지 분류 서빙과 같아요.

예제 파일 `fastapi/06_머신러닝_모델_서빙/ex03_image_app.py`

ex03_image_app.py

```python
import io
import os
from contextlib import asynccontextmanager

import numpy as np
from fastapi import FastAPI, HTTPException, Request, UploadFile
from PIL import Image, UnidentifiedImageError
from pydantic import BaseModel

from model_core import ColorModel

# 이 파일이 있는 폴더 (모델 파일 위치 기준)
HERE = os.path.dirname(__file__)


def preprocess(img: Image.Image, size=(32, 32)) -> np.ndarray:   # 5장의 전처리와 같은 방식
    arr = np.asarray(img.convert("RGB").resize(size), dtype=np.float32) / 255.0
    return arr[np.newaxis, ...]                                    # (1, 32, 32, 3)


# 클래스 하나의 점수 / 응답 전체 모양
class ClassScore(BaseModel):
    label: str
    probability: float


class ImagePrediction(BaseModel):
    filename: str
    top: ClassScore
    scores: list[ClassScore]
    model_version: str


# 서버 시작 때 색 분류 모델 불러오기
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = ColorModel(os.path.join(HERE, "model", "color_model.npz"))
    yield


app = FastAPI(title="이미지 색 분류 API", lifespan=lifespan)


@app.post("/predict/image", response_model=ImagePrediction)
async def predict_image(file: UploadFile, request: Request):
    try:
        img = Image.open(io.BytesIO(await file.read()))
    except UnidentifiedImageError:
```

ex03_image_app.py

```
        raise HTTPException(400, "이미지를 읽을 수 없어요")
    model = request.app.state.model
    probs = model.predict_proba(preprocess(img))[0]
    # 확률이 높은 순서로 정렬해서 1등(top)과 전체 점수를 돌려주기
    scores = sorted([ClassScore(label=c, probability=round(float(p), 4)) for c, p in zip(model.classes, probs)],
                    key=lambda s: s.probability, reverse=True)
    return ImagePrediction(filename=file.filename, top=scores[0], scores=scores, model_version=model.version)
```

한 줄씩 읽기

줄	코드	설명
16	`def preprocess(img: Image.Image, size=(32, 32)) -> np.ndarray: # 5장의 전처리와 같은 방식`	★ 이미지 → (1, 32, 32, 3), 0 ~ 1
27	`class ImagePrediction(BaseModel):`	응답: 1등 클래스 + 전체 클래스 점수 + 모델 버전
51	`probs = model.predict_proba(preprocess(img))[0]`	전처리 → 모델 → 확률 (배치 1개 중 첫 번째)
53	`scores = sorted([ClassScore(label=c, probability=round(float(p), 4)) for c, p in zip(model.classes, probs)],`	확률 높은 순으로 정렬 (lambda key, Python 8장)

예제 파일 `fastapi/06_머신러닝_모델_서빙/ex03_image_client.py`

ex03_image_client.py

```
from pathlib import Path
from fastapi.testclient import TestClient
from ex03_image_app import app

# 샘플 이미지가 있는 폴더
S = Path(__file__).parent / "samples"
with TestClient(app) as client:
    # 4장을 차례로 업로드해서 1등과 나머지 확률 출력
    for name in ["red.png", "green.png", "blue.png", "purple.png"]:
        res = client.post("/predict/image", files={"file": (name, (S / name).read_bytes(), "image/png")})
        r = res.json()
        others = ", ".join(f"{s['label']} {s['probability']:.1%}" for s in r["scores"][1:])
        print(f"{name:11} → {r['top']['label']:5} {r['top']['probability']:.1%}   (나머지: {others})")
    # 이미지가 아닌 내용을 보내면 400
    bad = client.post("/predict/image", files={"file": ("x.png", b"oops", "image/png")})
    print("잘못된 파일:", bad.status_code, bad.json())
```

한 줄씩 읽기

줄	코드	설명
13	`print(f"{name:11} → {r['top']['label']:5} {r['top']['probability']:.1%} (나머지: {others})")`	1등 클래스와 확률, 나머지 확률
15	`bad = client.post("/predict/image", files={"file": ("x.png", b"oops", "image/png")})`	이미지가 아니면 400

▶ 실행 결과

```
red.png     → red   100.0%   (나머지: green 0.0%, blue 0.0%)
green.png   → green 100.0%   (나머지: blue 0.0%, red 0.0%)
blue.png    → blue  100.0%   (나머지: green 0.0%, red 0.0%)
purple.png  → blue  99.8%   (나머지: red 0.2%, green 0.0%)
잘못된 파일: 400 {'detail': '이미지를 읽을 수 없어요'}
```

purple(보라색)은 학습한 클래스(red, green, blue)에 **없는** 색인데도, 모델은 가장 가까운 blue라고 **99.8%로 자신 있게** 답했어요. 실제 딥러닝 모델도 처음 보는 종류의 입력에 높은 확률로 틀린 답을 하는 경우(과신, overconfidence)가 흔해요. 그래서 확률만 믿지 말고, 입력 검사·예측 기록·모니터링으로 이상한 입력을 찾아내는 장치가 필요해요.

8. 딥러닝 모델로 바꿔 끼우기 (참고 코드) 심화

서빙 앱(`ex01_serving_app.py`, `ex03_image_app.py`)은 모델 객체의 `predict_proba(배열)`과 `version`만 사용해요. 그래서 딥러닝 학습이 끝난 뒤에는 **같은 모양의 모델 클래스**를 만들어 lifespan에서 그 클래스를 불러오면 끝이에요.

> **주의 | 아래 두 코드는 실행하지 않은 참고 코드예요**
> 이 교안을 만든 컴퓨터에는 PyTorch, TensorFlow가 설치되어 있지 않아서 실행 결과를 확인하지 못했어요. 각 라이브러리를 설치한 뒤 **자신의 모델 파일 경로와 입력 모양에 맞게** 고쳐서 사용하세요.

PyTorch 모델 클래스 (참고 · 미실행)

```python
# ── 참고: PyTorch 모델로 바꾸기 (torch 설치 필요, 이 교안에서는 실행하지 않았어요) ──
import torch

class TorchModel:
    def __init__(self, path: str):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.net = torch.jit.load(path, map_location=self.device)   # 또는 모델 클래스 + load_state_dict
        self.net.eval()                                              # 추론 모드 (Dropout 등 끄기)
        self.version = "torch-1.0"

    def predict_proba(self, x):                      # x: NumPy (N, H, W, C), 0 ~ 1
        t = torch.from_numpy(x).permute(0, 3, 1, 2).float().to(self.device)   # NHWC → NCHW
        with torch.no_grad():                        # 기울기 계산 끄기 → 빠르고 메모리 절약
            logits = self.net(t)
            return torch.softmax(logits, dim=1).cpu().numpy()

# lifespan 에서:  app.state.model = TorchModel("model/model.pt")
```

TensorFlow / Keras 모델 클래스 (참고 · 미실행)

```python
# ── 참고: TensorFlow / Keras 모델로 바꾸기 (tensorflow 설치 필요, 이 교안에서는 실행하지 않았어요) ──
import tensorflow as tf

class KerasModel:
    def __init__(self, path: str):
        self.net = tf.keras.models.load_model(path)   # 예: "model/mnist.keras"
        self.version = "keras-1.0"

    def predict_proba(self, x):                       # x: NumPy (N, 28, 28, 1), 0 ~ 1  ← 5장 전처리 그대로
        return self.net.predict(x, verbose=0)         # (N, 클래스 수) 확률

# lifespan 에서:  app.state.model = KerasModel("model/mnist.keras")
```

딥러닝 서빙 체크리스트	이유
`model.eval()` + `torch.no_grad()` (PyTorch)	Dropout·BatchNorm을 추론 모드로, 기울기 계산을 꺼서 빠르게
전처리를 학습 때와 똑같이 (크기, 정규화 평균·표준편차, 채널 순서 NHWC/NCHW)	다르면 정확도가 크게 떨어져요
모델은 lifespan에서 한 번만 불러오기	요청마다 불러오면 매우 느려요

딥러닝 서빙 체크리스트	이유
배치 예측 API 제공	GPU는 여러 개를 한 번에 계산할 때 효율적
응답에 모델 버전·확률 포함	나중에 문제를 추적하고 판단 보류를 하기 위해
큰 모델은 무거운 계산이므로 `def` 엔드포인트 (async 안에서 직접 계산 금지)	4장: async 안의 무거운 계산은 서버를 멈춰요
더 빠르게: ONNX Runtime, TorchServe, NVIDIA Triton	대규모 서비스용 전문 서빙 도구

9. 정리

단계	핵심 코드
저장	`np.savez(가중치, 전처리 정보)` + `model_card.json` (PyTorch: `torch.save`, Keras: `model.save`)
모델 클래스	`load()`, `predict_proba(X)`, `version`
불러오기	`lifespan`에서 `app.state.model = Model.load(경로)`
꺼내기	`Depends(get_model)` — 없으면 503
입출력	Pydantic `StudentFeatures` → `Prediction(probability, label, model_version)`
상태	`/health`, `/model/info`
배치	`list[...] = Field(max_length=100)` + 배열로 한 번에 예측

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. `model_core.PassModel`을 직접 불러와(API 없이) 공부 3시간·수면 7시간·휴대폰 2시간인 학생의 합격 확률을 소수 3자리로 출력하세요.

문제 2. `ex01_serving_app`의 `/predict/batch`에 공부 시간 0, 2, 4, 6시간(수면 7, 휴대폰 2)인 4명을 보내, 공부 시간과 합격 확률을 표처럼 출력하세요.

문제 3. `/predict`의 응답에 `advice`(합격 확률이 0.5 미만이면 "공부 시간을 늘려 보세요", 아니면 "지금처럼!")를 추가한 새 앱을 만들어 보세요. (힌트: 기존 앱의 모델 클래스와 같은 방식으로 lifespan에서 불러오기)

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
import numpy as np
from model_core import PassModel

model = PassModel.load("model/pass_model.npz")
p = model.predict_proba(np.array([[3, 7, 2]]))[0]
print(f"합격 확률: {p:.3f}  (모델 버전 {model.version})")
```

▶ 실행 결과

```
합격 확률: 0.932  (모델 버전 1.0.0)
```

서빙 앱 없이도 모델 클래스만으로 예측할 수 있어요. 모델 코드와 API 코드를 나눠 두면 이렇게 따로 시험하기 쉬워요.

문제 2 정답

직접 입력해 보세요

```python
from fastapi.testclient import TestClient
from ex01_serving_app import app

hours = [0, 2, 4, 6]
with TestClient(app) as client:
    body = {"students": [{"study_h": h, "sleep_h": 7, "phone_h": 2} for h in hours]}
    preds = client.post("/predict/batch", json=body).json()["predictions"]
for h, p in zip(hours, preds):
    print(f"공부 {h}시간 → {p['pass_probability']:.1%} ({p['label']})")
```

▶ 실행 결과

```
공부 0시간 → 1.1% (불합격)
공부 2시간 → 56.6% (합격)
공부 4시간 → 99.3% (합격)
공부 6시간 → 100.0% (합격)
```

공부 시간이 늘수록 합격 확률이 올라가는 것을 볼 수 있어요. 시각화 6장의 EDA에서 본 관계를 모델이 학습한 거예요.

문제 3 정답

직접 입력해 보세요

```python
from contextlib import asynccontextmanager
import numpy as np
from fastapi import FastAPI, Request
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field
from model_core import PassModel


class Features(BaseModel):
    study_h: float = Field(ge=0, le=16)
    sleep_h: float = Field(ge=0, le=16)
    phone_h: float = Field(ge=0, le=24)


```

직접 입력해 보세요

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = PassModel.load("model/pass_model.npz")
    yield


app = FastAPI(lifespan=lifespan)


@app.post("/predict")
def predict(f: Features, request: Request):
    p = float(request.app.state.model.predict_proba(np.array([[f.study_h, f.sleep_h, f.phone_h]]))[0])
    return {"pass_probability": round(p, 3), "advice": "공부 시간을 늘려 보세요" if p < 0.5 else "지금처럼!"}


with TestClient(app) as c:
    print(c.post("/predict", json={"study_h": 1, "sleep_h": 8, "phone_h": 5}).json())
    print(c.post("/predict", json={"study_h": 5, "sleep_h": 7, "phone_h": 1}).json())
```

▶ 실행 결과

```
{'pass_probability': 0.014, 'advice': '공부 시간을 늘려 보세요'}
{'pass_probability': 1.0, 'advice': '지금처럼!'}
```

07장

프로젝트 구조 · 테스트 · 배포

실무형 폴더 구조로 정리하고, pytest로 자동 검사하고, 다른 컴퓨터에서도 실행되게 만듭니다

항목	내용
예제 폴더	`fastapi/07_테스트와_배포/serving_project`
구성	`app/`(설정·스키마·모델·라우터), `tests/`, `train.py`, `requirements.txt`, `Dockerfile`
실행 파일	`serving_project/train.py`, `ex01_run_tests.py`
이전 장	06 머신러닝 모델 서빙
예상 학습 시간	4 ~ 5시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- 서빙 프로젝트를 설정·스키마·모델·라우터로 나눠 구성할 수 있어요.
- 환경 변수 기반 설정 객체를 만들 수 있어요.
- pytest + TestClient로 API 자동 테스트를 작성하고 실행할 수 있어요.
- `requirements.txt`로 필요한 패키지를 정리할 수 있어요.
- uvicorn 실행 옵션(포트, 호스트, 워커)을 알아요.
- Dockerfile이 무엇이고 어떻게 쓰는지 알아요.

도움말 | 이번 장의 구성

- 1. 실무형 폴더 구조 **[필수]**
- 2. 설정 · 스키마 · 모델 **[필수]**
- 3. 라우터와 메인 앱 **[필수]**
- 4. pytest 로 자동 테스트 **[필수]**
- 5. 실행과 패키지 정리 **[필수]**
- 6. Docker 로 배포 (참고) **[심화]**
- 7. 다음 단계와 정리
- 연습문제 · 정답 및 해설

웹 초보 안내 | 이 장을 쉽게 말하면

6장의 서버를 **회사에서 쓰는 모양**으로 정리해요. 파일을 역할별 폴더로 나누고, 코드를 고칠 때마다 자동으로 검사하는 **테스트**를 만들고, 다른 컴퓨터에서도 똑같이 돌아가게 **포장(Docker)** 하는 방법을 봐요.

- [필수] 1~3절 : 폴더 구조 읽기 — 새 코드는 거의 없고 6장 코드를 나눠 담은 것이에요
- [필수] 4~5절 : `pytest` 로 자동 테스트, `requirements.txt`
- [심화] 6절 : Docker — 설치가 필요하고 배포 단계의 내용이라 참고로 읽어요

1. 실무형 폴더 구조 필수

비유로 이해하기 | 부엌 정리하기

혼자 요리할 때는 재료와 도구가 섞여 있어도 괜찮지만, 식당 주방은 **재료 칸 · 도구 칸 · 조리대**가 나뉘어 있어야 여러 사람이 함께 일할 수 있어요. 서빙 프로젝트도 역할별로 파일을 나누면 고치기 쉽고, 테스트하기 쉽고, 다른 사람과 함께 일하기 좋아요.

서빙 프로젝트 구조

```
serving_project/
├── app/                    ← 서버 코드
│   ├── __init__.py
│   ├── config.py           ← 설정 (환경 변수)
│   ├── schemas.py          ← 입력·출력 모양 (Pydantic)
│   ├── model.py            ← 모델 불러오기·예측
│   ├── deps.py             ← 의존성 (모델 꺼내기)
│   ├── main.py             ← 앱 만들기 + lifespan + 라우터 등록
│   └── routers/
│       ├── health.py       ← /health
│       └── predict.py      ← /predict, /predict/batch
├── model/                  ← 학습된 모델 파일 (train.py 가 만듦)
├── tests/
│   └── test_api.py         ← 자동 테스트
├── train.py                ← 모델 학습·저장
├── pytest.ini              ← pytest 설정
├── requirements.txt        ← 필요한 패키지 목록
└── Dockerfile              ← 배포용 컨테이너 설명서
```

파일	바꾸게 되는 상황
config.py	모델 경로·이름·배치 크기 설정이 바뀔 때
schemas.py	모델 입력 항목이 늘거나 응답에 값을 추가할 때
model.py	**딥러닝 모델로 바꿀 때** (6장 8절의 PyTorch/Keras 클래스로 교체)
routers/	새 API(예: /predict/image)를 추가할 때
tests/	기능을 바꿀 때마다 함께

2. 설정 · 스키마 · 모델 필수

예제 파일 fastapi/07_테스트와_배포/serving_project/app/config.py

config.py

```python
# 설정: 환경 변수로 바꿀 수 있는 값들을 한 곳에
import os
from dataclasses import dataclass, field
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent          # serving_project 폴더


# frozen=True: 만든 뒤에는 값을 바꿀 수 없는 설정 객체
@dataclass(frozen=True)
class Settings:
    # default_factory: 설정 객체를 "만들 때마다" 환경 변수를 다시 읽어요
    app_name: str = field(default_factory=lambda: os.getenv("APP_NAME", "pass-predictor-api"))
    model_path: Path = field(default_factory=lambda: Path(os.getenv("MODEL_PATH", BASE_DIR / "model" / "pass_model.npz")))
    max_batch: int = field(default_factory=lambda: int(os.getenv("MAX_BATCH", "100")))
```

config.py

```
16
17
18 # 설정 객체를 돌려주는 함수 (의존성 · 테스트에서 바꿔 끼우기 쉬워요)
19 def get_settings() -> Settings:
20     return Settings()
```

한 줄씩 읽기

줄	코드	설명
6	`BASE_DIR = Path(__file__).resolve().parent.parent # serving_project 폴더`	이 파일 기준으로 프로젝트 폴더 찾기 (pandas 8장의 pathlib)
10	`@dataclass(frozen=True)`	바꿀 수 없는 설정 객체 (Python 11장 dataclass)
14	`model_path: Path = field(default_factory=lambda: Path(os.getenv("MODEL_PATH", BASE_DIR / "model" / "pass_model.npz")))`	★ 환경 변수가 있으면 그 값 (4장). `default_factory`로 **만들 때마다** 읽어요 (아래 주의 상자)
19	`def get_settings() -> Settings:`	설정을 꺼내는 함수 — 테스트에서 환경 변수를 바꾸면 새 설정이 만들어져요.

> **주의 | 테스트가 잡아낸 실수: 기본값은 import 할 때 한 번만 계산돼요**
>
> 처음에는 `model_path: Path = Path(os.getenv(...))`처럼 기본값 자리에 바로 `os.getenv`를 썼어요. 그런데 4절의 테스트 `test_missing_model_returns_503`이 **실패**했어요. 클래스의 기본값은 **파일을 import 하는 순간 한 번만** 계산되기 때문에, 그 뒤에 환경 변수를 바꿔도 반영되지 않았던 거예요.
>
> `field(default_factory=lambda: ...)`로 바꾸면 `Settings()`를 **만들 때마다** 함수가 실행되어 최신 환경 변수를 읽어요. 이렇게 사람이 놓치기 쉬운 실수를 찾아 주는 것이 자동 테스트의 힘이에요.

예제 파일 `fastapi/07_테스트와_배포/serving_project/app/schemas.py`

schemas.py

```
1  # 요청 · 응답의 모양(스키마)만 모아 둔 파일
2  from pydantic import BaseModel, Field
3
4
5  # 한 학생의 입력값 3개와 허용 범위
6  class StudentFeatures(BaseModel):
7      study_h: float = Field(ge=0, le=16, examples=[4.5])
8      sleep_h: float = Field(ge=0, le=16, examples=[7.0])
9      phone_h: float = Field(ge=0, le=24, examples=[2.0])
10
11
12 # 응답 하나
13 class Prediction(BaseModel):
14     pass_probability: float
15     label: str
16     model_version: str
17
18
19 # 여러 명 요청 · 응답
20 class BatchRequest(BaseModel):
21     students: list[StudentFeatures] = Field(min_length=1)
22
23
24 class BatchResponse(BaseModel):
25     count: int
```

schemas.py

```
    predictions: list[Prediction]
```

한 줄씩 읽기

줄	코드	설명
6	`class StudentFeatures(BaseModel):`	6장과 같은 입력·출력 스키마를 한 파일에

예제 파일 `fastapi/07_테스트와_배포/serving_project/app/model.py`

model.py

```
import json
from pathlib import Path
import numpy as np


# 저장된 모델 파일(.npz)을 읽어 예측하는 클래스
class PassModel:
    def __init__(self, path: Path):
        # 가중치 · 절편 · 표준화용 평균 · 표준편차 꺼내기
        data = np.load(path)
        self.w, self.b = data["w"], float(data["b"])
        self.mean, self.std = data["mean"], data["std"]
        # 설명서(model_card.json)가 있으면 버전 정보 읽기
        card = path.with_name("model_card.json")
        self.card = json.loads(card.read_text(encoding="utf-8")) if card.exists() else {}
        self.version = self.card.get("version", "unknown")

    # 학습 때와 똑같이 표준화한 뒤 합격 확률 계산
    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        Z = (X - self.mean) / self.std
        return 1 / (1 + np.exp(-(Z @ self.w + self.b)))
```

한 줄씩 읽기

줄	코드	설명
7	`class PassModel:`	모델 클래스. 딥러닝으로 바꿀 때 **이 파일만** 바꾸면 돼요.

예제 파일 `fastapi/07_테스트와_배포/serving_project/app/deps.py`

deps.py

```
from typing import Annotated
from fastapi import Depends, HTTPException, Request
from app.model import PassModel


# 의존성: 앱에 불러 둔 모델 꺼내기 (없으면 503 오류)
def get_model(request: Request) -> PassModel:
    model = getattr(request.app.state, "model", None)
    if model is None:
        raise HTTPException(503, "모델이 준비되지 않았어요")
    return model


# API 함수에서 model: ModelDep 로 쓰면 이 의존성이 실행돼요
ModelDep = Annotated[PassModel, Depends(get_model)]
```

한 줄씩 읽기

줄	코드	설명
8	`model = getattr(request.app.state, "model", None)`	`getattr(객체, 이름, 기본값)`: 없으면 None
15	`ModelDep = Annotated[PassModel, Depends(get_model)]`	라우터들이 함께 쓰는 의존성 (4장)

3. 라우터와 메인 앱 필수

예제 파일 `fastapi/07_테스트와_배포/serving_project/app/routers/predict.py`

predict.py

```python
import numpy as np
from fastapi import APIRouter, HTTPException, Request

from app.deps import ModelDep
from app.schemas import BatchRequest, BatchResponse, Prediction, StudentFeatures

# 예측 주소들: 모두 /predict 로 시작
router = APIRouter(prefix="/predict", tags=["predict"])


# 입력 목록 → numpy 배열 (학습 때와 같은 열 순서)
def _to_array(items: list[StudentFeatures]) -> np.ndarray:
    return np.array([[s.study_h, s.sleep_h, s.phone_h] for s in items])


# 확률 → 응답 모양 (0.5 이상이면 합격)
def _pred(p: float, version: str) -> Prediction:
    return Prediction(pass_probability=round(float(p), 4), label="합격" if p >= 0.5 else "불합격",
model_version=version)


# POST /predict: 한 명
@router.post("", response_model=Prediction)
def predict(student: StudentFeatures, model: ModelDep):
    return _pred(model.predict_proba(_to_array([student]))[0], model.version)


# POST /predict/batch: 여러 명. 설정의 max_batch 보다 많으면 413 오류
@router.post("/batch", response_model=BatchResponse)
def predict_batch(req: BatchRequest, model: ModelDep, request: Request):
    limit = request.app.state.settings.max_batch
    if len(req.students) > limit:
        raise HTTPException(413, f"한 번에 최대 {limit}명까지 예측할 수 있어요")
    probs = model.predict_proba(_to_array(req.students))
    return BatchResponse(count=len(probs), predictions=[_pred(p, model.version) for p in probs])
```

한 줄씩 읽기

줄	코드	설명
8	`router = APIRouter(prefix="/predict", tags=["predict"])`	3장의 APIRouter
4	`from app.deps import ModelDep`	패키지 경로로 가져오기 (`app.deps`)
30	`limit = request.app.state.settings.max_batch`	설정값으로 배치 크기 제한
32	`raise HTTPException(413, f"한 번에 최대 {limit}명까지 예측할 수 있어요")`	너무 큰 요청은 413

예제 파일 `fastapi/07_테스트와_배포/serving_project/app/main.py`

main.py

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI

from app.config import get_settings
from app.model import PassModel
from app.routers import health, predict


# 서버 시작: 설정 읽기 → 모델 파일이 있으면 불러오기 / 종료: 모델 해제
@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = get_settings()
    app.state.settings = settings
    app.state.model = PassModel(settings.model_path) if settings.model_path.exists() else None
    yield
    app.state.model = None


def create_app() -> FastAPI:                        # 앱을 만드는 함수 (테스트에서 새로 만들기 쉬움)
    app = FastAPI(title=get_settings().app_name, version="1.0.0", lifespan=lifespan)
    # 앱에 라우터 두 개 연결
    app.include_router(health.router)
    app.include_router(predict.router)
    return app


# uvicorn app.main:app 으로 실행할 때 쓰는 앱
app = create_app()
```

한 줄씩 읽기

줄	코드	설명
14	`app.state.model = PassModel(settings.model_path) if settings.model_path.exists() else None`	모델 파일이 있을 때만 불러오기 (없으면 503, 6장)
19	`def create_app() -> FastAPI: # 앱을 만드는 함수 (테스트에서 새로 만들기 쉬움)`	★ **앱 공장 함수**: 테스트마다 깨끗한 새 앱을 만들 수 있어요.
28	`app = create_app()`	uvicorn이 가져갈 `app` 객체 (`uvicorn app.main:app`)

예제 파일 `fastapi/07_테스트와_배포/serving_project/train.py`

먼저 모델을 학습해 `model/`에 저장해요. (6장 `ex00_train_model.py`와 같은 방법을 짧게 정리했어요.)

train.py

```python
# 모델 학습 → model/ 폴더에 저장 (06장 ex00 과 같은 방법)
import json
from pathlib import Path
import numpy as np

OUT = Path(__file__).parent / "model"
OUT.mkdir(exist_ok=True)
# 학생 400명 데이터 만들기 (06장과 같은 규칙)
rng = np.random.default_rng(seed=2026)
n = 400
study = rng.uniform(0, 6, n)
sleep = 8.5 - study * 0.35 + rng.normal(0, 0.6, n)
phone = (5 - study * 0.5 + rng.normal(0, 1, n)).clip(0.5)
score = (50 + study * 7 - phone * 2 + rng.normal(0, 6, n)).clip(0, 100)
```

train.py

```
X, y = np.column_stack([study, sleep, phone]), (score >= 60).astype(float)

# 80% 학습 / 20% 평가로 나누고 표준화
idx = rng.permutation(n)
tr, te = idx[:320], idx[320:]
mean, std = X[tr].mean(axis=0), X[tr].std(axis=0)
Z = (X - mean) / std
# 경사하강법으로 로지스틱 회귀 학습
w, b = np.zeros(3), 0.0
for _ in range(3000):
    p = 1 / (1 + np.exp(-(Z[tr] @ w + b)))
    w -= 0.1 * Z[tr].T @ (p - y[tr]) / len(tr)
    b -= 0.1 * (p - y[tr]).mean()
acc = float(((1 / (1 + np.exp(-(Z[te] @ w + b))) >= 0.5) == y[te]).mean())

# 가중치 · 전처리 정보와 설명서 저장
np.savez(OUT / "pass_model.npz", w=w, b=b, mean=mean, std=std)
(OUT / "model_card.json").write_text(json.dumps(
    {"name": "pass-predictor", "version": "1.0.0", "test_accuracy": round(acc, 4)}, indent=2),
encoding="utf-8")
print(f"학습 완료: 정확도 {acc:.1%} → {sorted(p.name for p in OUT.iterdir())}")
```

한 줄씩 읽기

줄	코드	설명
31	`np.savez(OUT / "pass_model.npz", w=w, b=b, mean=mean, std=std)`	가중치 + 전처리 정보 저장

▶ 실행 결과

```
학습 완료: 정확도 91.2% → ['model_card.json', 'pass_model.npz']
```

4. pytest 로 자동 테스트 필수

비유로 이해하기 | 자동차 출고 검사

자동차는 공장에서 나가기 전에 브레이크, 전조등, 경적을 **매번 같은 순서로** 검사해요. 코드를 고칠 때마다 손으로 `/docs`에서 하나하나 눌러 보는 대신, **테스트 코드**가 자동으로 확인하게 만들 수 있어요. 모델을 새 버전으로 바꿨을 때 API가 깨지지 않았는지 1초 만에 알 수 있어요.

pytest 규칙	설명
파일 이름	`test_*.py`
함수 이름	`test_`로 시작
검사	`assert 조건` — 조건이 False면 그 테스트 실패
fixture	`@pytest.fixture`: 테스트에 필요한 준비물(클라이언트)을 만들어 매개변수로 넣어 줌
parametrize	`@pytest.mark.parametrize`: 같은 테스트를 여러 값으로 반복
monkeypatch	테스트하는 동안만 환경 변수·속성을 바꿨다가 자동으로 되돌림

예제 파일 `fastapi/07_테스트와_배포/serving_project/tests/test_api.py`

테스트 파일이에요. 실행은 아래 `ex01_run_tests.py`로 해요.

test_api.py

```python
import pytest
from fastapi.testclient import TestClient

from app.main import create_app

# 시험용 입력: 합격할 학생(GOOD), 불합격할 학생(BAD)
GOOD = {"study_h": 5, "sleep_h": 6.5, "phone_h": 1}
BAD = {"study_h": 0.5, "sleep_h": 8.5, "phone_h": 6}


@pytest.fixture
def client():                                    # 테스트마다 새 앱 + lifespan 실행
    with TestClient(create_app()) as c:
        yield c


# 테스트 함수 이름은 test_ 로 시작 / assert 뒤의 조건이 거짓이면 테스트 실패
def test_health(client):
    res = client.get("/health")
    assert res.status_code == 200
    assert res.json() == {"status": "ok", "model_loaded": True}


def test_predict_good_student(client):
    body = client.post("/predict", json=GOOD).json()
    assert body["label"] == "합격"
    assert 0.5 <= body["pass_probability"] <= 1
    assert body["model_version"] == "1.0.0"


def test_predict_bad_student(client):
    assert client.post("/predict", json=BAD).json()["label"] == "불합격"


@pytest.mark.parametrize("field, value", [("study_h", -1), ("sleep_h", 30), ("phone_h", "많이")],
                         ids=["negative", "too_big", "not_number"])
def test_invalid_input_returns_422(client, field, value):     # 같은 테스트를 값 3개로
    res = client.post("/predict", json={**GOOD, field: value})
    assert res.status_code == 422


def test_batch(client):
    res = client.post("/predict/batch", json={"students": [GOOD, BAD, GOOD]})
    assert res.status_code == 200
    assert [p["label"] for p in res.json()["predictions"]] == ["합격", "불합격", "합격"]


# 한 번에 최대 2명으로 설정을 바꿔 413 이 나는지 시험
def test_batch_limit(client, monkeypatch):
    monkeypatch.setattr(client.app.state, "settings", type("S", (), {"max_batch": 2})())
    res = client.post("/predict/batch", json={"students": [GOOD] * 3})
    assert res.status_code == 413


# 모델 파일이 없을 때 503 이 나는지 시험
def test_missing_model_returns_503(monkeypatch):
    monkeypatch.setenv("MODEL_PATH", "no/such/model.npz")       # 환경 변수를 테스트 동안만 바꾸기
    with TestClient(create_app()) as c:
```

test_api.py

```
        assert c.get("/health").json()["model_loaded"] is False
        assert c.post("/predict", json=GOOD).status_code == 503
```

한 줄씩 읽기

줄	코드	설명
12	`def client(): # 테스트마다 새 앱 + lifespan 실행`	★ fixture: `with TestClient(...)`로 lifespan까지 실행한 클라이언트를 테스트에 넣어 줘요.
21	`assert res.json() == {"status": "ok", "model_loaded": True}`	응답이 기대와 정확히 같은지
27	`assert 0.5 <= body["pass_probability"] <= 1`	확률 범위 검사
35	`@pytest.mark.parametrize("field, value", [("study_h", -1), ("sleep_h", 30), ("phone_h", "많이")],`	잘못된 입력 3가지를 한 함수로 테스트 → 테스트 3개. `ids`는 결과에 보일 짧은 이름
49	`def test_batch_limit(client, monkeypatch):`	설정의 max_batch를 2로 바꿔서 3명 요청 → 413
57	`monkeypatch.setenv("MODEL_PATH", "no/such/model.npz") # 환경 변수를 테스트 동안만 바꾸기`	모델이 없는 상황 테스트 → 503

예제 파일 `fastapi/07_테스트와_배포/ex01_run_tests.py`

ex01_run_tests.py

```
# serving_project 의 테스트를 실행하고 결과를 보여 주는 파일
# (터미널에서  cd serving_project  →  pytest -v  와 같아요)
import os
import sys
from pathlib import Path
import pytest

# 테스트는 serving_project 폴더 기준으로 실행해야 해요
project = Path(__file__).parent / "serving_project"
os.chdir(project)
sys.path.insert(0, str(project))
# pytest.main(): 파이썬 코드 안에서 pytest 실행 (-v: 테스트마다 결과 표시)
code = pytest.main(["-v", "-p", "no:cacheprovider", "--no-header", "-W", "ignore"])
print("종료 코드:", int(code), "(0 이면 모두 통과)")
```

한 줄씩 읽기

줄	코드	설명
10	`os.chdir(project)`	pytest를 프로젝트 폴더에서 실행
13	`code = pytest.main(["-v", "-p", "no:cacheprovider", "--no-header", "-W", "ignore"])`	`-v`: 테스트 이름을 하나씩 자세히. 터미널에서 `pytest -v`와 같아요.

▶ 실행 결과

```
========================================= test session starts ==========================================
collecting ... collected 9 items

tests/test_api.py::test_health PASSED                                                          [ 11%]
tests/test_api.py::test_predict_good_student PASSED                                            [ 22%]
tests/test_api.py::test_predict_bad_student PASSED                                             [ 33%]
tests/test_api.py::test_invalid_input_returns_422[negative] PASSED                             [ 44%]
tests/test_api.py::test_invalid_input_returns_422[too_big] PASSED                              [ 55%]
tests/test_api.py::test_invalid_input_returns_422[not_number] PASSED                           [ 66%]
tests/test_api.py::test_batch PASSED                                                           [ 77%]
tests/test_api.py::test_batch_limit PASSED                                                     [ 88%]
```

▶ 실행 결과

```
tests/test_api.py::test_missing_model_returns_503 PASSED                [100%]

========================================= 9 passed in 0.47s =========================================
종료 코드: 0 (0 이면 모두 통과)
```

9개 테스트(parametrize로 3개가 늘어남)가 모두 `PASSED`예요. 맨 아래 걸린 시간은 컴퓨터마다 달라요. 코드를 고친 뒤 하나라도 `FAILED`가 나오면, 무엇이 기대와 달랐는지 pytest가 자세히 보여 줘요.

터미널에서 직접 테스트하기

```
cd fastapi/07_테스트와_배포/serving_project
pip install -r requirements.txt
python train.py          # 모델 만들기 (처음 한 번)
pytest -v                # 테스트 실행
```

> **도움말 | 테스트를 습관으로**
> 새 기능(예: `/predict/image`)을 만들면 테스트도 함께 추가하세요. GitHub Actions 같은 CI 도구를 쓰면 코드를 올릴 때마다 `pytest`가 자동으로 실행돼요.

5. 실행과 패키지 정리 필수

예제 파일 `fastapi/07_테스트와_배포/serving_project/requirements.txt`
`pip install -r requirements.txt` 한 줄로 다른 컴퓨터에도 같은 환경을 만들어요.

requirements.txt

```
fastapi>=0.115
uvicorn[standard]>=0.30
numpy>=2.0
pydantic>=2.7
# 테스트용
pytest>=8.0
httpx2
```

한 줄씩 읽기

줄	코드	설명
1	`fastapi>=0.115`	패키지>=버전: 이 버전 이상
2	`uvicorn[standard]>=0.30`	`[standard]`: 더 빠른 부가 기능 포함 설치

실행 방법	명령	언제
개발	`uvicorn app.main:app --reload`	코드를 고치며 확인할 때
다른 기기에서 접속	`uvicorn app.main:app --host 0.0.0.0 --port 8000`	같은 네트워크의 휴대폰·다른 PC에서
여러 프로세스	`uvicorn app.main:app --workers 4`	요청이 많을 때 (CPU 코어 수만큼). 모델은 워커마다 따로 불러와져 메모리가 그만큼 더 필요해요.
환경변수와 함께	`MODEL_PATH=model/v2.npz uvicorn app.main:app`	새 모델 버전으로 바꿀 때 (Windows는 `set MODEL_PATH=...`)

▶ 실제로 실행한 터미널 화면 (프로젝트 폴더에서 서버 실행 → curl 요청)

```
[터미널 1] $ MAX_BATCH=2 uvicorn app.main:app      (Windows 명령 프롬프트: set MAX_BATCH=2 후 uvicorn
app.main:app)
INFO:     Started server process [7064]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:10160 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:10161 - "POST /predict/batch HTTP/1.1" 200 OK
INFO:     127.0.0.1:10162 - "POST /predict/batch HTTP/1.1" 413 Content Too Large

[터미널 2]
$ curl http://127.0.0.1:8000/health
{"status":"ok","model_loaded":true}
$ curl -X POST http://127.0.0.1:8000/predict/batch -H "Content-Type: application/json" -d '{"students":
[{"study_h": 1, "sleep_h": 8, "phone_h": 5}, {"study_h": 5, "sleep_h": 7, "phone_h": 1}]}'
{"count":2,"predictions":[{"pass_probability":0.0144,"label":"불합격","model_version":"1.0.0"},{"pass_probabilit
y":0.9997,"label":"합격","model_version":"1.0.0"}]}
$ curl -X POST http://127.0.0.1:8000/predict/batch -H "Content-Type: application/json" -d '{"students":
[{"study_h": 1, "sleep_h": 8, "phone_h": 5}, {"study_h": 5, "sleep_h": 7, "phone_h": 1}, {"study_h": 3,
"sleep_h": 7, "phone_h": 2}]}'
{"detail":"한 번에 최대 2명까지 예측할 수 있어요"}
```

주의 | requirements.txt 의 httpx2

TestClient가 쓰는 패키지예요. 이 교안을 만들 때의 FastAPI 버전은 `httpx2`를 권장했어요. 설치한 FastAPI 버전에 따라 `httpx`를 요구할 수도 있으니, 오류 메시지가 알려 주는 패키지를 설치하세요.

6. Docker 로 배포 (참고) 심화

심화 · 선택 | 심화 · Docker 는 참고로 읽어요

Docker 는 별도 프로그램 설치가 필요해요. 이 절은 "이런 방법으로 서버를 포장해 배포한다" 는 흐름만 읽고, 직접 해 보는 것은 선택이에요.

비유로 이해하기 | 이삿짐 컨테이너

"내 컴퓨터에서는 되는데 서버에서는 안 돼요"는 아주 흔한 문제예요. 파이썬 버전, 패키지 버전, 운영체제가 달라서죠. **Docker**는 코드 + 파이썬 + 패키지 + 모델을 통째로 **컨테이너**에 담아, 어느 컴퓨터에서나 똑같이 실행되게 해 줘요. 클라우드(AWS, GCP, Azure) 배포의 표준 방법이에요.

주의 | 이 절의 Docker 명령은 실행하지 않은 참고 자료예요

이 교안을 만들 때 Docker로 이미지를 빌드·실행해 보지는 않았어요. Docker Desktop을 설치한 뒤 아래 순서대로 시도해 보세요.

예제 파일 `fastapi/07_테스트와_배포/serving_project/Dockerfile`

Dockerfile

```
# 1) 파이썬이 설치된 가벼운 기본 이미지
FROM python:3.12-slim

# 2) 작업 폴더
WORKDIR /app

# 3) 패키지 먼저 설치 (코드가 바뀌어도 이 단계는 캐시로 재사용돼 빨라요)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

Dockerfile

```
10
11 # 4) 코드와 모델 복사
12 COPY app ./app
13 COPY model ./model
14
15 # 5) 서버 실행 (컨테이너 밖에서 접속하려면 0.0.0.0)
16 EXPOSE 8000
17 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

한 줄씩 읽기

줄	코드	설명
2	`FROM python:3.12-slim`	파이썬이 설치된 가벼운 리눅스 이미지에서 시작
9	`RUN pip install --no-cache-dir -r requirements.txt`	패키지 설치
13	`COPY model ./model`	★ 학습된 **모델 파일도 함께** 담아요 (또는 실행할 때 외부 저장소에서 받기)
17	`CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]`	컨테이너가 시작되면 실행할 명령. 컨테이너 밖에서 접속하려면 `0.0.0.0`

Docker 명령 (참고 · 미실행)

```
1 cd fastapi/07_테스트와_배포/serving_project
2 python train.py                                  # model/ 폴더 준비
3 docker build -t pass-api:1.0 .                   # 이미지 만들기
4 docker run -p 8000:8000 pass-api:1.0             # 실행 → http://127.0.0.1:8000/docs
5 docker run -p 8000:8000 -e MAX_BATCH=10 pass-api:1.0   # 환경 변수 바꿔 실행
```

7. 다음 단계와 정리

더 배우면 좋은 것	설명
딥러닝 모델로 교체	6장 8절의 PyTorch/Keras 클래스로 `app/model.py` 교체
인증	API 키(4장) → OAuth2/JWT
기록과 모니터링	예측 입력·결과·처리 시간 로그, Prometheus·Grafana
비동기 작업 큐	오래 걸리는 예측은 Celery·RQ로 백그라운드 처리 후 결과 조회
전문 서빙 도구	TorchServe, NVIDIA Triton, BentoML, ONNX Runtime
클라우드 배포	Docker 이미지 → AWS ECS/App Runner, GCP Cloud Run, Azure Container Apps

개념	핵심
구조	`config` / `schemas` / `model` / `deps` / `routers` / `main`
앱 공장	`create_app()` — 테스트마다 새 앱
테스트	`test_*.py`, `assert`, `@pytest.fixture`, `parametrize`, `monkeypatch`
패키지	`requirements.txt` → `pip install -r`
실행	`uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers N`
배포	`Dockerfile` → `docker build` → `docker run -p 8000:8000`

> **알아 두기 | FastAPI 과정을 마치며**
> API 기초(1장) → 데이터 검사(2장) → CRUD(3장) → 서버 구조(4장) → 이미지(5장) → 모델 서빙(6장) → 테스트·배포(7장)까지 마쳤어요. 이제 딥러닝 학습을 마친 모델을 `app/model.py`에 연결하면, 여러분의 모델을 누구나 쓸 수 있는 **서비스**로 만들 수 있어요. 수고 많았어요!

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요. 정답 코드는 모두 실제로 실행해 확인한 코드예요.

문제 1. `tests/test_api.py`에 "`/predict` 응답에 `pass_probability`, `label`, `model_version` 세 키가 **모두** 있는지" 검사하는 테스트 함수를 추가한다고 생각하고, 같은 내용을 아래처럼 TestClient로 직접 확인해 보세요.

▶ 이렇게 출력되면 성공

```
키 확인: True
['label', 'model_version', 'pass_probability']
```

문제 2. `requirements.txt`를 읽어서 **주석(#)과 빈 줄을 뺀** 패키지 이름만 출력하세요. (`>=` 앞부분, `[standard]` 제외)

▶ 이렇게 출력되면 성공

```
['fastapi', 'uvicorn', 'numpy', 'pydantic', 'pytest', 'httpx2']
```

연습문제 정답 및 해설

문제 1 정답

직접 입력해 보세요

```python
import sys
sys.path.insert(0, "serving_project")            # 프로젝트 폴더를 import 경로에
import os
os.environ["MODEL_PATH"] = "serving_project/model/pass_model.npz"

from fastapi.testclient import TestClient
from app.main import create_app

with TestClient(create_app()) as client:
    body = client.post("/predict", json={"study_h": 3, "sleep_h": 7, "phone_h": 2}).json()
    print("키 확인:", {"pass_probability", "label", "model_version"} <= set(body))
    print(sorted(body))
```

▶ 실행 결과

```
키 확인: True
['label', 'model_version', 'pass_probability']
```

`집합A <= 집합B`는 "A가 B에 모두 들어 있는가"(부분집합)예요 (Python 7장). 테스트 파일에서는 `assert {...} <= set(body)`로 쓰면 돼요.

문제 2 정답

직접 입력해 보세요

```python
import re
from pathlib import Path

names = []
for line in Path("serving_project/requirements.txt").read_text(encoding="utf-8").splitlines():
    line = line.strip()
    if not line or line.startswith("#"):
        continue
    names.append(re.split(r"[\[>=<]", line)[0])
print(names)
```

▶ 실행 결과

```
['fastapi', 'uvicorn', 'numpy', 'pydantic', 'pytest', 'httpx2']
```

`re.split(r"[\[>=<]", 문자열)`: `[`, `>`, `=`, `<` 중 하나를 만나면 나누기 — 첫 조각이 패키지 이름이에요.

부록

부록 A. 용어 사전

용어	뜻	처음 나오는 곳
클라이언트 / 서버	요청을 보내는 쪽 / 요청을 받아 응답하는 쪽	0장 1절
API	프로그램끼리 정해진 방식으로 데이터를 주고받는 창구	0장 1절
HTTP	웹에서 요청·응답을 주고받는 약속(규칙)	0장 3절
URL	요청을 보낼 주소. 호스트 + 포트 + 경로 + 쿼리	0장 2절
localhost / 127.0.0.1	내 컴퓨터 자신을 가리키는 주소	0장 2절
포트	한 컴퓨터 안에서 서버 프로그램을 구분하는 번호 (기본 8000)	0장 2절
JSON	데이터를 주고받는 글자 형식. 파이썬 딕셔너리와 비슷	0장 4절
가상환경 (venv)	프로젝트 전용 패키지 상자	0장 6절
uvicorn	FastAPI 앱을 실제 웹 서버로 실행해 주는 프로그램	0장 7절
엔드포인트	API 주소 하나하나 (`/hello`, `/predict` …)	1장 2절
데코레이터 `@app.get`	"이 주소로 요청이 오면 아래 함수를 실행" 이라고 등록하는 표시	1장 3절
`/docs` (Swagger UI)	FastAPI 가 자동으로 만들어 주는 시험용 API 문서 화면	1장 4절
TestClient	서버를 켜지 않고 코드로 요청을 보내는 가짜 브라우저	1장 5절
경로 매개변수	주소 안의 값 `/items/{item_id}`	1장 6절
쿼리 매개변수	주소 뒤 `?` 의 값 `?skip=0&limit=10`	1장 6절
요청 본문 (body)	POST·PUT 요청에 담아 보내는 데이터(JSON)	2장 2절
Pydantic / BaseModel	데이터의 모양과 규칙을 정하고 자동 검사하는 도구	2장 1절
422	보낸 값의 형식·범위가 규칙과 달라요 (자동 검사 실패)	1장 6절
response_model	응답의 모양을 정하고 필요 없는 값을 숨기기	2장 4절
CRUD	만들기 · 읽기 · 수정 · 삭제	3장 1절
HTTPException	404 같은 오류 응답을 직접 보내는 방법	3장 3절
PUT / PATCH	전체 수정 / 보낸 값만 일부 수정	3장 4절
APIRouter	API 를 여러 파일로 나누는 도구	3장 5절
fetch	자바스크립트에서 HTTP 요청을 보내는 함수	3장 6절
의존성 (Depends)	여러 API 가 함께 쓰는 준비 작업을 함수로 만들어 넣어 주는 방법	4장 1절
헤더 (Header)	요청·응답에 붙는 부가 정보 (API 키, 내용 종류 …)	4장 2절
async / await	기다리는 동안 다른 요청을 처리하는 비동기 방식	4장 3절
lifespan	서버가 켜질 때·꺼질 때 할 일 (모델 불러오기)	4장 4절
미들웨어	모든 요청·응답이 지나가는 관문	4장 5절
CORS	다른 주소의 웹 페이지가 API 를 부를 수 있게 허용하는 설정	4장 5절
환경 변수	코드를 고치지 않고 바꾸는 설정 값	4장 5절
UploadFile	업로드된 파일을 받는 형식	5장 3절
content-type	데이터의 종류 (`image/png`, `application/json` …)	5장 3절
모델 서빙	학습한 모델을 API 로 제공하는 것	6장 1절

용어	뜻	처음 나오는 곳
전처리	모델에 넣기 전에 입력을 학습 때와 같은 모양으로 바꾸는 일	5장 5절
pytest	테스트 함수를 자동으로 찾아 실행하는 도구	7장 4절
requirements.txt	프로젝트에 필요한 패키지 목록	7장 5절
Docker	프로그램과 실행 환경을 통째로 포장해 어디서나 똑같이 실행하는 도구	7장 6절

부록 B. 명령어와 코드 모음

하고 싶은 일	명령 · 코드
가상환경 만들기 / 켜기	`python -m venv .venv` / `.venv\Scripts\activate` (Windows)
패키지 설치	`pip install fastapi uvicorn python-multipart pillow numpy httpx2 pytest`
서버 켜기	`uvicorn 파일이름:app --reload` (예제 폴더에서)
다른 포트로 켜기	`uvicorn 파일이름:app --reload --port 8001`
서버 끄기	서버 터미널에서 `Ctrl + C`
API 문서	`http://127.0.0.1:8000/docs`
앱 만들기	`app = FastAPI()`
GET API	`@app.get("/주소")` + `def 함수(): return {...}`
경로 / 쿼리 값 받기	`@app.get("/items/{item_id}")` + `def f(item_id: int, limit: int = 10)`
본문(JSON) 받기	`class Item(BaseModel): ...` + `@app.post("/items")` + `def f(item: Item)`
값의 범위 정하기	`price: int = Field(ge=0, le=100000)`
오류 응답	`raise HTTPException(status_code=404, detail="없어요")`
코드로 요청 보내기	`client = TestClient(app)` → `client.get("/주소")`, `client.post("/주소", json={...})`
응답 확인	`res.status_code`, `res.json()`
테스트 실행	`pytest -v` (프로젝트 폴더에서)

부록 C. 오류 해결 모음

각 장의 "자주 하는 실수" 표를 한곳에 모았어요. 오류 메시지의 일부로 찾아보세요.

장	증상	원인	해결
0장	`'python' 은(는) 내부 또는 외부 명령…이 아닙니다`	파이썬이 설치되지 않았거나 PATH에 없음	설치할 때 **Add python.exe to PATH** 체크, 또는 `py` 명령 사용
0장	`No module named 'fastapi'`	가상환경이 꺼진 터미널에서 실행	줄 앞에 `(.venv)` 가 있는지 확인 → 켜고 다시 실행
0장	`uvicorn` 을 찾을 수 없음	설치 안 됨 / 가상환경 꺼짐	`pip install uvicorn` 또는 `python -m uvicorn 파일:app`
0장	`Could not import module "ex03_first_server"`	다른 폴더에서 서버를 켬	`cd` 로 예제 폴더에 들어가서 실행
0장	`address already in use` / `10048`	8000번 방에 이미 다른 서버가 있음	켜 둔 서버를 `Ctrl+C` 로 끄기, 또는 `--port 8001`
0장	브라우저에 "사이트에 연결할 수 없음"	서버가 꺼져 있음	터미널 1 에 `Uvicorn running…` 이 보이는지 확인

장	증상	원인	해결
1장	`Error loading ASGI app. Could not import module`	파일 이름이 틀렸거나 다른 폴더에서 실행	예제 폴더로 `cd`, `파일명:app` 확인
1장	`Attribute "app" not found`	`:` 뒤 이름이 앱 변수와 다름	`app = FastAPI()` 변수 이름 확인
1장	`[Errno 10048] ... address already in use`	8000번 포트를 이미 사용 중	다른 서버 끄기 또는 `--port 8001`
1장	코드를 바꿨는데 결과가 그대로	`--reload` 없이 실행	`--reload` 또는 서버 재시작
1장	`ModuleNotFoundError: No module named 'httpx'`	TestClient 준비 안 됨	`pip install httpx2`
1장	422 Unprocessable Entity	입력 형식이 힌트와 다름	응답의 `detail`에서 어느 값이 틀렸는지 확인
2장	GET 요청에 본문 모델 사용	브라우저·문서에서 보내기 어려움	본문은 POST/PUT/PATCH로
2장	`client.post(..., data=...)`로 JSON 전송	422	`json=`
2장	`Field(gt=0)`를 기본값 자리에 안 씀	검사 안 됨	`x: int = Field(gt=0)`
2장	응답에 비밀번호·내부 값이 나감	정보 유출	`response_model`로 출력 모델 지정
2장	validator에 `@classmethod` 빠뜨림	오류	`@field_validator` 아래 `@classmethod`
2장	Pydantic v1 문법 (`@validator`, `.dict()`)	경고 또는 오류	v2: `@field_validator`, `.model_dump()`
3장	`/todos/{id}` 보다 `/todos/done` 을 아래에 정의	"done"을 id로 해석해 422	고정 주소를 **먼저** 정의
3장	PATCH에서 `model_dump()`만 사용	안 보낸 값이 None으로 덮어씀	`exclude_unset=True`
3장	204 응답에 return 값	본문이 무시되거나 오류	204는 return 없이
3장	`global` 빠뜨리고 next_id 증가	`UnboundLocalError`	`global next_id` (Python 8장)
3장	routers 폴더 import 실패	`ModuleNotFoundError`	메인 파일이 있는 폴더에서 실행, `__init__.py` 확인
5장	python-multipart 미설치	`Form data requires "python-multipart"` 오류	`pip install python-multipart`
5장	`file.read()`에 await 빠뜨림	코루틴 객체가 나옴	`await file.read()` (async def 안에서)
5장	content_type만 믿고 처리	가짜 파일로 서버 오류	`Image.open`으로 내용 확인
5장	NumPy 값을 그대로 return	`TypeError: Object of type float32 is not JSON serializable`	`float()`, `.tolist()`
5장	학습과 다른 전처리	예측이 엉망	전처리 함수를 학습·서빙에서 공유
5장	클라이언트에서 `json=`으로 파일 전송	422	`files=`